==
LIFELONG AI — FULL SOURCE DUMP
Generated: Sun Jun 21 15:26:37 UTC 2026
Commit: 9dc4d191630e1ed2e0bf986ad30a4e5fd7b90fce
Branch: main
==

—— FOLDER STRUCTURE (source only) ——————————————————————
.env.local.example
README.md
app/admin/page.tsx
app/api/admin/route.ts
app/api/chat/route.ts
app/api/corrections/route.ts
app/api/embeddings/route.ts
app/api/entities/route.ts
app/api/export/route.ts
app/api/health/route.ts
app/api/identity-flags/route.ts
app/api/import/route.ts
app/api/ingest/route.ts
app/api/memory/route.ts
app/api/reminders/route.ts
app/api/review/route.ts
app/api/tasks/route.ts
app/chat/page.tsx
app/globals.css
app/layout.tsx
app/memory/page.tsx
app/page.tsx
app/tasks/page.tsx
app/thoughts/page.tsx
app/today/page.tsx
components/LayoutShell.tsx
components/TabBar.tsx
components/chat/ChatInput.tsx
components/chat/ChatMessages.tsx
components/tasks/TaskFilters.tsx
components/tasks/TaskList.tsx
components/thoughts/RecentList.tsx
components/thoughts/ThoughtInput.tsx
docs/LEARNINGS_AND_CHANGES.md
docs/architecture.html
lib/cognitive-load.ts
lib/context-assembly.ts
lib/embeddings.ts
lib/groq.ts
lib/kg-processor.ts
lib/retrieval.ts
lib/supabase/client.ts

lib/supabase/server.ts
lib/truth-arbitration.ts
lib/types.ts
next.config.js
package.json
public/manifest.json
scripts/dump-repo.sh
scripts/evaluate-core.mjs
scripts/generate-icons.mjs
scripts/generate-lifelong-stress.mjs
scripts/generate-synthetic.ts
scripts/verify-env.mjs
scripts/verify-sql.mjs
supabase/.temp/cli-latest
supabase/.temp/gotrue-version
supabase/.temp/linked-project.json
supabase/.temp/pooler-url
supabase/.temp/postgres-version
supabase/.temp/project-ref
supabase/.temp/rest-version
supabase/.temp/storage-migration
supabase/.temp/storage-version
supabase/cleanup-accidental-sql.sql
supabase/cleanup-wife-alias.sql
supabase/functions/extract-event/index.ts
supabase/migrations/0002_lifelong_memory_core.sql
supabase/migrations/0003_projection_idempotency.sql
supabase/migrations/0004_canonical_projection_keys.sql
supabase/migrations/0005_entity_and_review_cleanup.sql
supabase/migrations/20260621072130_fts_index.sql
supabase/reset-and-seed-lifelong-stress.sql
supabase/reset-blank-slate.sql
supabase/schema.sql
supabase/seed-lifelong-core.sql
supabase/seed-lifelong-stress.sql
supabase/seed-synthetic.sql
tailwind.config.ts
tsconfig.json

—— FILE CONTENTS

FILE: .env.local.example

NEXT_PUBLIC_SUPABASE_URL=https://your-project-ref.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-supabase-anon-key

GROQ_API_KEY=your-groq-api-key

FILE: README.md

Lifelong AI Personal Assistant (Second Brain)

Welcome to the **Lifelong AI Personal Assistant**. This application is designed to act as your ultimate "Second Brain" — a private, highly intelligent knowledge graph that learns about you, your life, your work, and your tasks simply by listening to your daily thoughts.

Unlike traditional note-taking apps where you have to manually organize folders, tag pages, and build databases, this application uses a **Semantic Knowledge Graph**. You simply type your thoughts naturally, and the AI automatically extracts the entities, maps their relationships, calculates vector embeddings for semantic search, and dynamically structures your tasks.

🏗️ Architecture & Data Flow

To understand how your Second Brain thinks, it helps to look at the data pipeline. Every time you type a thought, the system performs a multi-stage background extraction process.

```mermaid
graph TD
    A[User types in 'Thoughts' Tab] --> B(Frontend creates 'system_event')
    B --> C{Background Processor}

    subgraph AI Extraction Engine
        C --> D[Generate Vector Embeddings]
        D --> E[Semantic Chunk Table]
        C --> F[Groq LLM Llama-3.3-70B]
    end

    subgraph Knowledge Graph Database
        F --> G[(Canonical Entities)]
        F --> H[(Entity Aliases)]
        F --> I[(Entity Relations)]
        F --> J[(Memory Records)]
        F --> K[(Task Projections)]
    end

    G --> L[Chat Tab & Memory UI]
    H --> L
```

```
    I --> L
    J --> L
    K --> M[Tasks Tab Hierarchical UI]
```

The Knowledge Graph Tables Explained
The underlying database utilizes several interconnected PostgreSQL tables to represent your life dynamically:

1. **`system_event`**: The raw text of every thought or chat message you send. This is the source of truth.
2. **`semantic_chunk`**: The application uses the `xenova/all-MiniLM-L6-v2` model to turn your raw text into mathematical arrays (embeddings). This allows the AI to perform "Semantic Search" (e.g., finding memories about "automobiles" when you search for "car").
3. **`canonical_entity`**: The unique master record for a person, project, or organization.
4. **`entity_alias`**: Real-world people have many names. This table maps nicknames and honorifics (e.g., mapping "John Smith", "Johnny", and "Mr. Smith") back to the single `canonical_entity`.
5. **`entity_relation`**: Maps how entities connect to each other (e.g., `Person A` is `MARRIED_TO` `Person B`).
6. **`epistemic_claim`**: The system's internal confidence tracker. Every fact extracted is assigned a confidence score, ensuring the AI knows the difference between a hard fact and an inferred guess.
7. **`memory_record`**: Specific facts, dates, preferences, and goals extracted from your thoughts.
8. **`task_projection`**: Actionable chores and milestones automatically parsed from your text. Tasks automatically link to `canonical_entities` (Projects) if they are part of a larger initiative.

🧠 Core UI Features

1. **The Thoughts Tab (Ingestion)**
   - Type naturally. The background engine powered by Groq's insanely fast `llama-3.3-70b-versatile` model reads your sentences and structures them into the Knowledge Graph in real-time.

2. **The Memory Tab (Recall)**
   - View your extracted facts structurally. It organizes dates, life events, preferences, and goals with full transparency on where the AI sourced the information.

3. **The Tasks Tab (Action)**
   - The AI automatically distinguishes between massive "Projects" and actionable "Tasks".
   - Your Tasks tab automatically groups sub-tasks and milestones recursively under their respective parent projects in a beautiful, collapsible UI.

4. **The Chat Tab (Interaction)**
   - Chat naturally with your Second Brain. It queries the vector embeddings and the knowledge graph simultaneously to answer specific questions with total context of your life.

🚀 How to Deploy Your Own Second Brain (For Non-Coders)

You do not need to know how to code to deploy this application. Follow these exact steps to set up your own completely free, private instance.

Step 1: Set up your Database (Supabase)
Supabase is the secure PostgreSQL engine where your Knowledge Graph will be permanently stored.
1. Go to [Supabase](https://supabase.com/) and create a free account.
2. Click **"New Project"**, give it a name (e.g., "My Second Brain"), and create a secure database password. Wait a few minutes for the database to provision.
3. Once created, go to the **SQL Editor** tab on the left sidebar.
4. Open the `supabase/migrations` folder in this code repository. Copy the SQL setup scripts found there and paste them into the Supabase SQL Editor. Click **Run** to generate all the necessary tables.
5. Finally, go to **Project Settings -> API**. You will need to copy two strings later: the `Project URL` and the `anon public` key.

Step 2: Get your AI Brain (Groq)
Groq provides the incredible AI brain that powers the semantic extraction at lightning speeds.
1. Go to the [Groq Cloud Console](https://console.groq.com/) and sign up for a free account.
2. Navigate to **API Keys** and click **"Create API Key"**.
3. Copy this key somewhere safe. You will need it in the next step.

Step 3: Deploy the App (Vercel)
Vercel is the platform that will actually host the website interface you interact with.
1. Create a free account on [Vercel](https://vercel.com/) and link it to your GitHub account.
2. Click **"Add New Project"** and select this repository from your GitHub list.
3. Before you click Deploy, look for the **Environment Variables** section. You need to add three specific variables here so the website knows how to talk to your database and your AI:
   - **Name:** `NEXT_PUBLIC_SUPABASE_URL` | **Value:** (Paste the Supabase Project URL from Step 1)
   - **Name:** `NEXT_PUBLIC_SUPABASE_ANON_KEY` | **Value:** (Paste the Supabase anon public key from Step 1)
   - **Name:** `GROQ_API_KEY` | **Value:** (Paste the Groq API Key from Step 2)
4. Click **Deploy**. Vercel will now automatically build your app.

Step 4: Start Thinking!
Once Vercel finishes, it will give you a live URL. Click it!

Navigate to the **Thoughts** tab and start typing:
"My name is [Your Name]. I am working on a new project called..."

Your personal Second Brain is now online and learning!

🛠️ Developer Operations

- **Framework:** Next.js 14 (App Router)
- **Database:** Supabase (PostgreSQL)
- **AI/LLM:** Groq SDK utilizing `llama-3.3-70b-versatile` (for dense JSON extraction) and `llama-3.1-8b-instant` (for rapid chat routing).
- **Styling:** Tailwind CSS

Wiping Data / Exporting
- **Blank Slate:** If you ever need to completely wipe your personal data to start from a blank slate *without deleting the actual tables*, run the SQL script found in `scripts/blank_slate.sql` in your Supabase SQL editor.
- **Exporting:** If you wish to export your entire Knowledge Graph database to a JSON file for local analysis or backup, simply run `node export_db.js` from the root folder in your terminal.

FILE: app/admin/page.tsx

```
'use client';

import { useEffect, useState } from 'react';
import type {
  CognitiveLoad,
  MemoryReviewItem,
  OperationalHealth,
} from '@/lib/types';

interface AdminPayload {
  operational: OperationalHealth;
  load: CognitiveLoad | null;
  recent_failures: {
    id: string;
    event_id: string | null;
    extracted_at: string;
    error_message: string | null;
  }[];
}

interface IdentityFlagItem {
```

```
  id: string;
  similarity: number | null;
  flag_reason: string | null;
  entity_a?: { canonical_name: string; classification: string } | null;
  entity_b?: { canonical_name: string; classification: string } | null;
}

export default function AdminPage() {
  const [admin, setAdmin] = useState<AdminPayload | null>(null);
  const [reviews, setReviews] = useState<MemoryReviewItem[]>([]);
  const [flags, setFlags] = useState<IdentityFlagItem[]>([]);

  interface CanonicalEntityItem {
    id: string;
    canonical_name: string;
    entity_type: string;
    summary: string;
    created_at: string;
  }

  const [entities, setEntities] = useState<CanonicalEntityItem[]>([]);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    async function load() {
      setIsLoading(true);
      try {
        const [adminRes, reviewRes, flagsRes, entitiesRes] = await Promise.all([
          fetch('/api/admin'),
          fetch('/api/review'),
          fetch('/api/identity-flags'),
          fetch('/api/entities'),
        ]);

        if (adminRes.ok) setAdmin(await adminRes.json() as AdminPayload);
        if (reviewRes.ok) {
          const data = (await reviewRes.json()) as { items: MemoryReviewItem[] };
          setReviews(data.items);
        }
        if (flagsRes.ok) {
          const data = (await flagsRes.json()) as { flags: IdentityFlagItem[] };
          setFlags(data.flags);
        }
        if (entitiesRes.ok) {
          const data = (await entitiesRes.json()) as { entities: CanonicalEntityItem[] };
          setEntities(data.entities);
        }
      } finally {
        setIsLoading(false);
      }
```

```
  }

  void load();
}, []);

async function updateReview(id: string, status: 'APPROVED' | 'REJECTED') {
  await fetch('/api/review', {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ id, status }),
  });
  setReviews(prev => prev.filter(item => item.id !== id));
}

async function approveAllReviews() {
  await fetch('/api/review', {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ id: 'all', status: 'APPROVED' }),
  });
  setReviews([]);
}

async function updateFlag(id: string, resolution: 'MERGED' | 'KEPT_SEPARATE') {
  await fetch('/api/identity-flags', {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ id, resolution }),
  });
  setFlags(prev => prev.filter(item => item.id !== id));
}

async function deleteEntity(id: string) {
  if (!confirm('Are you sure you want to permanently delete this canonical entity?
This will orphan its memories.')) return;
  await fetch('/api/entities?id=' + id, { method: 'DELETE' });
  setEntities(prev => prev.filter(item => item.id !== id));
}

const ops = admin?.operational;

return (
  <div className="flex flex-col h-full">
    <div className="px-4 pt-5 pb-3 border-b border-[#1a2740]">
      <h1 className="text-[18px] font-semibold text-rose-300">
        Admin
      </h1>
      <p className="text-[12px] text-slate-600 font-mono mt-0.5">
        Operational health and review queues
      </p>
```

```
    </div>

    <div className="flex-1 overflow-y-auto px-4 py-4 space-y-5">
      {isLoading ? (
        <p className="text-[13px] font-mono text-slate-600 animate-pulse">
          Loading system state...
        </p>
      ) : (
        <>
          <section>
            <h2 className="text-[12px] font-mono text-rose-300 mb-2">
              HEALTH
            </h2>
            <div className="grid grid-cols-2 gap-2">
              <Metric label="Events" value={ops?.total_events ?? 0} hint="Raw thought/
chat/task events saved." />
              <Metric label="Memories" value={ops?.active_memories ?? 0}
hint="Durable facts and decisions available to chat." />
              <Metric label="Pending embeds" value={ops?.pending_embeddings ?? 0}
hint="Search vectors waiting to be generated. Retrieval still works from memory/
text." />
              <Metric label="Reviews" value={ops?.pending_memory_reviews ?? 0}
hint="Memory items needing your approval." />
              <Metric label="Identity flags" value={ops?.pending_identity_flags ?? 0}
hint="Possible duplicate people/projects needing review." />
              <Metric label="Due reminders" value={ops?.due_reminders ?? 0}
hint="Reminders due now or overdue." />
            </div>
            <a
              href="/api/export"
              className="mt-3 inline-flex rounded-md border border-[#1a2740] px-3
py-2 text-[12px] font-mono text-slate-400"
            >
              Download export
            </a>
          </section>

          <section>
            <div className="flex items-center justify-between mb-2">
              <h2 className="text-[12px] font-mono text-yellow-300">
                MEMORY REVIEWS
              </h2>
              {reviews.length > 0 && (
                <button
                  onClick={() => void approveAllReviews()}
                  className="text-[11px] font-mono text-emerald-400 border border-
emerald-500/40 px-2 py-1 rounded hover:bg-emerald-500/10 transition-colors"
                >
                  Approve All
                </button>
```

```
            )}
          </div>
          <Empty show={reviews.length === 0} text="No memory reviews pending." /
>
          {reviews.slice(0, 8).map(item => (
            <div key={item.id} className="rounded-lg border border-[#1a2740] bg-
[#0c1222] p-3 mb-2">
              <p className="text-[13px] text-slate-200">{item.reason}</p>
              {item.memory && (
                <p className="text-[12px] text-slate-500 mt-1">
                  {item.memory.title}: {item.memory.summary}
                </p>
              )}
              <p className="text-[10px] font-mono text-slate-700 mt-2">
                severity {item.severity}
              </p>
              <div className="flex gap-2 mt-3">
                <button
                  onClick={() => void updateReview(item.id, 'APPROVED')}
                  className="flex-1 rounded-md border border-emerald-500/50 px-3
py-2 text-[12px] font-mono text-emerald-300"
                >
                  Approve
                </button>
                <button
                  onClick={() => void updateReview(item.id, 'REJECTED')}
                  className="flex-1 rounded-md border border-red-500/50 px-3 py-2
text-[12px] font-mono text-red-300"
                >
                  Reject
                </button>
              </div>
            </div>
          ))}
        </section>

        <section>
          <h2 className="text-[12px] font-mono text-cyan-300 mb-2">
            IDENTITY FLAGS
          </h2>
          <Empty show={flags.length === 0} text="No identity flags pending." />
          {flags.slice(0, 8).map(flag => (
            <div key={flag.id} className="rounded-lg border border-[#1a2740] bg-
[#0c1222] p-3 mb-2">
              <div className="flex items-start justify-between gap-3">
                <div>
                  <p className="text-[13px] text-slate-200">
                    {flag.entity_a?.canonical_name ?? 'Unknown'} ↔
{flag.entity_b?.canonical_name ?? 'Unknown'}
                  </p>
```

```
                <p className="text-[11px] font-mono text-slate-600 mt-1">
                  similarity {Math.round(Number(flag.similarity ?? 0) * 100)}%
                </p>
                {flag.flag_reason && (
                  <p className="text-[12px] text-slate-500 mt-2">
                    {flag.flag_reason}
                  </p>
                )}
              </div>
            </div>
            <div className="flex gap-2 mt-3">
              <button
                onClick={() => void updateFlag(flag.id, 'MERGED')}
                className="flex-1 rounded-md border border-emerald-500/50 px-3
py-2 text-[12px] font-mono text-emerald-300"
              >
                Same
              </button>
              <button
                onClick={() => void updateFlag(flag.id, 'KEPT_SEPARATE')}
                className="flex-1 rounded-md border border-slate-500/50 px-3 py-2
text-[12px] font-mono text-slate-300"
              >
                Different
              </button>
            </div>
          </div>
        ))}
      </section>

      <section>
        <h2 className="text-[12px] font-mono text-red-300 mb-2">
          EXTRACTION FAILURES
        </h2>
        <Empty show={(admin?.recent_failures.length ?? 0) === 0} text="No recent
extraction failures." />
        {admin?.recent_failures.map(failure => (
          <div key={failure.id} className="rounded-lg border border-[#1a2740] bg-
[#0c1222] p-3 mb-2">
            <p className="text-[12px] text-red-300">
              {failure.error_message ?? 'Unknown extraction error'}
            </p>
            <p className="text-[10px] font-mono text-slate-700 mt-2">
              {new Date(failure.extracted_at).toLocaleString()}
            </p>
          </div>
        ))}
      </section>

      <section>
```

```
            <h2 className="text-[12px] font-mono text-fuchsia-300 mb-2">
              CANONICAL ENTITIES
            </h2>
            <Empty show={entities.length === 0} text="No canonical entities found." />
            <div className="grid grid-cols-1 gap-2">
              {entities.slice(0, 10).map(ent => (
                <div key={ent.id} className="rounded-lg border border-[#1a2740] bg-
[#0c1222] p-3 flex justify-between items-start">
                  <div>
                    <p className="text-[13px] text-slate-200">{ent.canonical_name}</p>
                    <p className="text-[11px] font-mono text-slate-600
mt-0.5">{ent.entity_type}</p>
                    {ent.summary && <p className="text-[12px] text-slate-500
mt-1">{ent.summary}</p>}
                  </div>
                  <button
                    onClick={() => void deleteEntity(ent.id)}
                    className="text-[11px] font-mono text-red-400 border border-
red-500/40 px-2 py-1 rounded hover:bg-red-500/10 transition-colors"
                  >
                    Delete
                  </button>
                </div>
              ))}
            </div>
          </section>
        </>
      )}
    </div>
  </div>
  );
}

function Metric({ label, value, hint }: { label: string; value: number; hint: string }) {
  return (
    <div className="rounded-lg border border-[#1a2740] bg-[#0c1222] p-3">
      <p className="text-[10px] font-mono text-slate-600">{label}</p>
      <p className="text-[20px] font-semibold text-slate-200 mt-1">{value}</p>
      <p className="text-[10px] text-slate-700 mt-1 leading-snug">{hint}</p>
    </div>
  );
}

function Empty({ show, text }: { show: boolean; text: string }) {
  if (!show) return null;
  return (
    <p className="text-[13px] text-slate-600 font-mono border border-[#1a2740]
rounded-lg p-3">
      {text}
    </p>
```

```
  );
}
```

FILE: app/api/admin/route.ts

```
import { NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';
import { getCognitiveLoad } from '@/lib/cognitive-load';
import type { OperationalHealth } from '@/lib/types';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

export async function GET(): Promise<NextResponse> {
  try {
    const supabase = getSupabaseServer();

    const [healthResult, load, recentFailures] = await Promise.all([
      supabase.from('operational_health').select('*').single(),
      getCognitiveLoad().catch(() => null),
      supabase
        .from('extraction_log')
        .select('id, event_id, extracted_at, error_message')
        .eq('success', false)
        .order('extracted_at', { ascending: false })
        .limit(10),
    ]);

    if (healthResult.error) {
      console.error('[api/admin] operational_health error:', healthResult.error);
      return NextResponse.json({ error: 'Admin health query failed' }, { status: 503 });
    }

    return NextResponse.json({
      operational: healthResult.data as OperationalHealth,
      load,
      recent_failures: recentFailures.data ?? [],
    });

  } catch (err) {
    console.error('[api/admin] unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

FILE: app/api/chat/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';
import { streamChatResponse, computeHash } from '@/lib/groq';
import { assembleContext, buildMessages } from '@/lib/context-assembly';
import { processKnowledgeGraph } from '@/lib/kg-processor';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

// Max time for streaming response — Vercel hobby: 60s
export const maxDuration = 60;

// ─── POST /api/chat
// ──────────────────────────────────────────────────────────

export async function POST(req: NextRequest): Promise<NextResponse> {
  try {
    // ─── Parse input
    // ──────────────────────────────────────────────────────

    const body = await req.json().catch(() => null);

    if (!body || typeof body !== 'object') {
      return NextResponse.json(
        { error: 'Invalid JSON body' },
        { status: 400 }
      );
    }

    const message   = body.message   as string   | undefined;
    const embedding = body.embedding as number[] | undefined;

    if (!message || typeof message !== 'string' || message.trim().length === 0) {
      return NextResponse.json(
        { error: 'message is required' },
        { status: 400 }
      );
    }

    const sanitisedMessage = message
      .replace(/<[^>]*>/g, '')
      .replace(/[\u0000-\u001F]/g, ' ')
      .trim()
      .slice(0, 4000);
```

```
    // Validate embedding dimensions if provided
    const validEmbedding =
      Array.isArray(embedding) && embedding.length === 384
        ? embedding
        : undefined;

    const supabase = getSupabaseServer();

    // ─── Step 1: Save user message to event log
──────────────────────────────────────────────────────

    const { data: insertedEvent, error: insertError } = await supabase
      .from('system_event')
      .insert({
        source_tab: 'CHAT',
        payload:    sanitisedMessage,
        metadata:   { role: 'user' },
      })
      .select('id')
      .single();

    if (insertError) {
      console.error('[api/chat] Failed to save user message:', insertError);
      // Non-fatal — continue with chat
    }

    const eventId = insertedEvent?.id ?? 'temp_' + Date.now();

    // ─── Step 2: Assemble initial context
──────────────────────────────────────────────────────────────

    let manifest = await assembleContext(sanitisedMessage, validEmbedding);

    // ─── Step 3: Run intelligent graph extraction & mutation
──────────────────────────────

    try {
      await processKnowledgeGraph(supabase, eventId, sanitisedMessage, 'CHAT',
manifest);

      // Re-assemble context if the graph processor succeeded, so the LLM has the
latest state
      manifest = await assembleContext(sanitisedMessage, validEmbedding);
    } catch (err) {
      console.error('[api/chat] Graph processing failed:', err);
      // Non-fatal, proceed with original manifest
    }

    // ─── Step 4: Build message array for Groq
──────────────────────────────────────────────────────

    const messages = buildMessages(manifest, sanitisedMessage);

    // ─── Step 5: Check response cache
```

```
  const promptHash = computeHash(
    messages.map(m => m.role + ':' + m.content).join('|')
  );

  const { data: cached } = await supabase
    .from('response_cache')
    .select('response_text')
    .eq('prompt_hash', promptHash)
    .gt('expires_at', new Date().toISOString())
    .single();

  if (cached && cached.response_text) {
    // Cache hit — stream the cached response instantly
    void saveAssistantMessage(cached.response_text, manifest.activeTasks);

    const encoder    = new TextEncoder();
    const cachedText = cached.response_text;

    const stream = new ReadableStream<Uint8Array>({
      start(controller) {
        controller.enqueue(encoder.encode(cachedText));
        controller.close();
      },
    });

    return new NextResponse(stream, {
      status:  200,
      headers: {
        'Content-Type':  'text/plain; charset=utf-8',
        'X-Cache':       'HIT',
        'X-Load-State':  manifest.loadState,
      },
    });
  }

  // ─── Step 6: Stream from Groq ──────────────────────────────────────────

  const groqStream = await streamChatResponse(messages);

  // Accumulate full response for caching and event log
  let fullResponse = '';

  const transformStream = new TransformStream<Uint8Array, Uint8Array>({
    transform(chunk, controller) {
      const text = new TextDecoder().decode(chunk);
      fullResponse += text;
      controller.enqueue(chunk);
```

```
      },
      flush() {
        // After stream completes — save to cache and event log async
        void saveAssistantMessage(fullResponse, manifest.activeTasks);
        void cacheResponse(promptHash, fullResponse);
      },
    });

    const responseStream = groqStream.pipeThrough(transformStream);

    return new NextResponse(responseStream, {
      status:  200,
      headers: {
        'Content-Type':              'text/plain; charset=utf-8',
        'Transfer-Encoding':          'chunked',
        'X-Cache':                   'MISS',
        'X-Load-State':               manifest.loadState,
        'X-Override-Count':           String(manifest.overrides.length),
        'X-Context-Tokens':           String(manifest.tokensUsed),
      },
    });

  } catch (err) {
    console.error('[api/chat] Unexpected error:', err);
    return NextResponse.json(
      { error: 'Something went wrong. Please try again.' },
      { status: 500 }
    );
  }
}

// —— Helpers ——————————————————————————————————————————————————————————

async function saveAssistantMessage(
  content:     string,
  activeTasks?: { id: string; title: string }[]
): Promise<void> {
  if (!content || content.trim().length === 0) return;
  try {
    const supabase = getSupabaseServer();

    // FIX: detect which tasks this response discussed by title match.
    // Stored in metadata so context-assembly.ts can detect stale
    // references without depending on literal UUIDs in chat text.
    const referencedIds = (activeTasks ?? [])
      .filter(t => content.toLowerCase().includes(t.title.toLowerCase().slice(0, 20)))
      .map(t => t.id);
```

```
    await supabase.from('system_event').insert({
      source_tab: 'CHAT',
      payload:    content.trim().slice(0, 10000),
      metadata:   { role: 'assistant', referenced_task_ids: referencedIds },
    });
  } catch (err) {
    console.error('[api/chat] saveAssistantMessage error:', err);
  }
}

async function cacheResponse(
  promptHash:   string,
  responseText: string
): Promise<void> {
  if (!responseText || responseText.trim().length === 0) return;
  try {
    const supabase  = getSupabaseServer();
    const expiresAt = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000).toISOString();
    await supabase
      .from('response_cache')
      .upsert(
        {
          prompt_hash:   promptHash,
          response_text: responseText.trim().slice(0, 20000),
          expires_at:    expiresAt,
        },
        { onConflict: 'prompt_hash' }
      );
  } catch (err) {
    console.error('[api/chat] cacheResponse error:', err);
  }
}
```

FILE: app/api/corrections/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

const ACTIONS = ['CORRECT', 'MERGE', 'SPLIT', 'ARCHIVE', 'REJECT',
'REEXTRACT'] as const;

export async function POST(req: NextRequest): Promise<NextResponse> {
```

```
  try {
    const body = await req.json().catch(() => null);
    if (!body || typeof body !== 'object') {
      return NextResponse.json({ error: 'Invalid JSON body' }, { status: 400 });
    }

    const memoryId = body.memory_id as string | undefined;
    const sourceEventId = body.source_event_id as string | undefined;
    const action = body.action as (typeof ACTIONS)[number] | undefined;
    const correctionText = body.correction_text as string | undefined;

    if (!action || !ACTIONS.includes(action) || !correctionText?.trim()) {
      return NextResponse.json({ error: 'action and correction_text are required' },
{ status: 400 });
    }

    const supabase = getSupabaseServer();
    const { data, error } = await supabase
      .from('memory_correction')
      .insert({
        memory_id: memoryId ?? null,
        source_event_id: sourceEventId ?? null,
        action,
        correction_text: correctionText.trim().slice(0, 5000),
        metadata: { source: 'api/corrections' },
      })
      .select('*')
      .single();

    if (error) {
      console.error('[api/corrections] insert error:', error);
      return NextResponse.json({ error: 'Correction insert failed' }, { status: 503 });
    }

    if (memoryId && (action === 'ARCHIVE' || action === 'REJECT')) {
      await supabase
        .from('memory_record')
        .update({
          current_status: action === 'ARCHIVE' ? 'ARCHIVED' : 'REJECTED',
          updated_at: new Date().toISOString(),
        })
        .eq('id', memoryId);
    }

    return NextResponse.json({ correction: data }, { status: 201 });

  } catch (err) {
    console.error('[api/corrections] unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
```

```
}
```

FILE: app/api/embeddings/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';
import type { EmbeddingBatch, EmbeddingsResponse } from '@/lib/types';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

// Hardcoded here instead of importing from lib/embeddings.ts —
// that file dynamically imports @xenova/transformers, which contains
// a native binary meant for the browser only. Importing it anywhere
// in a server file breaks the Next.js server build.
const MODEL_VERSION = 'xenova/all-MiniLM-L6-v2@2.17.2';

export async function POST(req: NextRequest): Promise<NextResponse> {
  try {
    const body = await req.json().catch(() => null);

    if (!body || typeof body !== 'object') {
      return NextResponse.json({ error: 'Invalid JSON body' }, { status: 400 });
    }

    const chunks = body.chunks as EmbeddingBatch[] | undefined;

    if (!Array.isArray(chunks) || chunks.length === 0) {
      return NextResponse.json(
        { error: 'chunks must be a non-empty array' },
        { status: 400 }
      );
    }

    const validChunks = chunks.filter(c => {
      return (
        typeof c.event_id  === 'string' &&
        c.event_id.length   > 0         &&
        Array.isArray(c.embedding)       &&
        c.embedding.length === 384       &&
        c.embedding.every(v => typeof v === 'number' && isFinite(v))
      );
    });

    if (validChunks.length === 0) {
      return NextResponse.json(
```

```
        { error: 'No valid chunks. Each chunk needs event_id and 384-dim
embedding.' },
        { status: 400 }
      );
    }

    const supabase = getSupabaseServer();
    let   updated   = 0;

    for (const chunk of validChunks) {
      const { error } = await supabase
        .from('semantic_chunk')
        .update({
          embedding:       chunk.embedding,
          embedding_model: MODEL_VERSION,
        })
        .eq('originating_event_id', chunk.event_id)
        .is('embedding', null);

      if (error) {
        console.error('[api/embeddings] Update error for event', chunk.event_id, error);
      } else {
        updated++;
      }
    }

    const response: EmbeddingsResponse = { updated };
    return NextResponse.json(response, { status: 200 });

  } catch (err) {
    console.error('[api/embeddings] Unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

FILE: app/api/entities/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

export async function DELETE(req: NextRequest): Promise<NextResponse> {
  try {
    const { searchParams } = new URL(req.url);
```

```
    const id = searchParams.get('id');

    if (!id) {
      return NextResponse.json({ error: 'id is required' }, { status: 400 });
    }

    const supabase = getSupabaseServer();

    // The foreign keys in memory_record and entity_alias are usually ON DELETE
CASCADE or ON DELETE SET NULL
    const { error } = await supabase
      .from('canonical_entity')
      .delete()
      .eq('id', id);

    if (error) {
      console.error('[api/entities] delete error:', error);
      return NextResponse.json({ error: 'Failed to delete entity' }, { status: 503 });
    }

    return NextResponse.json({ success: true });

  } catch (err) {
    console.error('[api/entities] unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}

export async function GET(): Promise<NextResponse> {
  try {
    const supabase = getSupabaseServer();

    const { data, error } = await supabase
      .from('canonical_entity')
      .select('id, canonical_name, entity_type, summary, created_at')
      .order('created_at', { ascending: false });

    if (error) {
      console.error('[api/entities] fetch error:', error);
      return NextResponse.json({ error: 'Failed to fetch entities' }, { status: 503 });
    }

    return NextResponse.json({ entities: data ?? [] });

  } catch (err) {
    console.error('[api/entities] unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

FILE: app/api/export/route.ts

```
import { NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

const TABLES = [
  'system_event',
  'memory_record',
  'canonical_entity',
  'entity_alias',
  'entity_relation',
  'task_projection',
  'reminder_schedule',
  'memory_correction',
] as const;

export async function GET(): Promise<NextResponse> {
  try {
    const supabase = getSupabaseServer();
    const payload: Record<string, unknown> = {
      exported_at: new Date().toISOString(),
      version:     1,
    };
    const counts: Record<string, number> = {};

    for (const table of TABLES) {
      const { data, error } = await supabase
        .from(table)
        .select('*')
        .limit(10000);

      if (error) {
        console.error('[api/export] table error:', table, error);
        return NextResponse.json({ error: 'Export failed at ' + table }, { status: 503 });
      }

      payload[table] = data ?? [];
      counts[table]  = (data ?? []).length;
    }

    try {
      await supabase.from('export_log').insert({
        table_counts: counts,
```

```
        metadata:     { source: 'api/export' },
      });
    } catch {
      // Export logging should never block the export itself.
    }

    return NextResponse.json(payload, {
      headers: {
        'Content-Disposition': 'attachment; filename="lifelong-ai-export.json"',
      },
    });

  } catch (err) {
    console.error('[api/export] unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

FILE: app/api/health/route.ts

```
import { NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';
import { getCognitiveLoad } from '@/lib/cognitive-load';
import type { HealthStatus } from '@/lib/types';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

type HealthRow = {
  overall_score:      number | null;
  embedding_coverage: number | null;
  extraction_success: number | null;
  computed_at:        string;
};

async function getLatestHealth(): Promise<HealthRow | null> {
  try {
    const supabase = getSupabaseServer();
    const { data, error } = await supabase
      .from('cognitive_health_log')
      .select('overall_score, embedding_coverage, extraction_success, computed_at')
      .order('computed_at', { ascending: false })
      .limit(1)
      .single();

    if (error || !data) return null;
```

```
    return data as HealthRow;
  } catch {
    return null;
  }
}

async function getPendingFlagsCount(): Promise<number> {
  try {
    const supabase = getSupabaseServer();
    const { count, error } = await supabase
      .from('identity_flag')
      .select('id', { count: 'exact', head: true })
      .is('resolved_at', null);

    if (error) return 0;
    return count ?? 0;
  } catch {
    return 0;
  }
}

async function getOperationalCounts(): Promise<{
  pending_embeddings: number;
  memory_reviews: number;
  active_memories: number;
}> {
  try {
    const supabase = getSupabaseServer();
    const { data, error } = await supabase
      .from('operational_health')
      .select('pending_embeddings, pending_memory_reviews, active_memories')
      .single();

    if (error || !data) {
      return { pending_embeddings: 0, memory_reviews: 0, active_memories: 0 };
    }

    const row = data as Record<string, unknown>;
    return {
      pending_embeddings: Number(row.pending_embeddings) || 0,
      memory_reviews: Number(row.pending_memory_reviews) || 0,
      active_memories: Number(row.active_memories) || 0,
    };
  } catch {
    return { pending_embeddings: 0, memory_reviews: 0, active_memories: 0 };
  }
}

export async function GET(): Promise<NextResponse> {
  try {
```

```
    const [health, identityFlags, loadData, ops] = await Promise.all([
      getLatestHealth(),
      getPendingFlagsCount(),
      getCognitiveLoad().catch(() => null),
      getOperationalCounts(),
    ]);

    const response: HealthStatus = {
      overall_score:      health?.overall_score      ?? null,
      embedding_coverage: health?.embedding_coverage ?? null,
      extraction_success: health?.extraction_success ?? null,
      entity_conflicts:   0,
      identity_flags:     identityFlags,
      computed_at:        health?.computed_at        ?? null,
      load_state:         loadData?.load_state       ?? 'COMFORTABLE',
      raw_load:           loadData?.raw_load         ?? 0,
      pending_embeddings: ops.pending_embeddings,
      memory_reviews:     ops.memory_reviews,
      active_memories:    ops.active_memories,
    };

    return NextResponse.json(response, {
      status: 200,
      headers: { 'Cache-Control': 'private, max-age=300' },
    });

  } catch (err) {
    console.error('[api/health] Unexpected error:', err);
    return NextResponse.json(
      { error: 'Health check failed' },
      { status: 500 }
    );
  }
}
```

FILE: app/api/identity-flags/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

type Resolution = 'MERGED' | 'KEPT_SEPARATE' | 'PENDING';

export async function GET(): Promise<NextResponse> {
```

```
  try {
    const supabase = getSupabaseServer();

    const { data, error } = await supabase
      .from('identity_flag')
      .select(`
        *,
        entity_a:entity_a_id(canonical_name, classification),
        entity_b:entity_b_id(canonical_name, classification)
      `)
      .is('resolved_at', null)
      .order('flagged_at', { ascending: false })
      .limit(50);

    if (error) {
      console.error('[api/identity-flags] list error:', error);
      return NextResponse.json({ error: 'Failed to fetch identity flags' }, { status: 503 });
    }

    return NextResponse.json({ flags: data ?? [] });

  } catch (err) {
    console.error('[api/identity-flags] unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}

async function mergeEntityPair(keepId: string, mergeId: string): Promise<void> {
  if (keepId === mergeId) return;

  const supabase = getSupabaseServer();

  await supabase.from('entity_alias').update({ canonical_id:
keepId }).eq('canonical_id', mergeId).then(() => null);
  await supabase.from('task_projection').update({ associated_project:
keepId }).eq('associated_project', mergeId).then(() => null);
  await supabase.from('task_projection').update({ assigned_person:
keepId }).eq('assigned_person', mergeId).then(() => null);
  await supabase.from('memory_record').update({ associated_entity_id:
keepId }).eq('associated_entity_id', mergeId).then(() => null);
  await supabase.from('entity_relation').update({ source_entity_id:
keepId }).eq('source_entity_id', mergeId).then(() => null);
  await supabase.from('entity_relation').update({ target_entity_id:
keepId }).eq('target_entity_id', mergeId).then(() => null);
  await supabase.from('epistemic_claim').update({ subject_entity_id:
keepId }).eq('subject_entity_id', mergeId).then(() => null);
  await supabase.from('canonical_entity').delete().eq('id', mergeId).then(() => null);
}

export async function PATCH(req: NextRequest): Promise<NextResponse> {
```

```
  try {
    const body = await req.json().catch(() => null);
    if (!body || typeof body !== 'object') {
      return NextResponse.json({ error: 'Invalid JSON body' }, { status: 400 });
    }

    const id         = body.id as string | undefined;
    const resolution = body.resolution as Resolution | undefined;

    if (!id || !resolution || !['MERGED', 'KEPT_SEPARATE',
'PENDING'].includes(resolution)) {
      return NextResponse.json({ error: 'id and valid resolution are required' }, { status:
400 });
    }

    const supabase = getSupabaseServer();

    if (resolution === 'MERGED') {
      const { data: flagRow, error: flagError } = await supabase
        .from('identity_flag')
        .select('entity_a_id, entity_b_id')
        .eq('id', id)
        .single();

      if (flagError || !flagRow) {
        return NextResponse.json({ error: 'Identity flag not found' }, { status: 404 });
      }

      const keepId = (flagRow as { entity_a_id: string; entity_b_id: string }).entity_a_id;
      const mergeId = (flagRow as { entity_a_id: string; entity_b_id:
string }).entity_b_id;

      await mergeEntityPair(keepId, mergeId);
    }

    const { data, error } = await supabase
      .from('identity_flag')
      .update({
        resolution,
        resolved_at: resolution === 'PENDING' ? null : new Date().toISOString(),
        resolved_by: resolution === 'PENDING' ? null : 'USER',
      })
      .eq('id', id)
      .select('*')
      .single();

    if (error) {
      console.error('[api/identity-flags] patch error:', error);
      return NextResponse.json({ error: 'Identity flag update failed' }, { status: 503 });
    }
```

```
    return NextResponse.json({ flag: data });

  } catch (err) {
    console.error('[api/identity-flags] patch unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

FILE: app/api/import/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';
import type { TabSource } from '@/lib/types';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

interface ImportEvent {
  payload: string;
  source_tab?: TabSource;
  recorded_at?: string;
  metadata?: Record<string, unknown>;
}

export async function POST(req: NextRequest): Promise<NextResponse> {
  try {
    const body = await req.json().catch(() => null);
    if (!body || typeof body !== 'object') {
      return NextResponse.json({ error: 'Invalid JSON body' }, { status: 400 });
    }

    const sourceLabel = String((body as { source_label?: string }).source_label ??
'manual_import');
    const events = (body as { events?: ImportEvent[] }).events;

    if (!Array.isArray(events) || events.length === 0) {
      return NextResponse.json({ error: 'events array is required' }, { status: 400 });
    }

    if (events.length > 500) {
      return NextResponse.json({ error: 'Import is limited to 500 events per request' },
{ status: 400 });
    }

    const supabase = getSupabaseServer();
```

```
    const { data: batch } = await supabase
      .from('ingestion_batch')
      .insert({
        source_label: sourceLabel,
        source_type:  'api',
        imported_rows: events.length,
      })
      .select('id')
      .single();

    const rows = events
      .filter(event => typeof event.payload === 'string' && event.payload.trim().length >
0)
      .map(event => ({
        source_tab: event.source_tab ?? 'THOUGHTS',
        payload: event.payload.trim().slice(0, 10000),
        recorded_at: event.recorded_at ?? new Date().toISOString(),
        metadata: {
          ...(event.metadata ?? {}),
          source: 'import',
          source_label: sourceLabel,
          ingestion_batch_id: (batch as { id?: string } | null)?.id ?? null,
        },
      }));

    const { error } = await supabase.from('system_event').insert(rows);

    if (error) {
      console.error('[api/import] insert error:', error);
      return NextResponse.json({ error: 'Import insert failed' }, { status: 503 });
    }

    if ((batch as { id?: string } | null)?.id) {
      await supabase
        .from('ingestion_batch')
        .update({
          success_rows: rows.length,
          failed_rows: events.length - rows.length,
          completed_at: new Date().toISOString(),
        })
        .eq('id', (batch as { id: string }).id);
    }

    return NextResponse.json({ imported: rows.length, batch_id: (batch as { id?:
string } | null)?.id });

  } catch (err) {
    console.error('[api/import] unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
```

```
  }
}
```

FILE: app/api/ingest/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';
import { assembleContext } from '@/lib/context-assembly';
import { processKnowledgeGraph } from '@/lib/kg-processor';
import type { IngestResponse, TabSource } from '@/lib/types';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

export async function POST(req: NextRequest): Promise<NextResponse> {
  try {
    // ── Parse and validate input
```

```
    const body = await req.json().catch(() => null);

    if (!body || typeof body !== 'object') {
      return NextResponse.json(
        { error: 'Invalid JSON body' },
        { status: 400 }
      );
    }

    const payload    = body.payload    as string | undefined;
    const source_tab = body.source_tab as TabSource | undefined;
    const skip_extraction = body.skip_extraction === true;

    if (!payload || typeof payload !== 'string' || payload.trim().length === 0) {
      return NextResponse.json(
        { error: 'payload is required and must be a non-empty string' },
        { status: 400 }
      );
    }

    if (!source_tab || !['THOUGHTS', 'TASKS', 'CHAT',
'SYSTEM'].includes(source_tab)) {
      return NextResponse.json(
        { error: 'source_tab must be THOUGHTS, TASKS, CHAT, or SYSTEM' },
        { status: 400 }
      );
    }
```

```
  if (payload.length > 6000) {
    return NextResponse.json(
      { error: 'payload exceeds maximum length of 6,000 characters' },
      { status: 400 }
    );
  }

  // ——— Sanitise ———————————————————————————————————————————————————————

  const sanitised = payload
    .replace(/<[^>]*>/g, '')           // strip HTML tags
    .replace(/[\u0000-\u001F]/g, ' ')  // strip control characters
    .trim();

  // ——— Write to immutable event log ———————————————————————————————————

  // This is the ONLY write. Extraction happens async via DB webhook.
  const supabase = getSupabaseServer();

  const { data, error } = await supabase
    .from('system_event')
    .insert({
      source_tab,
      payload:  sanitised,
      metadata: {
        user_agent:     req.headers.get('user-agent')?.slice(0, 100) ?? 'unknown',
        content_length: sanitised.length,
      },
    })
    .select('id')
    .single();

  if (error) {
    console.error('[api/ingest] Supabase insert error:', error);
    return NextResponse.json(
      { error: 'Failed to save. Please try again.' },
      { status: 503 }
    );
  }

  const eventId = (data as { id: string }).id;

  // ——— Run intelligent graph extraction & mutation ————————————————————

  // Await this to ensure mutations complete before the UI polls for new tasks
  try {
    if (!skip_extraction) {
```

```
        const manifest = await assembleContext(sanitised, undefined);
        await processKnowledgeGraph(supabase, eventId, sanitised, source_tab,
manifest);
      }
    } catch (err) {
      console.error('[api/ingest] Graph processing failed:', err);
    }

    const response: IngestResponse = {
      success:  true,
      event_id: eventId,
    };

    return NextResponse.json(response, { status: 201 });

  } catch (err) {
    console.error('[api/ingest] Unexpected error:', err);
    return NextResponse.json(
      { error: 'Internal server error' },
      { status: 500 }
    );
  }
}
```

FILE: app/api/memory/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';
import type { MemoryRecord, MemorySearchRecord } from '@/lib/types';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

export async function GET(req: NextRequest): Promise<NextResponse> {
  try {
    const supabase         = getSupabaseServer();
    const { searchParams } = new URL(req.url);
    const query            = searchParams.get('q')?.trim() ?? '';
    const type             = searchParams.get('type')?.trim();
    const limit            = Math.min(Number(searchParams.get('limit') ?? 50), 100);

    if (query.length > 0) {
      const { data, error } = await supabase.rpc('search_memory_records', {
        p_query: query,
        p_limit: limit,
      });
```

```
    if (error) {
      console.error('[api/memory] search error:', error);
      return NextResponse.json({ error: 'Memory search failed' }, { status: 503 });
    }

    return NextResponse.json({
      memories: cleanMemoryRows((data ?? []) as MemorySearchRecord[]),
      mode:     'search',
    });
  }

  let builder = supabase
    .from('memory_record')
    .select(`
      *,
      entity:associated_entity_id(
        id,
        canonical_name,
        classification,
        summary_portrait
      )
    `)
    .eq('current_status', 'ACTIVE')
    .order('created_at', { ascending: false })
    .limit(limit);

  if (type && type !== 'all') {
    builder = builder.eq('memory_type', type);
  }

  const [memoryResult, entityResult, taskResult] = await Promise.all([
    builder,
    supabase
      .from('canonical_entity')
      .select('id, canonical_name, classification, summary_portrait')
      .order('canonical_name', { ascending: true })
      .limit(100),
    supabase
      .from('task_projection')
      .select('id, title, current_status, due_at, estimated_effort,
project:associated_project(canonical_name),
person:assigned_person(canonical_name)')
      .neq('current_status', 'COMPLETED')
      .neq('current_status', 'ABANDONED')
      .order('created_at', { ascending: false })
      .limit(20),
  ]);

  if (memoryResult.error) {
```

```
      console.error('[api/memory] list error:', memoryResult.error);
      return NextResponse.json({ error: 'Memory list failed' }, { status: 503 });
    }

    return NextResponse.json({
      memories: cleanMemoryRows((memoryResult.data ?? []) as MemoryRecord[]),
      entities: entityResult.data ?? [],
      tasks: taskResult.data ?? [],
      mode:     'list',
    });

  } catch (err) {
    console.error('[api/memory] unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}

function cleanMemoryRows<T extends MemoryRecord |
MemorySearchRecord>(rows: T[]): T[] {
  const seen = new Set<string>();
  const cleaned: T[] = [];

  // Patterns that indicate archive/background content
  const archivePatterns = /background context|archive note|reference only|do not
extract|historical context|deprecated|outdated/i;

  for (const row of rows) {
    const text = [
      row.title,
      row.value ?? '',
      row.summary,
    ].join(' ');

    // Skip archive/background notes completely
    if (archivePatterns.test(text)) {
      continue;
    }

    // Skip notes marked as "not a new task"
    if (/\bnot\s+a\s+new\s+task\b/i.test(text)) {
      continue;
    }

    const key = canonicalMemoryDisplayKey(row);
    if (seen.has(key)) continue;
    seen.add(key);
    cleaned.push(row);
  }

  return cleaned;
```

```
}

function canonicalMemoryDisplayKey(row: MemoryRecord | MemorySearchRecord):
string {
  const text = [row.memory_type, row.title, row.value ?? '', row.summary]
    .join(' ')
    .toLowerCase()
    .replace(/\bbirthdate\b/g, 'birthday')
    .replace(/[^a-z0-9\s]/g, ' ')
    .replace(/\s+/g, ' ')
    .trim();

  const birthday = text.match(/([a-z]+)\s+(birthday).*?([0-9]{1,2}\s+[a-z]+\s+[0-9]{4})/);
  if (birthday) return row.memory_type + ':' + birthday[1] + ':' + birthday[2] + ':' +
birthday[3];

  return text;
}

export async function PATCH(req: NextRequest): Promise<NextResponse> {
  try {
    const body = await req.json().catch(() => null);
    if (!body || typeof body !== 'object') {
      return NextResponse.json({ error: 'Invalid JSON body' }, { status: 400 });
    }

    const id      = body.id as string | undefined;
    const status  = body.status as string | undefined;
    const summary = body.summary as string | undefined;
    const value   = body.value as string | null | undefined;

    if (!id) {
      return NextResponse.json({ error: 'id is required' }, { status: 400 });
    }

    const updateData: Record<string, unknown> = {
      updated_at: new Date().toISOString(),
    };

    if (status) updateData.current_status = status;
    if (typeof summary === 'string') updateData.summary = summary.slice(0, 2000);
    if (typeof value !== 'undefined') updateData.value = value;

    const supabase = getSupabaseServer();
    const { data, error } = await supabase
      .from('memory_record')
      .update(updateData)
      .eq('id', id)
      .select('*')
      .single();
```

```
    if (error) {
      console.error('[api/memory] patch error:', error);
      return NextResponse.json({ error: 'Memory update failed' }, { status: 503 });
    }

    return NextResponse.json({ memory: data as MemoryRecord });

  } catch (err) {
    console.error('[api/memory] patch unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

FILE: app/api/reminders/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';
import type { ReminderSchedule, ReminderState } from '@/lib/types';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

const VALID_STATES: ReminderState[] = [
  'SCHEDULED', 'FIRED', 'SNOOZED', 'CANCELLED', 'DONE',
];

export async function GET(): Promise<NextResponse> {
  try {
    const supabase = getSupabaseServer();
    const { data, error } = await supabase
      .from('reminder_schedule')
      .select('*')
      .in('state', ['SCHEDULED', 'SNOOZED'])
      .order('reminder_at', { ascending: true })
      .limit(50);

    if (error) {
      console.error('[api/reminders] list error:', error);
      return NextResponse.json({ error: 'Failed to fetch reminders' }, { status: 503 });
    }

    return NextResponse.json({ reminders: (data ?? []) as ReminderSchedule[] });

  } catch (err) {
    console.error('[api/reminders] unexpected error:', err);
```

```
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}

export async function PATCH(req: NextRequest): Promise<NextResponse> {
  try {
    const body = await req.json().catch(() => null);
    if (!body || typeof body !== 'object') {
      return NextResponse.json({ error: 'Invalid JSON body' }, { status: 400 });
    }

    const id    = body.id as string | undefined;
    const state = body.state as ReminderState | undefined;

    if (!id || !state || !VALID_STATES.includes(state)) {
      return NextResponse.json({ error: 'id and valid state are required' }, { status:
400 });
    }

    const supabase = getSupabaseServer();
    const { data, error } = await supabase
      .from('reminder_schedule')
      .update({
        state,
        updated_at: new Date().toISOString(),
        fired_at: state === 'FIRED' ? new Date().toISOString() : null,
      })
      .eq('id', id)
      .select('*')
      .single();

    if (error) {
      console.error('[api/reminders] patch error:', error);
      return NextResponse.json({ error: 'Failed to update reminder' }, { status: 503 });
    }

    return NextResponse.json({ reminder: data as ReminderSchedule });

  } catch (err) {
    console.error('[api/reminders] patch unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

FILE: app/api/review/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';
import type { MemoryReviewItem, ReviewState } from '@/lib/types';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

const VALID_STATES: ReviewState[] = [
  'NEEDS_REVIEW', 'APPROVED', 'REJECTED', 'AUTO_ACCEPTED',
];

export async function GET(): Promise<NextResponse> {
  try {
    const supabase = getSupabaseServer();
    const { data, error } = await supabase
      .from('memory_review_queue')
      .select('*, memory:memory_id(*)')
      .eq('status', 'NEEDS_REVIEW')
      .order('severity', { ascending: false })
      .order('created_at', { ascending: false })
      .limit(50);

    if (error) {
      console.error('[api/review] list error:', error);
      return NextResponse.json({ error: 'Failed to fetch review queue' }, { status:
503 });
    }

    return NextResponse.json({ items: (data ?? []) as MemoryReviewItem[] });

  } catch (err) {
    console.error('[api/review] unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}

export async function PATCH(req: NextRequest): Promise<NextResponse> {
  try {
    const body = await req.json().catch(() => null);
    if (!body || typeof body !== 'object') {
      return NextResponse.json({ error: 'Invalid JSON body' }, { status: 400 });
    }

    const id     = body.id as string | undefined;
    const status = body.status as ReviewState | undefined;
    const notes  = body.notes as string | undefined;

    if (!id || !status || !VALID_STATES.includes(status)) {
      return NextResponse.json({ error: 'id and valid status are required' }, { status:
400 });
```

```
}

const supabase = getSupabaseServer();

if (id === 'all') {
  const { data: items, error: fetchError } = await supabase
    .from('memory_review_queue')
    .select('id, memory_id')
    .eq('status', 'NEEDS_REVIEW');

  if (fetchError) {
    return NextResponse.json({ error: 'Fetch failed' }, { status: 503 });
  }

  for (const item of (items ?? [])) {
    await supabase
      .from('memory_review_queue')
      .update({
        status,
        reviewed_at: new Date().toISOString(),
        reviewed_by: 'owner',
      })
      .eq('id', item.id);

    if (item.memory_id) {
      await supabase
        .from('memory_record')
        .update({
          review_status: status,
          current_status: status === 'REJECTED' ? 'REJECTED' : 'ACTIVE',
          updated_at: new Date().toISOString(),
        })
        .eq('id', item.memory_id);
    }
  }

  return NextResponse.json({ success: true, count: items?.length ?? 0 });
}

const { data, error } = await supabase
  .from('memory_review_queue')
  .update({
    status,
    notes: notes ?? null,
    reviewed_at: new Date().toISOString(),
    reviewed_by: 'owner',
  })
  .eq('id', id)
  .select('*, memory:memory_id(*)')
  .single();
```

```
    if (error) {
      console.error('[api/review] patch error:', error);
      return NextResponse.json({ error: 'Review update failed' }, { status: 503 });
    }

    const item = data as MemoryReviewItem;
    if (item.memory_id) {
      await supabase
        .from('memory_record')
        .update({
          review_status: status,
          current_status: status === 'REJECTED' ? 'REJECTED' : 'ACTIVE',
          updated_at: new Date().toISOString(),
        })
        .eq('id', item.memory_id);
    }

    return NextResponse.json({ item });

  } catch (err) {
    console.error('[api/review] patch unexpected error:', err);
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 });
  }
}
```

FILE: app/api/tasks/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import { getSupabaseServer } from '@/lib/supabase/server';
import { getCognitiveLoad } from '@/lib/cognitive-load';
import type { TaskProjection, TasksResponse, TaskState } from '@/lib/types';

export const runtime = 'nodejs';
export const dynamic = 'force-dynamic';

const VALID_STATUSES: TaskState[] = [
  'PENDING', 'ACTIVE', 'BLOCKED', 'COMPLETED', 'ABANDONED',
];

// Shared join fields string
const SELECT_FIELDS =
  '*, ' +
  'project:associated_project(canonical_name), ' +
  'person:assigned_person(canonical_name)';
```

```
// Row shape returned by Supabase after join
interface RawTaskRow {
  id:                  string;
  originating_event_id: string | null;
  associated_project:   string | null;
  assigned_person:      string | null;
  title:               string;
  current_status:       TaskState;
  estimated_effort:     number;
  due_at:              string | null;
  completed_at:         string | null;
  created_at:          string;
  updated_at:           string;
  project:             { canonical_name: string } | null;
  person:               { canonical_name: string } | null;
}

function flattenRow(row: RawTaskRow): TaskProjection {
  return {
    id:                  row.id,
    originating_event_id: row.originating_event_id,
    associated_project:   row.associated_project,
    assigned_person:      row.assigned_person,
    title:               row.title,
    current_status:       row.current_status,
    estimated_effort:     row.estimated_effort,
    due_at:              row.due_at,
    completed_at:         row.completed_at,
    created_at:          row.created_at,
    updated_at:           row.updated_at,
    project_name:         row.project?.canonical_name ?? null,
    person_name:          row.person?.canonical_name  ?? null,
  };
}

// ——— GET /api/tasks
————————————————————————————————————————————————————————————————————————

export async function GET(req: NextRequest): Promise<NextResponse> {
  try {
    const supabase         = getSupabaseServer();
    const { searchParams } = new URL(req.url);
    const statusParam      = searchParams.get('status');

    let query = supabase
      .from('task_projection')
      .select(SELECT_FIELDS)
      .order('created_at', { ascending: false });

    if (statusParam === 'all') {
```

```
    // Explicit all means include completed/abandoned so the Done tab can work.
  } else if (statusParam) {
    if (!VALID_STATUSES.includes(statusParam as TaskState)) {
      return NextResponse.json(
        { error: 'Invalid status value' },
        { status: 400 }
      );
    }
    query = query.eq('current_status', statusParam);
  } else {
    query = query
      .neq('current_status', 'COMPLETED')
      .neq('current_status', 'ABANDONED');
  }

  const { data, error } = await query;

  if (error) {
    console.error('[api/tasks] GET error:', error);
    return NextResponse.json(
      { error: 'Failed to fetch tasks' },
      { status: 503 }
    );
  }

  const tasks = ((data ?? []) as unknown as RawTaskRow[]).map(flattenRow);

  const [load, flagCount] = await Promise.all([
    getCognitiveLoad(),
    getPendingFlagCount(),
  ]);

  const response: TasksResponse = {
    tasks,
    load,
    identity_flags_pending: flagCount,
  };

  return NextResponse.json(response, {
    status: 200,
    headers: {
      'X-Cognitive-Load-State': load.load_state,
      'X-Cognitive-Load-Units': String(load.raw_load),
    },
  });

} catch (err) {
  console.error('[api/tasks] GET unexpected error:', err);
  return NextResponse.json(
    { error: 'Internal server error' },
```

```
      { status: 500 }
    );
  }
}

// ─── PATCH /api/tasks ──────────────────────────────────────────────────────

export async function PATCH(req: NextRequest): Promise<NextResponse> {
  try {
    const body = await req.json().catch(() => null);

    if (!body || typeof body !== 'object') {
      return NextResponse.json(
        { error: 'Invalid JSON body' },
        { status: 400 }
      );
    }

    const id     = body.id     as string | undefined;
    const status = body.status as TaskState | undefined;

    if (!id || typeof id !== 'string') {
      return NextResponse.json(
        { error: 'id is required' },
        { status: 400 }
      );
    }

    if (!status || !VALID_STATUSES.includes(status)) {
      return NextResponse.json(
        { error: 'status must be one of: ' + VALID_STATUSES.join(', ') },
        { status: 400 }
      );
    }

    const supabase = getSupabaseServer();

    // Step 1 — record mutation as a system event (CQRS)
    const { error: eventError } = await supabase
      .from('system_event')
      .insert({
        source_tab: 'TASKS',
        payload:    JSON.stringify({
          action:     'TASK_STATUS_UPDATE',
          task_id:    id,
          new_status: status,
          timestamp:  new Date().toISOString(),
        }),
        metadata: { mutation: true, task_id: id },
```

```
      });

    if (eventError) {
      // Non-fatal — log and continue
      console.error('[api/tasks] PATCH event insert error:', eventError);
    }

    // Step 2 — update task projection
    const updateData: Record<string, unknown> = {
      current_status: status,
      updated_at:     new Date().toISOString(),
      completed_at:   status === 'COMPLETED' ? new Date().toISOString() : null,
    };

    const { data, error: updateError } = await supabase
      .from('task_projection')
      .update(updateData)
      .eq('id', id)
      .select(SELECT_FIELDS)
      .single();

    if (updateError) {
      console.error('[api/tasks] PATCH update error:', updateError);
      return NextResponse.json(
        { error: 'Failed to update task' },
        { status: 503 }
      );
    }

    if (!data) {
      return NextResponse.json(
        { error: 'Task not found' },
        { status: 404 }
      );
    }

    const task = flattenRow(data as unknown as RawTaskRow);

    return NextResponse.json({ task }, { status: 200 });

  } catch (err) {
    console.error('[api/tasks] PATCH unexpected error:', err);
    return NextResponse.json(
      { error: 'Internal server error' },
      { status: 500 }
    );
  }
}

// ─── Helper
```

```
async function getPendingFlagCount(): Promise<number> {
  try {
    const supabase = getSupabaseServer();
    const { count, error } = await supabase
      .from('identity_flag')
      .select('id', { count: 'exact', head: true })
      .is('resolved_at', null);

    if (error) return 0;
    return count ?? 0;
  } catch {
    return 0;
  }
}
```

FILE: app/chat/page.tsx

```
'use client';

import { useState, useEffect, useCallback, useRef } from 'react';
import ChatMessages, { type Message } from '@/components/chat/ChatMessages';
import ChatInput                      from '@/components/chat/ChatInput';
import { supabase }                   from '@/lib/supabase/client';
import { generateEmbedding }          from '@/lib/embeddings';
import type { HealthStatus, LoadState } from '@/lib/types';

type HealthIndicator = 'healthy' | 'fair' | 'poor' | 'unknown';

function toIndicator(score: number | null): HealthIndicator {
  if (score === null)  return 'unknown';
  if (score >= 80)     return 'healthy';
  if (score >= 55)     return 'fair';
  return 'poor';
}

const INDICATOR_COLOR: Record<HealthIndicator, string> = {
  healthy: 'bg-emerald-400',
  fair:    'bg-yellow-400',
  poor:    'bg-red-400',
  unknown: 'bg-slate-600',
};

const LOAD_COLOR: Record<LoadState, string> = {
  COMFORTABLE: 'text-emerald-400',
```

```
  FULL:        'text-yellow-400',
  OVERLOADED:  'text-orange-400',
  CRITICAL:    'text-red-400',
};

export default function ChatPage() {
  const [messages,    setMessages]    = useState<Message[]>([]);
  const [isStreaming, setIsStreaming] = useState(false);
  const [error,       setError]       = useState<string | null>(null);
  const [health,      setHealth]      = useState<HealthStatus | null>(null);
  const abortRef                      = useRef<AbortController | null>(null);

  // ─── Load health status on mount ──────────────────────────────────────────

  useEffect(() => {
    fetch('/api/health')
      .then(r => r.json())
      .then((data: HealthStatus) => setHealth(data))
      .catch(() => null);
  }, []);

  // ─── Load chat history from Supabase on mount ─────────────────────────────

  useEffect(() => {
    async function loadHistory() {
      const { data, error: err } = await supabase
        .from('system_event')
        .select('id, recorded_at, payload, metadata')
        .eq('source_tab', 'CHAT')
        .order('recorded_at', { ascending: false })
        .limit(30);

      if (err || !data) return;

      type EventRow = {
        id:          string;
        recorded_at: string;
        payload:     string;
        metadata:    Record<string, unknown>;
      };

      const history = ((data as EventRow[]).reverse()).map(row => ({
        id:      row.id,
        role:    (row.metadata?.role ?? 'user') as 'user' | 'assistant',
        content: row.payload,
        time:    row.recorded_at,
      }));

      setMessages(history);
```

```
  }

  void loadHistory();
}, []);

// ——— Send message ———————————————————————————————————————————————

const handleSend = useCallback(async (text: string) => {
  if (isStreaming) return;
  setError(null);

  // Optimistic user message
  const userMsg: Message = {
    id:      'user-' + Date.now().toString(),
    role:    'user',
    content: text,
    time:    new Date().toISOString(),
  };
  setMessages(prev => [...prev, userMsg]);
  setIsStreaming(true);

  try {
    // Generate embedding in browser before sending
    const embedding = await generateEmbedding(text).catch(() => null);

    // Cancel any existing stream
    if (abortRef.current) abortRef.current.abort();
    abortRef.current = new AbortController();

    const res = await fetch('/api/chat', {
      method:  'POST',
      headers: { 'Content-Type': 'application/json' },
      body:    JSON.stringify({
        message:   text,
        embedding: embedding ?? undefined,
      }),
      signal: abortRef.current.signal,
    });

    if (!res.ok || !res.body) {
      throw new Error('Request failed: ' + String(res.status));
    }

    // Stream response token by token
    const assistantMsg: Message = {
      id:      'assistant-' + Date.now().toString(),
      role:    'assistant',
      content: '',
      time:    new Date().toISOString(),
```

```
    };
    setMessages(prev => [...prev, assistantMsg]);

    const reader  = res.body.getReader();
    const decoder = new TextDecoder();

    while (true) {
      const { done, value } = await reader.read();
      if (done) break;

      const chunk = decoder.decode(value, { stream: true });
      setMessages(prev =>
        prev.map(m =>
          m.id === assistantMsg.id
            ? { ...m, content: m.content + chunk }
            : m
        )
      );
    }

  } catch (err) {
    if (err instanceof Error && err.name === 'AbortError') return;
    console.error('[ChatPage] send error:', err);
    setError('Something went wrong. Please try again.');
    // Remove optimistic user message on error
    setMessages(prev => prev.filter(m => m.id !== userMsg.id));
  } finally {
    setIsStreaming(false);
  }
}, [isStreaming]);

const indicator = toIndicator(health?.overall_score ?? null);
const loadState = health?.load_state ?? 'COMFORTABLE';

return (
  <div className="flex flex-col h-full">

    {/* Header */}
    <div className="px-4 pt-5 pb-3 border-b border-[#1a2740]
                    flex items-center justify-between">
      <div>
        <h1 className="text-[18px] font-semibold text-violet-400">
          💬 Chat
        </h1>
        <p className="text-[12px] text-slate-600 font-mono mt-0.5">
          Ask anything — memory is active
        </p>
      </div>

      {/* Memory health indicator */}
```

```
  <div className="flex items-center gap-1.5">
    <span
      className={[
        'w-2 h-2 rounded-full',
        INDICATOR_COLOR[indicator],
      ].join(' ')}
      title={'Memory health: ' + indicator}
    />
    <span className="text-[11px] font-mono text-slate-600 capitalize">
      {indicator}
    </span>
  </div>
</div>

{/* Load state banner — shown when overloaded or critical */}
{(loadState === 'OVERLOADED' || loadState === 'CRITICAL') && (
  <div className={[
    'px-4 py-2 border-b border-[#1a2740] flex items-center gap-2',
    'bg-[#0c1222]',
  ].join(' ')}>
    <span className={[
      'text-[12px] font-mono',
      LOAD_COLOR[loadState],
    ].join(' ')}>
      ● {loadState} — {health?.raw_load ?? 0} load units open
    </span>
  </div>
)}

{/* Messages — scrollable */}
<div className="flex-1 overflow-hidden flex flex-col">
  <ChatMessages
    messages={messages}
    isStreaming={isStreaming}
  />
</div>

{/* Error */}
{error && (
  <div className="px-4 py-2 border-t border-[#1a2740]">
    <p className="text-[13px] text-red-400 font-mono">{error}</p>
  </div>
)}

{/* Input — fixed above tab bar */}
<ChatInput
  onSend={(text) => void handleSend(text)}
  isDisabled={isStreaming}
/>
```

```
    </div>
  );
}
```

FILE: app/globals.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

/* ——— Base resets ——————————————————————————————————————— */
*, *::before, *::after {
  -webkit-tap-highlight-color: transparent;
  box-sizing: border-box;
}

html {
  height: 100%;
  height: -webkit-fill-available;
}

body {
  min-height: 100%;
  min-height: -webkit-fill-available;
  background-color: #06080f;
  color: #cbd5e1;
  overscroll-behavior: none;
}

/* ——— Scrollbar ————————————————————————————————————————— */
::-webkit-scrollbar       { width: 4px; height: 4px; }
::-webkit-scrollbar-track { background: transparent; }
::-webkit-scrollbar-thumb { background: #243659; border-radius: 2px; }

/* ——— PWA safe areas (iPhone notch / home bar) ——— */
.safe-top    { padding-top:    env(safe-area-inset-top,    0px); }
.safe-bottom { padding-bottom: env(safe-area-inset-bottom, 0px); }

/* ——— Focus ring ———————————————————————————————————————— */
:focus-visible {
  outline: 2px solid #38bdf8;
  outline-offset: 2px;
  border-radius: 4px;
}
```

```
/* ─── Text selection ──────────────────────────────────────────────────
*/
::selection {
  background: rgba(56, 189, 248, 0.25);
}

/* ─── Typing indicator dots ──────────────────────────────────────────── */
.typing-dot-1 { animation: typing-dot 1.4s infinite 0.0s; }
.typing-dot-2 { animation: typing-dot 1.4s infinite 0.2s; }
.typing-dot-3 { animation: typing-dot 1.4s infinite 0.4s; }

@keyframes typing-dot {
  0%, 60%, 100% { opacity: 0.3; transform: translateY(0);    }
  30%           { opacity: 1.0; transform: translateY(-3px); }
}

/* ─── Message stream animation ────────────────────────────────────────── */
@keyframes fade-in {
  from { opacity: 0; transform: translateY(4px); }
  to   { opacity: 1; transform: translateY(0);   }
}
.animate-fade-in { animation: fade-in 0.15s ease-out; }

/* ─── Swipe action hint ──────────────────────────────────────────────── */
@keyframes swipe-hint {
  0%   { transform: translateX(0);   }
  30%  { transform: translateX(8px); }
  60%  { transform: translateX(0);   }
  100% { transform: translateX(0);   }
}


──────────────────────────────────────────────────────────────────────
──────────────────────
FILE: app/layout.tsx
──────────────────────────────────────────────────────────────────────
──────────────────────
import type { Metadata, Viewport } from 'next';
import './globals.css';
import TabBar          from '@/components/TabBar';
import LayoutShell     from '@/components/LayoutShell';

export const metadata: Metadata = {
  title:       'Lifelong AI',
  description: 'Your personal cognitive assistant',
  manifest:    '/manifest.json',
  appleWebApp: {
    capable:         true,
    statusBarStyle:  'black-translucent',
```

```
    title:          'Lifelong AI',
  },
  icons: {
    apple: '/icons/icon-192.png',
    icon:  '/icons/icon-192.png',
  },
};

export const viewport: Viewport = {
  width:              'device-width',
  initialScale:       1,
  maximumScale:       1,
  userScalable:       false,
  themeColor:         '#06080f',
  viewportFit:        'cover',
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en" className="h-full">
      <body className="h-full bg-[#06080f] text-slate-300 antialiased overflow-
hidden">
        {/*
          LayoutShell handles the browser-side init:
          - loadEmbeddingModel() — silent background download
          - syncPendingEmbeddings() — fills NULL embedding rows
        */}
        <LayoutShell>
          {/* Main scrollable content area — padded for fixed tab bar */}
          <main className="h-full overflow-y-auto pb-14">
            {children}
          </main>
          <TabBar />
        </LayoutShell>
      </body>
    </html>
  );
}
```

FILE: app/memory/page.tsx

```
'use client';
```

```
import { useCallback, useEffect, useState } from 'react';
import type { MemoryRecord, MemorySearchRecord } from '@/lib/types';

type DisplayMemory = MemoryRecord | MemorySearchRecord;
type EntityNode = {
  id: string;
  canonical_name: string;
  classification: string;
  summary_portrait: string | null;
};
type TaskNode = {
  id: string;
  title: string;
  current_status: string;
  due_at: string | null;
  estimated_effort: number;
  project?: { canonical_name: string } | null;
  person?: { canonical_name: string } | null;
};

const FILTERS = [
  'all',
  'PROFILE_FACT',
  'RELATIONSHIP',
  'DECISION',
  'GOAL',
  'PREFERENCE',
  'REMINDER',
] as const;

export default function MemoryPage() {
  const [memories, setMemories] = useState<DisplayMemory[]>([]);
  const [entities, setEntities] = useState<EntityNode[]>([]);
  const [tasks, setTasks] = useState<TaskNode[]>([]);
  const [query, setQuery] = useState('');
  const [filter, setFilter] = useState<(typeof FILTERS)[number]>('all');
  const [isLoading, setIsLoading] = useState(true);

  const load = useCallback(async () => {
    setIsLoading(true);
    try {
      const params = new URLSearchParams();
      if (query.trim()) params.set('q', query.trim());
      if (filter !== 'all' && !query.trim()) params.set('type', filter);
      params.set('limit', '80');

      const res = await fetch('/api/memory?' + params.toString());
      if (!res.ok) return;

      const data = (await res.json()) as {
```

```
        memories: DisplayMemory[];
        entities?: EntityNode[];
        tasks?: TaskNode[];
      };
      setMemories(data.memories);
      setEntities(data.entities ?? []);
      setTasks(data.tasks ?? []);
    } finally {
      setIsLoading(false);
    }
  }, [query, filter]);

  useEffect(() => {
    const timer = setTimeout(() => void load(), 250);
    return () => clearTimeout(timer);
  }, [load]);

  return (
    <div className="flex flex-col h-full">
      <div className="px-4 pt-5 pb-3 border-b border-[#1a2740]">
        <div className="flex items-start justify-between gap-3">
          <div>
            <h1 className="text-[18px] font-semibold text-cyan-300">
              Memory
            </h1>
            <p className="text-[12px] text-slate-600 font-mono mt-0.5">
              Search durable facts, decisions, relationships
            </p>
          </div>
          <a
            href="/admin"
            className="text-[11px] font-mono text-slate-600 border border-[#1a2740]
rounded-md px-2 py-1"
          >
            Admin
          </a>
        </div>
      </div>

      <div className="px-4 py-3 border-b border-[#1a2740]">
        <input
          value={query}
          onChange={e => setQuery(e.target.value)}
          placeholder="Search memory..."
          className="w-full rounded-lg bg-[#0c1222] border border-[#1a2740] px-3
py-2 text-[14px] text-slate-200 placeholder-slate-600 focus:outline-none
focus:border-cyan-500"
        />

        <div className="flex overflow-x-auto gap-2 mt-3 pb-1">
```

```
          {FILTERS.map(item => (
            <button
              key={item}
              onClick={() => setFilter(item)}
              className={[
                'shrink-0 rounded-md px-3 py-1.5 text-[11px] font-mono border',
                filter === item
                  ? 'border-cyan-400 text-cyan-300 bg-cyan-400/10'
                  : 'border-[#1a2740] text-slate-600 bg-[#0c1222]',
              ].join(' ')}
            >
              {item === 'all' ? 'All' : item.replace('_', ' ')}
            </button>
          ))}
        </div>
      </div>

      <div className="flex-1 overflow-y-auto px-4 py-3">
        {isLoading ? (
          <p className="text-[13px] font-mono text-slate-600 animate-pulse">
            Loading memory...
          </p>
        ) : memories.length === 0 ? (
          <p className="text-[13px] text-slate-600 font-mono border border-[#1a2740]
rounded-lg p-3">
            No memory records yet. Add thoughts and wait for extraction.
          </p>
        ) : (
          <>
            {!query.trim() && filter === 'all' && (
              <KnowledgeMap entities={entities} tasks={tasks} />
            )}
            <GroupedMemoryList memories={memories} />
          </>
        )}
      </div>
    </div>
  );
}

function KnowledgeMap({ entities, tasks }: { entities: EntityNode[]; tasks:
TaskNode[] }) {
  const people = entities.filter(e => e.classification === 'PERSON');
  const projects = entities.filter(e => ['PROJECT', 'GOAL'].includes(e.classification));

  return (
    <section className="mb-5 rounded-xl border border-cyan-500/20 bg-
cyan-950/10 p-3">
      <h2 className="text-[12px] font-mono text-cyan-300 mb-2">
        Knowledge Map
```

```
      </h2>
      <p className="text-[12px] text-slate-500 leading-relaxed mb-3">
        This is the current assistant graph view: people, projects, and active
commitments.
        Raw memories below are supporting evidence, not the main model.
      </p>

      <div className="grid grid-cols-1 gap-2">
        <MapBlock label="People" items={people.map(e => e.canonical_name)}
empty="No people identified yet." />
        <MapBlock label="Projects / Goals" items={projects.map(e =>
e.canonical_name)} empty="No projects identified yet." />
        <MapBlock label="Active Commitments" items={tasks.map(t => t.title)}
empty="No active commitments." />
      </div>
    </section>
  );
}

function MapBlock({ label, items, empty }: { label: string; items: string[]; empty:
string }) {
  const unique = Array.from(new Set(items)).slice(0, 12);
  return (
    <div className="rounded-lg border border-[#1a2740] bg-[#0c1222] p-3">
      <p className="text-[10px] font-mono text-slate-600 mb-2">{label}</p>
      {unique.length === 0 ? (
        <p className="text-[12px] text-slate-700">{empty}</p>
      ) : (
        <div className="flex flex-wrap gap-1.5">
          {unique.map(item => (
            <span key={item} className="rounded-full border border-cyan-500/20 px-2
py-1 text-[11px] text-cyan-100 bg-cyan-500/5">
              {item}
            </span>
          ))}
        </div>
      )}
    </div>
  );
}

function GroupedMemoryList({ memories }: { memories: DisplayMemory[] }) {
  const groups = [
    ['People & Relationships', memories.filter(m =>
      m.memory_type === 'PROFILE_FACT' ||
      m.memory_type === 'RELATIONSHIP'
    )],
    ['Projects & Commitments', memories.filter(m =>
      m.memory_type === 'GOAL' ||
      m.memory_type === 'DECISION' ||
```

```
      m.memory_type === 'COMMITMENT'
    )],
    ['Knowledge', memories.filter(m => m.memory_type === 'KNOWLEDGE')],
    ['Preferences & Routines', memories.filter(m =>
      m.memory_type === 'PREFERENCE' ||
      m.memory_type === 'ROUTINE'
    )],
    ['Reminders', memories.filter(m => m.memory_type === 'REMINDER')],
  ] as const;

  const seen = new Set<string>();

  return (
    <div className="space-y-5">
      {groups.map(([label, items]) => {
        const visible = items.filter(item => {
          if (seen.has(item.id)) return false;
          seen.add(item.id);
          return true;
        });
        if (visible.length === 0) return null;

        return (
          <section key={label}>
            <h2 className="text-[11px] font-mono text-cyan-300 mb-2">
              {label}
            </h2>
            <ul className="space-y-2">
              {visible.map(memory => (
                <MemoryCard key={memory.id} memory={memory} />
              ))}
            </ul>
          </section>
        );
      })}
    </div>
  );
}

function MemoryCard({ memory }: { memory: DisplayMemory }) {
  return (
    <li className="rounded-lg border border-[#1a2740] bg-[#0c1222] p-3">
      <div className="flex items-start justify-between gap-3">
        <h3 className="text-[14px] text-slate-200 leading-snug">
          {memory.title}
        </h3>
        <span className="text-[10px] font-mono text-cyan-300 shrink-0">
          {memory.memory_type.replace('_', ' ')}
        </span>
      </div>
```

```
      {memory.value && (
        <p className="text-[13px] text-slate-300 mt-2">
          {memory.value}
        </p>
      )}
      <p className="text-[12px] text-slate-500 mt-2 leading-relaxed">
        {memory.summary}
      </p>
      <p className="text-[10px] font-mono text-slate-700 mt-2">
        confidence {Math.round(Number(memory.confidence_score) * 100)}%
        {memory.source_event_id ? ' · sourced' : ''}
      </p>
    </li>
  );
}
```

FILE: app/page.tsx

```
import { redirect } from 'next/navigation';

// Root route always redirects to Thoughts tab
export default function RootPage() {
  redirect('/thoughts');
}
```

FILE: app/tasks/page.tsx

```
'use client';

import { useState, useEffect, useCallback } from 'react';
import TaskList                from '@/components/tasks/TaskList';
import TaskFilters, {
  type FilterValue,
}                              from '@/components/tasks/TaskFilters';
import { supabase }            from '@/lib/supabase/client';
import { describeLoadState }   from '@/lib/cognitive-load';
import type {
  TaskProjection,
  TasksResponse,
  TaskState,
  CognitiveLoad,
} from '@/lib/types';
```

```
export default function TasksPage() {
  const [allTasks,    setAllTasks]    = useState<TaskProjection[]>([]);
  const [filter,      setFilter]      = useState<FilterValue>('all');
  const [isLoading,   setIsLoading]   = useState(true);
  const [load,        setLoad]        = useState<CognitiveLoad | null>(null);
  const [flagCount,   setFlagCount]   = useState(0);

  // ——— Fetch tasks from API
——————————————————————————————————————————————————————————————————————

  const fetchTasks = useCallback(async () => {
    setIsLoading(true);
    try {
      const res = await fetch('/api/tasks?status=all');
      if (!res.ok) throw new Error('Failed to fetch tasks');
      const data = (await res.json()) as TasksResponse;
      setAllTasks(data.tasks);
      setLoad(data.load);
      setFlagCount(data.identity_flags_pending);
    } catch (err) {
      console.error('[TasksPage] fetch error:', err);
    } finally {
      setIsLoading(false);
    }
  }, []);

  useEffect(() => {
    void fetchTasks();
  }, [fetchTasks]);

  useEffect(() => {
    const refreshOnFocus = () => void fetchTasks();
    window.addEventListener('focus', refreshOnFocus);
    document.addEventListener('visibilitychange', refreshOnFocus);
    return () => {
      window.removeEventListener('focus', refreshOnFocus);
      document.removeEventListener('visibilitychange', refreshOnFocus);
    };
  }, [fetchTasks]);

  // ——— Realtime subscription
——————————————————————————————————————————————————————————————————————

  useEffect(() => {
    const channel = supabase
      .channel('tasks-realtime')
      .on(
        'postgres_changes',
        {
          event:  '*',
```

```
        schema: 'public',
        table:  'task_projection',
      },
      () => void fetchTasks()
    )
    .subscribe();

  return () => { void supabase.removeChannel(channel); };
}, [fetchTasks]);

// ——— Status mutation
————————————————————————————————————————————————————————————————————————————————

const updateStatus = useCallback(
  async (id: string, status: TaskState) => {
    // Optimistic update
    setAllTasks(prev =>
      prev.map(t =>
        t.id === id
          ? {
              ...t,
              current_status: status,
              completed_at:
                status === 'COMPLETED'
                  ? new Date().toISOString()
                  : t.completed_at,
            }
          : t
      )
    );

    try {
      const res = await fetch('/api/tasks', {
        method:  'PATCH',
        headers: { 'Content-Type': 'application/json' },
        body:    JSON.stringify({ id, status }),
      });
      if (!res.ok) throw new Error('PATCH failed');
    } catch (err) {
      console.error('[TasksPage] updateStatus error:', err);
      // Revert optimistic update on error
      void fetchTasks();
    }
  },
  [fetchTasks]
);

const handleComplete = useCallback(
  (id: string) => void updateStatus(id, 'COMPLETED'),
  [updateStatus]
```

```
  );

  const handleBlock = useCallback(
    (id: string) => void updateStatus(id, 'BLOCKED'),
    [updateStatus]
  );

  const handleAbandon = useCallback(
    (id: string) => void updateStatus(id, 'ABANDONED'),
    [updateStatus]
  );

  // ─── Client-side filtering ──────────────────────────────────────────────────────────────────────

  const filteredTasks =
    filter === 'all'
      ? allTasks.filter(
          t => t.current_status !== 'COMPLETED' && t.current_status !== 'ABANDONED'
        )
      : allTasks.filter(t => t.current_status === filter);

  // ─── Count per filter ──────────────────────────────────────────────────────────────────────

  const counts: Record<FilterValue, number> = {
    all:       allTasks.filter(
      t => t.current_status !== 'COMPLETED' && t.current_status !== 'ABANDONED'
    ).length,
    PENDING:   allTasks.filter(t => t.current_status === 'PENDING').length,
    ACTIVE:    allTasks.filter(t => t.current_status === 'ACTIVE').length,
    BLOCKED:   allTasks.filter(t => t.current_status === 'BLOCKED').length,
    COMPLETED: allTasks.filter(t => t.current_status === 'COMPLETED').length,
    ABANDONED: allTasks.filter(t => t.current_status === 'ABANDONED').length,
  };

  const loadDesc = load ? describeLoadState(load.load_state) : null;

  return (
    <div className="flex flex-col h-full">

      {/* Header */}
      <div className="px-4 pt-5 pb-3 border-b border-[#1a2740]">
        <div className="flex items-start justify-between">
          <div>
            <h1 className="text-[18px] font-semibold text-orange-400">
              ✅ Tasks
            </h1>
            <p className="text-[12px] text-slate-600 font-mono mt-0.5">
```

```
        Extracted automatically from your thoughts
      </p>
    </div>

    {/* Refresh */}
    <button
      onClick={() => void fetchTasks()}
      className="text-[11px] font-mono text-slate-600
                 active:text-orange-400 transition-colors
                 min-h-[44px] px-2"
      aria-label="Refresh tasks"
    >
      ↺
    </button>
  </div>

  {/* Cognitive load banner */}
  {loadDesc && load && (
    <div className="mt-2 flex items-center gap-1.5">
      <span className="text-[12px]">{loadDesc.emoji}</span>
      <span className={[
        'text-[12px] font-mono',
        loadDesc.colorClass,
      ].join(' ')}>
        {loadDesc.label} — {load.raw_load} units,{' '}
        {load.open_tasks} tasks open
      </span>
    </div>
  )}

  {/* Identity flags notification */}
  {flagCount > 0 && (
    <div className="mt-2 flex items-center gap-1.5 px-3 py-2
                    bg-[#0c1222] border border-[#1a2740] rounded-lg">
      <span className="text-[12px]">⚡</span>
      <span className="text-[12px] font-mono text-yellow-400">
        {flagCount} possible duplicate contact
        {flagCount > 1 ? 's' : ''} — review identity flags
      </span>
    </div>
  )}
</div>

{/* Filters */}
<TaskFilters
  active={filter}
  onChange={setFilter}
  counts={counts}
/>
```

```
      {/* Task list — scrollable */}
      <div className="flex-1 overflow-y-auto">
        <TaskList
          tasks={filteredTasks}
          isLoading={isLoading}
          onComplete={handleComplete}
          onBlock={handleBlock}
          onAbandon={handleAbandon}
        />
      </div>

    </div>
  );
}
```

FILE: app/thoughts/page.tsx

```
'use client';

import { useState, useEffect, useCallback } from 'react';
import ThoughtInput from '@/components/thoughts/ThoughtInput';
import RecentList   from '@/components/thoughts/RecentList';
import { supabase  } from '@/lib/supabase/client';

interface RecentEntry {
  id:          string;
  recorded_at: string;
  payload:     string;
}

export default function ThoughtsPage() {
  const [entries,   setEntries]   = useState<RecentEntry[]>([]);
  const [isLoading, setIsLoading] = useState(true);

  // ─── Load recent thoughts
```

```
  const loadEntries = useCallback(async () => {
    setIsLoading(true);
    try {
      const { data, error } = await supabase
        .from('system_event')
        .select('id, recorded_at, payload')
        .eq('source_tab', 'THOUGHTS')
        .order('recorded_at', { ascending: false })
        .limit(10);
```

```
    if (error) {
      console.error('[ThoughtsPage] load error:', error);
    } else {
      setEntries((data ?? []) as RecentEntry[]);
    }
  } finally {
    setIsLoading(false);
  }
}, []);

useEffect(() => {
  void loadEntries();
}, [loadEntries]);

// ——— Optimistic prepend on save ——————————————————————————————————————————————

const handleSaved = useCallback((payload: string) => {
  const optimistic: RecentEntry = {
    id:          'optimistic-' + Date.now().toString(),
    recorded_at: new Date().toISOString(),
    payload,
  };
  setEntries(prev => [optimistic, ...prev].slice(0, 10));

  // Reload after 3s to replace optimistic entry with real DB row
  setTimeout(() => void loadEntries(), 3000);
}, [loadEntries]);

return (
  <div className="flex flex-col h-full">

    {/* Header */}
    <div className="px-4 pt-5 pb-3 border-b border-[#1a2740]">
      <h1 className="text-[18px] font-semibold text-sky-400">
        💭 Thoughts
      </h1>
      <p className="text-[12px] text-slate-600 font-mono mt-0.5">
        Dump anything — no structure needed
      </p>
    </div>

    {/* Input */}
    <div className="border-b border-[#1a2740]">
      <ThoughtInput onSaved={handleSaved} />
    </div>

    {/* Recent list — scrollable */}
    <div className="flex-1 overflow-y-auto py-3">
```

```
        <RecentList
          entries={entries}
          isLoading={isLoading}
          onRefresh={() => void loadEntries()}
        />
      </div>

    </div>
  );
}
```

FILE: app/today/page.tsx

```
'use client';

import { useEffect, useState } from 'react';
import type {
  ReminderSchedule,
  TaskProjection,
  TasksResponse,
} from '@/lib/types';

export default function TodayPage() {
  const [tasks, setTasks] = useState<TaskProjection[]>([]);
  const [reminders, setReminders] = useState<ReminderSchedule[]>([]);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    async function load() {
      setIsLoading(true);
      try {
        const [taskRes, reminderRes] = await Promise.all([
          fetch('/api/tasks?status=all'),
          fetch('/api/reminders'),
        ]);

        if (taskRes.ok) {
          const data = (await taskRes.json()) as TasksResponse;
          setTasks(data.tasks.filter(t =>
            t.current_status !== 'COMPLETED' &&
            t.current_status !== 'ABANDONED'
          ));
        }

        if (reminderRes.ok) {
          const data = (await reminderRes.json()) as { reminders: ReminderSchedule[] };
          setReminders(data.reminders);
```

```
      }
    } finally {
      setIsLoading(false);
    }
  }

  void load();
}, []);

const dueSoon = tasks
  .filter(t => t.due_at)
  .sort((a, b) =>
    new Date(a.due_at ?? 0).getTime() - new Date(b.due_at ?? 0).getTime()
  )
  .slice(0, 5);

const highEffort = tasks
  .sort((a, b) => b.estimated_effort - a.estimated_effort)
  .slice(0, 5);

return (
  <div className="flex flex-col h-full">
    <div className="px-4 pt-5 pb-3 border-b border-[#1a2740]">
      <h1 className="text-[18px] font-semibold text-emerald-400">
        Today
      </h1>
      <p className="text-[12px] text-slate-600 font-mono mt-0.5">
        Current commitments and reminders
      </p>
    </div>

    <div className="flex-1 overflow-y-auto px-4 py-4 space-y-5">
      {isLoading ? (
        <p className="text-[13px] font-mono text-slate-600 animate-pulse">
          Loading today...
        </p>
      ) : (
        <>
          <section>
            <h2 className="text-[12px] font-mono text-emerald-400 mb-2">
              REMINDERS
            </h2>
            <ListEmpty show={reminders.length === 0} text="No scheduled
reminders." />
            {reminders.slice(0, 8).map(reminder => (
              <div key={reminder.id} className="border border-[#1a2740] bg-[#0c1222]
rounded-lg p-3 mb-2">
                <p className="text-[14px] text-slate-200">{reminder.title}</p>
                <p className="text-[11px] font-mono text-slate-600 mt-1">
                  {new Date(reminder.reminder_at).toLocaleString()}
```

```
              </p>
            </div>
          ))}
        </section>

        <section>
          <h2 className="text-[12px] font-mono text-orange-400 mb-2">
            DUE SOON
          </h2>
          <ListEmpty show={dueSoon.length === 0} text="No dated tasks yet." />
          {dueSoon.map(task => <TaskRow key={task.id} task={task} />)}
        </section>

        <section>
          <h2 className="text-[12px] font-mono text-sky-400 mb-2">
            HEAVIEST OPEN LOOPS
          </h2>
          <ListEmpty show={highEffort.length === 0} text="No open tasks yet." />
          {highEffort.map(task => <TaskRow key={task.id} task={task} />)}
        </section>
      </>
    )}
  </div>
 </div>
 );
}

function TaskRow({ task }: { task: TaskProjection }) {
 return (
  <div className="border border-[#1a2740] bg-[#0c1222] rounded-lg p-3 mb-2">
   <div className="flex items-start justify-between gap-3">
    <p className="text-[14px] text-slate-200 leading-snug">{task.title}</p>
    <span className="text-[10px] font-mono text-slate-500 shrink-0">
     {task.current_status}
    </span>
   </div>
   <p className="text-[11px] font-mono text-slate-600 mt-1">
    effort {task.estimated_effort}
    {task.due_at ? ' · due ' + new Date(task.due_at).toLocaleDateString() : ''}
   </p>
  </div>
 );
}

function ListEmpty({ show, text }: { show: boolean; text: string }) {
 if (!show) return null;
 return (
  <p className="text-[13px] text-slate-600 font-mono border border-[#1a2740]
rounded-lg p-3">
   {text}
```

```
    </p>
  );
}
```

FILE: components/LayoutShell.tsx

```
'use client';

import { useEffect, useState } from 'react';
import { loadEmbeddingModel, syncPendingEmbeddings } from '@/lib/embeddings';

interface LayoutShellProps {
  children: React.ReactNode;
}

export default function LayoutShell({ children }: LayoutShellProps) {
  const [isSyncing, setIsSyncing] = useState(false);

  useEffect(() => {
    let intervalId: ReturnType<typeof setInterval>;

    const doSync = async () => {
      // Small delay so it doesn't instantly flash for fast syncs
      const timer = setTimeout(() => setIsSyncing(true), 500);
      try {
        await syncPendingEmbeddings();
      } finally {
        clearTimeout(timer);
        setIsSyncing(false);
      }
    };

    const init = async () => {
      try {
        await loadEmbeddingModel();
        await doSync();

        intervalId = setInterval(() => {
          void doSync();
        }, 10000);
      } catch (err) {
        // Never surface embedding errors to the user
        console.error('[LayoutShell] init error:', err);
      }
    };
    void init();
```

```
    return () => {
      if (intervalId) clearInterval(intervalId);
    };
  }, []);

  return (
    <>
      {isSyncing && (
        <div className="fixed top-4 right-4 bg-slate-800 text-slate-300 text-xs px-3
py-1.5 rounded-full shadow-lg border border-slate-700 font-mono z-50 flex items-
center gap-2 animate-pulse">
          <span>⏳</span> Syncing memory...
        </div>
      )}
      {children}
    </>
  );
}
```

FILE: components/TabBar.tsx

```
'use client';

import Link from 'next/link';
import { usePathname } from 'next/navigation';

interface Tab {
  href:  string;
  label: string;
  emoji: string;
  activeColor: string;
  activeBorder: string;
}

const TABS: Tab[] = [
  {
    href:         '/thoughts',
    label:        'Thoughts',
    emoji:        '💭',
    activeColor:  'text-sky-400',
    activeBorder: 'border-sky-400',
  },
  {
    href:         '/chat',
    label:        'Chat',
```

```
    emoji:       '💬',
    activeColor:  'text-violet-400',
    activeBorder: 'border-violet-400',
  },
  {
    href:        '/today',
    label:       'Today',
    emoji:       '◉',
    activeColor:  'text-emerald-400',
    activeBorder: 'border-emerald-400',
  },
  {
    href:        '/tasks',
    label:       'Tasks',
    emoji:       '✅',
    activeColor:  'text-orange-400',
    activeBorder: 'border-orange-400',
  },
  {
    href:        '/memory',
    label:       'Memory',
    emoji:       '◇',
    activeColor:  'text-cyan-300',
    activeBorder: 'border-cyan-300',
  },
];

export default function TabBar() {
  const pathname = usePathname();

  return (
    <nav
      className={[
        'fixed bottom-0 left-0 right-0 z-50',
        'h-16 flex items-stretch',
        'bg-[#0a0f1e] border-t border-[#1a2740]',
        // iPhone home bar safe area
        'pb-[env(safe-area-inset-bottom,0px)]',
      ].join(' ')}
      aria-label="Main navigation"
    >
      {TABS.map(tab => {
        const isActive = pathname.startsWith(tab.href);

        return (
          <Link
            key={tab.href}
            href={tab.href}
            className={[
```

```
            'flex-1 flex flex-col items-center justify-center gap-0.5',
            'min-h-[52px] select-none transition-colors duration-150',
            'border-t-2',
            isActive
              ? tab.activeColor + ' ' + tab.activeBorder
              : 'text-slate-500 border-transparent',
          ].join(' ')}
          aria-current={isActive ? 'page' : undefined}
        >
          <span
            className="text-base leading-none"
            role="img"
            aria-hidden="true"
          >
            {tab.emoji}
          </span>
          <span className="text-[9px] font-mono leading-none">
            {tab.label}
          </span>
        </Link>
      );
    })}
  </nav>
 );
}
```

FILE: components/chat/ChatInput.tsx

```
'use client';

import { useState, useRef, useCallback } from 'react';

interface ChatInputProps {
 onSend:     (message: string) => void;
 isDisabled: boolean;
}

export default function ChatInput({ onSend, isDisabled }: ChatInputProps) {
 const [text,      setText]      = useState('');
 const textareaRef               = useRef<HTMLTextAreaElement>(null);
 const isEmpty                   = text.trim().length === 0;

 const handleChange = useCallback(
   (e: React.ChangeEvent<HTMLTextAreaElement>) => {
     setText(e.target.value);
     const el = e.target;
```

```
    el.style.height = 'auto';
    const lineHeight = 22;
    const maxHeight  = lineHeight * 5;
    el.style.height  = Math.min(el.scrollHeight, maxHeight) + 'px';
  },
  []
);

const handleSend = useCallback(() => {
  const trimmed = text.trim();
  if (!trimmed || isDisabled) return;
  onSend(trimmed);
  setText('');
  if (textareaRef.current) {
    textareaRef.current.style.height = 'auto';
  }
}, [text, isDisabled, onSend]);

const handleKeyDown = useCallback(
  (e: React.KeyboardEvent<HTMLTextAreaElement>) => {
    if ((e.ctrlKey || e.metaKey) && e.key === 'Enter') {
      e.preventDefault();
      handleSend();
    }
  },
  [handleSend]
);

return (
  <div className="flex items-end gap-2 px-3 py-3
                  border-t border-[#1a2740] bg-[#0a0f1e]">
    <textarea
      ref={textareaRef}
      value={text}
      onChange={handleChange}
      onKeyDown={handleKeyDown}
      placeholder="Ask anything..."
      inputMode="text"
      rows={1}
      disabled={isDisabled}
      className={[
        'flex-1 resize-none rounded-xl px-4 py-2.5',
        'bg-[#0c1222] border border-[#1a2740]',
        'text-slate-200 placeholder-slate-600',
        'text-[14px] leading-[22px]',
        'focus:outline-none focus:border-violet-600',
        'transition-colors duration-150',
        'min-h-[44px]',
        isDisabled ? 'opacity-50' : '',
      ].join(' ')}
```

```
        style={{ overflow: 'hidden' }}
        aria-label="Chat message input"
      />

      <button
        onClick={handleSend}
        disabled={isEmpty || isDisabled}
        className={[
          'flex items-center justify-center',
          'w-10 h-10 rounded-xl shrink-0',
          'transition-all duration-150',
          isEmpty || isDisabled
            ? 'bg-[#0c1222] text-slate-600 border border-[#1a2740]'
            : 'bg-violet-600 text-white active:bg-violet-700',
        ].join(' ')}
        aria-label="Send message"
      >
        <span className="text-lg leading-none">↑</span>
      </button>
    </div>
  );
}
```

FILE: components/chat/ChatMessages.tsx

```
'use client';

import { useEffect, useRef } from 'react';

export interface Message {
  id:      string;
  role:    'user' | 'assistant';
  content: string;
  time:    string; // ISO string
}

interface ChatMessagesProps {
  messages:   Message[];
  isStreaming: boolean;
}

function formatTime(iso: string): string {
  return new Date(iso).toLocaleTimeString([], {
    hour:   '2-digit',
    minute: '2-digit',
  });
}
```

```
function shouldShowTime(messages: Message[], index: number): boolean {
  if (index === 0) return true;
  const prev    = messages[index - 1];
  const current = messages[index];
  const diff    = new Date(current.time).getTime() - new Date(prev.time).getTime();
  // Show time if more than 1 hour gap
  return diff > 60 * 60 * 1000;
}

function TypingIndicator() {
  return (
    <div className="flex items-start gap-2 animate-fade-in">
      <div className="flex flex-col gap-1 max-w-[80%]">
        <span className="text-[10px] font-mono text-violet-400 ml-1">AI</span>
        <div className="bg-[#0c1222] border border-[#1a2740] rounded-2xl
                        rounded-tl-sm px-4 py-3 flex items-center gap-1.5">
          <span className="w-1.5 h-1.5 rounded-full bg-violet-400 typing-dot-1" />
          <span className="w-1.5 h-1.5 rounded-full bg-violet-400 typing-dot-2" />
          <span className="w-1.5 h-1.5 rounded-full bg-violet-400 typing-dot-3" />
        </div>
      </div>
    </div>
  );
}

export default function ChatMessages({
  messages,
  isStreaming,
}: ChatMessagesProps) {
  const bottomRef = useRef<HTMLDivElement>(null);

  // Auto-scroll to bottom on new messages
  useEffect(() => {
    bottomRef.current?.scrollIntoView({ behavior: 'smooth' });
  }, [messages, isStreaming]);

  if (messages.length === 0 && !isStreaming) {
    return (
      <div className="flex-1 flex flex-col items-center justify-center
                      px-6 text-center gap-4">
        <span className="text-5xl">💬</span>
        <div className="flex flex-col gap-1">
          <p className="text-slate-300 text-[15px] font-medium">
            Ask me anything
          </p>
          <p className="text-slate-600 text-[13px] leading-relaxed">
            I have memory of all your thoughts, tasks, and goals.
            Try asking about what&amp;apos;s pending, decisions you made,
            or whether you should take on something new.
```

```
        </p>
      </div>
      <div className="flex flex-col gap-2 w-full max-w-[280px] mt-2">
        {[
          'What tasks are still open?',
          'What have I been working on?',
          'What commitments did I make this week?',
        ].map(suggestion => (
          <div
            key={suggestion}
            className="text-[12px] font-mono text-slate-600 border
                       border-[#1a2740] rounded-lg px-3 py-2 text-left"
          >
            {suggestion}
          </div>
        ))}
      </div>
    </div>
  );
}

return (
  <div className="flex-1 overflow-y-auto px-4 py-4 flex flex-col gap-4">

    {messages.map((msg, index) => {
      const showTime = shouldShowTime(messages, index);
      const isUser   = msg.role === 'user';

      return (
        <div key={msg.id} className="flex flex-col gap-1 animate-fade-in">

          {/* Time divider */}
          {showTime && (
            <div className="flex items-center justify-center my-1">
              <span className="text-[10px] font-mono text-slate-700">
                {formatTime(msg.time)}
              </span>
            </div>
          )}

          {/* Message bubble */}
          <div className={[
            'flex items-end gap-2',
            isUser ? 'justify-end' : 'justify-start',
          ].join(' ')}>

            <div className={[
              'flex flex-col gap-1',
              isUser ? 'items-end max-w-[80%]' : 'items-start max-w-[88%]',
            ].join(' ')}>
```

```
              {/* AI label */}
              {!isUser && (
                <span className="text-[10px] font-mono text-violet-400 ml-1">
                  AI
                </span>
              )}

              {/* Bubble */}
              <div className={[
                'px-4 py-3 text-[14px] leading-relaxed whitespace-pre-wrap',
                isUser
                  ? 'bg-[#1a1040] text-slate-200 rounded-2xl rounded-br-sm'
                  : 'bg-[#0c1222] border border-[#1a2740] text-slate-200 rounded-2xl
rounded-tl-sm',
              ].join(' ')}>
                {msg.content}
              </div>

            </div>
          </div>

        </div>
      );
    })}

    {/* Typing indicator */}
    {isStreaming && <TypingIndicator />}

    {/* Scroll anchor */}
    <div ref={bottomRef} className="h-1" />

  </div>
 );
}
```

FILE: components/tasks/TaskFilters.tsx

```
'use client';

import type { TaskState } from '@/lib/types';

export type FilterValue = 'all' | TaskState;

interface TaskFiltersProps {
  active:   FilterValue;
  onChange: (value: FilterValue) => void;
```

```
  counts:   Record<FilterValue, number>;
}

const FILTERS: { value: FilterValue; label: string }[] = [
  { value: 'all',       label: 'All'       },
  { value: 'PENDING',   label: 'Pending'   },
  { value: 'ACTIVE',    label: 'Active'    },
  { value: 'BLOCKED',   label: 'Blocked'   },
  { value: 'COMPLETED', label: 'Done'      },
];

export default function TaskFilters({
  active,
  onChange,
  counts,
}: TaskFiltersProps) {
  return (
    <div className="flex items-center gap-2 px-4 py-3
                    overflow-x-auto scrollbar-none border-b border-[#1a2740]">
      {FILTERS.map(f => {
        const isActive = active === f.value;
        const count    = counts[f.value] ?? 0;

        return (
          <button
            key={f.value}
            onClick={() => onChange(f.value)}
            className={[
              'flex items-center gap-1.5 px-3 py-1.5 rounded-full',
              'text-[12px] font-mono whitespace-nowrap shrink-0',
              'transition-all duration-150 min-h-[36px]',
              'border',
              isActive
                ? 'bg-orange-500/15 border-orange-500 text-orange-400'
                : 'bg-transparent border-[#1a2740] text-slate-500',
            ].join(' ')}
            aria-pressed={isActive}
          >
            <span>{f.label}</span>
            {count > 0 && (
              <span className={[
                'text-[10px] px-1.5 py-0.5 rounded-full',
                isActive
                  ? 'bg-orange-500/20 text-orange-400'
                  : 'bg-[#1a2740] text-slate-600',
              ].join(' ')}>
                {count}
              </span>
            )}
          </button>
```

```
        );
      })}
    </div>
  );
}
```

FILE: components/tasks/TaskList.tsx

```
'use client';

import { useState, useCallback } from 'react';
import type { TaskProjection, TaskState } from '@/lib/types';

interface TaskListProps {
  tasks:      TaskProjection[];
  isLoading:  boolean;
  onComplete: (id: string) => void;
  onBlock:    (id: string) => void;
  onAbandon:  (id: string) => void;
}

const STATUS_BORDER: Record<TaskState, string> = {
  PENDING:   'border-l-orange-400',
  ACTIVE:    'border-l-sky-400',
  BLOCKED:   'border-l-red-400',
  COMPLETED: 'border-l-emerald-500',
  ABANDONED: 'border-l-slate-600',
};

const STATUS_BADGE: Record<TaskState, string> = {
  PENDING:   'text-orange-400 bg-orange-400/10',
  ACTIVE:    'text-sky-400    bg-sky-400/10',
  BLOCKED:   'text-red-400    bg-red-400/10',
  COMPLETED: 'text-emerald-400 bg-emerald-400/10',
  ABANDONED: 'text-slate-500  bg-slate-500/10',
};

const EFFORT_DOTS = [1, 2, 3, 4, 5];

export default function TaskList({
  tasks,
  isLoading,
  onComplete,
  onBlock,
  onAbandon,
}: TaskListProps) {
  const [expandedId, setExpandedId] = useState<string | null>(null);
```

```
  const [swipedId,   setSwipedId]   = useState<string | null>(null);
  const [expandedProjects, setExpandedProjects] = useState<Record<string,
boolean>>({});

  const toggleExpand = useCallback((id: string) => {
    setExpandedId(prev => (prev === id ? null : id));
  }, []);

  const toggleSwipe = useCallback((id: string) => {
    setSwipedId(prev => (prev === id ? null : id));
  }, []);

  const toggleProject = useCallback((project: string) => {
    setExpandedProjects(prev => ({ ...prev, [project]: !prev[project] }));
  }, []);

  if (isLoading) {
    return (
      <div className="flex items-center justify-center py-10">
        <span className="text-slate-600 font-mono text-[13px] animate-pulse">
          Loading tasks...
        </span>
      </div>
    );
  }

  if (tasks.length === 0) {
    return (
      <div className="flex flex-col items-center justify-center
                      py-14 px-6 text-center gap-3">
        <span className="text-4xl">✅</span>
        <p className="text-slate-500 text-[14px] leading-relaxed max-w-[260px]">
          Tasks appear automatically when you add thoughts.
          Nothing here yet.
        </p>
        <p className="text-slate-700 text-[12px] font-mono">
          Try adding a thought that mentions a task or commitment.
        </p>
      </div>
    );
  }

  // Group tasks
  const standaloneTasks: TaskProjection[] = [];
  const projectTasks: Record<string, TaskProjection[]> = {};

  tasks.forEach(task => {
    if (task.project_name) {
      if (!projectTasks[task.project_name]) {
        projectTasks[task.project_name] = [];
```

```
      }
      projectTasks[task.project_name].push(task);
    } else {
      standaloneTasks.push(task);
    }
  });

  const projectOrder = Object.keys(projectTasks).sort();

  const renderTask = (task: TaskProjection, isSubtask: boolean = false) => {
    const isExpanded = expandedId === task.id;
    const isSwiped   = swipedId   === task.id;
    const isDone     = task.current_status === 'COMPLETED';

    return (
      <li key={task.id} className={['relative overflow-hidden rounded-lg', isSubtask ?
'ml-4 border-l-2 border-[#1a2740]' : ''].join(' ')}>

        {/* Swipe action background */}
        {isSwiped && !isDone && (
          <div className="absolute inset-0 flex items-center
                          justify-end pr-4 bg-emerald-900/40
                          rounded-lg z-0">
            <button
              onClick={() => {
                setSwipedId(null);
                onComplete(task.id);
              }}
              className="text-[13px] font-mono text-emerald-400
                         bg-emerald-500/20 border border-emerald-500
                         px-4 py-2 rounded-lg min-h-[44px]"
            >
              ✓ Complete
            </button>
            {task.current_status !== 'BLOCKED' && (
              <button
                onClick={() => {
                  setSwipedId(null);
                  onBlock(task.id);
                }}
                className="ml-2 text-[13px] font-mono text-red-400
                           bg-red-500/20 border border-red-500
                           px-4 py-2 rounded-lg min-h-[44px]"
              >
                ⛔ Block
              </button>
            )}
          </div>
        )}
```

```
{/* Main task card */}
<div
  className={[
    'relative z-10 bg-[#0c1222] border border-[#1a2740]',
    'border-l-4 rounded-lg transition-transform duration-200',
    STATUS_BORDER[task.current_status],
    isSwiped ? 'translate-x-[-80px]' : 'translate-x-0',
  ].join(' ')}
>
  {/* Tap row */}
  <button
    onClick={() => toggleExpand(task.id)}
    onContextMenu={e => {
      e.preventDefault();
      toggleSwipe(task.id);
    }}
    className="w-full text-left px-4 py-3 min-h-[52px]"
    aria-expanded={isExpanded}
  >
    <div className="flex items-start justify-between gap-3">
      {/* Title */}
      <span className={[
        'text-[14px] leading-snug flex-1',
        isDone ? 'text-slate-500 line-through' : 'text-slate-200',
      ].join(' ')}>
        {task.title}
      </span>

      {/* Status badge */}
      <span className={[
        'text-[10px] font-mono px-2 py-0.5 rounded shrink-0 mt-0.5',
        STATUS_BADGE[task.current_status],
      ].join(' ')}>
        {task.current_status}
      </span>
    </div>
  </button>

  {/* Expanded detail */}
  {isExpanded && (
    <div className="px-4 pb-3 border-t border-[#1a2740] pt-3
                    flex flex-col gap-2">
      {/* Meta row */}
      <div className="flex flex-wrap gap-x-4 gap-y-1">
        {task.project_name && (
          <span className="text-[12px] font-mono text-slate-500">
            📁 {task.project_name}
          </span>
        )}
        {task.person_name && (
```

```
    <span className="text-[12px] font-mono text-slate-500">
      👤 {task.person_name}
    </span>
  )}
  {task.due_at && (
    <span className="text-[12px] font-mono text-slate-500">
      📅 {new Date(task.due_at).toLocaleDateString()}
    </span>
  )}
</div>

{/* Effort dots */}
<div className="flex items-center gap-1">
  <span className="text-[11px] font-mono text-slate-600 mr-1">
    effort
  </span>
  {EFFORT_DOTS.map(dot => (
    <span
      key={dot}
      className={[
        'w-2 h-2 rounded-full',
        dot <= task.estimated_effort
          ? 'bg-orange-400'
          : 'bg-[#1a2740]',
      ].join(' ')}
    />
  ))}
</div>

{/* Action buttons */}
{!isDone && (
  <div className="flex gap-2 mt-1">
    <button
      onClick={() => onComplete(task.id)}
      className="flex-1 text-[12px] font-mono text-emerald-400
                 border border-emerald-500/40 rounded-lg py-2
                 active:bg-emerald-500/10 min-h-[44px]
                 transition-colors"
    >
      ✓ Complete
    </button>
    {task.current_status !== 'BLOCKED' && (
      <button
        onClick={() => onBlock(task.id)}
        className="flex-1 text-[12px] font-mono text-orange-400
                   border border-orange-500/40 rounded-lg py-2
                   active:bg-orange-500/10 min-h-[44px]
                   transition-colors"
      >
```

```
                ⛔ Block
              </button>
            )}
            <button
              onClick={() => {
                if (confirm('Are you sure you want to abandon/delete this task?')) {
                  onAbandon(task.id);
                }
              }}
              className="flex-1 text-[12px] font-mono text-red-400
                         border border-red-500/40 rounded-lg py-2
                         active:bg-red-500/10 min-h-[44px]
                         transition-colors"
            >
              🗑️ Delete
            </button>
          </div>
        )}

        {/* ID for reference */}
        <p className="text-[10px] font-mono text-slate-700 mt-1">
          ID: {task.id}
        </p>
      </div>
    )}
  </div>
 </li>
 );
};

 return (
  <div className="flex flex-col px-4 py-3 gap-6">
   {/* Grouped Tasks */}
   {projectOrder.length > 0 && (
    <div className="flex flex-col gap-4">
     {projectOrder.map(projectName => {
      const pTasks = projectTasks[projectName];
      const isProjectExpanded = expandedProjects[projectName] !== false; //
default true

      return (
       <div key={projectName} className="flex flex-col gap-2">
        <button
         onClick={() => toggleProject(projectName)}
         className="flex items-center gap-2 text-left w-full py-1 group"
        >
         <span className="text-slate-400 font-mono text-[16px] group-
hover:text-slate-300">
          {isProjectExpanded ? '▼' : '▶'}
         </span>
```

```
                <span className="text-[15px] font-semibold text-slate-300 group-
hover:text-white">
                  {projectName}
                </span>
                <span className="text-[11px] text-slate-500 font-mono bg-slate-800/50
px-2 py-0.5 rounded-full ml-2">
                  {pTasks.length}
                </span>
              </button>

              {isProjectExpanded && (
                <ul className="flex flex-col gap-1.5 mt-1">
                  {pTasks.map(task => renderTask(task, true))}
                </ul>
              )}
            </div>
          );
        })}
      </div>
    )}

    {/* Standalone Tasks */}
    {standaloneTasks.length > 0 && (
      <div className="flex flex-col gap-2">
        {projectOrder.length > 0 && (
          <h3 className="text-[13px] font-mono text-slate-500 uppercase tracking-
wider mb-2 ml-1">
            Standalone Tasks
          </h3>
        )}
        <ul className="flex flex-col gap-1.5">
          {standaloneTasks.map(task => renderTask(task, false))}
        </ul>
      </div>
    )}
  </div>
 );
}
```

FILE: components/thoughts/RecentList.tsx

```
'use client';

import { useState } from 'react';

interface RecentEntry {
  id:          string;
```

```
  recorded_at: string;
  payload:     string;
}

interface RecentListProps {
  entries:    RecentEntry[];
  isLoading:  boolean;
  onRefresh:  () => void;
}

function relativeTime(iso: string): string {
  const diff = Date.now() - new Date(iso).getTime();
  const mins  = Math.floor(diff / 60000);
  const hours = Math.floor(diff / 3600000);
  const days  = Math.floor(diff / 86400000);

  if (mins  <  1) return 'just now';
  if (mins  < 60) return String(mins)  + 'm';
  if (hours < 24) return String(hours) + 'h';
  if (days  <  2) return 'yesterday';
  return             String(days)  + 'd';
}

export default function RecentList({
  entries,
  isLoading,
  onRefresh,
}: RecentListProps) {
  const [expandedId, setExpandedId] = useState<string | null>(null);

  const toggleExpand = (id: string) => {
    setExpandedId(prev => (prev === id ? null : id));
  };

  if (isLoading) {
    return (
      <div className="px-4 py-6 flex items-center justify-center">
        <span className="text-slate-600 font-mono text-[13px] animate-pulse">
          Loading...
        </span>
      </div>
    );
  }

  if (entries.length === 0) {
    return (
      <div className="px-4 py-10 flex flex-col items-center gap-3 text-center">
        <span className="text-4xl">💭</span>
        <p className="text-slate-500 text-[14px] leading-relaxed max-w-[260px]">
          Your thoughts will appear here after you send the first one.
```

```
        </p>
      </div>
    );
  }

  return (
    <div className="px-4">

      {/* Header */}
      <div className="flex items-center justify-between mb-3">
        <span className="text-[10px] font-mono tracking-widest text-slate-500
uppercase">
          Recent
        </span>
        <button
          onClick={onRefresh}
          className="text-[11px] font-mono text-slate-600 active:text-sky-400
                transition-colors min-h-[44px] px-2"
          aria-label="Refresh recent thoughts"
        >
          ↺ Refresh
        </button>
      </div>

      {/* Entries */}
      <ul className="flex flex-col gap-1">
        {entries.map(entry => {
          const isExpanded = expandedId === entry.id;
          const preview    = entry.payload.slice(0, 70);
          const hasMore    = entry.payload.length > 70;

          return (
            <li key={entry.id}>
              <button
                onClick={() => toggleExpand(entry.id)}
                className={[
                  'w-full text-left px-3 py-3 rounded-lg',
                  'bg-[#0c1222] border border-[#1a2740]',
                  'active:border-sky-800 transition-colors duration-100',
                  'min-h-[44px]',
                ].join(' ')}
                aria-expanded={isExpanded}
              >
                {/* Collapsed view */}
                {!isExpanded && (
                  <div className="flex items-start justify-between gap-2">
                    <span className="text-slate-300 text-[14px] leading-snug flex-1 text-
left">
                      {preview}
                      {hasMore && (
```

```
                  <span className="text-slate-600"> ...</span>
                )}
              </span>
              <span className="text-[11px] font-mono text-slate-600
                               shrink-0 mt-0.5">
                {relativeTime(entry.recorded_at)}
              </span>
            </div>
          )}

          {/* Expanded view */}
          {isExpanded && (
            <div className="flex flex-col gap-2">
              <p className="text-slate-200 text-[14px] leading-relaxed text-left">
                {entry.payload}
              </p>
              <div className="flex items-center justify-between">
                <span className="text-[11px] font-mono text-slate-600">
                  {new Date(entry.recorded_at).toLocaleString([], {
                    month:  'short',
                    day:    'numeric',
                    hour:   '2-digit',
                    minute: '2-digit',
                  })}
                </span>
                <span className="text-[11px] font-mono text-slate-600">
                  tap to collapse ↑
                </span>
              </div>
            </div>
          )}
        </button>
      </li>
    );
  })}
</ul>

</div>
);
}
```

FILE: components/thoughts/ThoughtInput.tsx

```
'use client';

import { useState, useRef, useCallback } from 'react';
```

```
interface ThoughtInputProps {
  onSaved: (payload: string) => void;
}

type SaveState = 'idle' | 'saving' | 'saved' | 'error';

export default function ThoughtInput({ onSaved }: ThoughtInputProps) {
  const [text,      setText]     = useState('');
  const [saveState, setSaveState] = useState<SaveState>('idle');
  const [skipExtraction, setSkipExtraction] = useState(false);
  const textareaRef              = useRef<HTMLTextAreaElement>(null);

  const isEmpty = text.trim().length === 0;

  // Auto-resize textarea between 3 and 8 rows
  const handleChange = useCallback(
    (e: React.ChangeEvent<HTMLTextAreaElement>) => {
      setText(e.target.value);
      const el = e.target;
      el.style.height = 'auto';
      const lineHeight = 24;
      const minHeight  = lineHeight * 3;
      const maxHeight  = lineHeight * 8;
      el.style.height  = Math.min(
        Math.max(el.scrollHeight, minHeight),
        maxHeight
      ) + 'px';
    },
    []
  );

  const handleSend = useCallback(async () => {
    if (isEmpty || saveState === 'saving') return;

    const payload = text.trim();
    setSaveState('saving');

    try {
      const res = await fetch('/api/ingest', {
        method:  'POST',
        headers: { 'Content-Type': 'application/json' },
        body:    JSON.stringify({ payload, source_tab: 'THOUGHTS', skip_extraction:
skipExtraction }),
      });

      if (!res.ok) {
        throw new Error('Server error: ' + String(res.status));
      }

      // Optimistic clear immediately on success
```

```
    setText('');
    if (textareaRef.current) {
      textareaRef.current.style.height = 'auto';
    }

    setSaveState('saved');
    onSaved(payload);

    // Reset to idle after 1.5s
    setTimeout(() => setSaveState('idle'), 1500);

  } catch (err) {
    console.error('[ThoughtInput] save error:', err);
    setSaveState('error');
  }
}, [text, isEmpty, saveState, onSaved, skipExtraction]);

// Send on Ctrl+Enter / Cmd+Enter
const handleKeyDown = useCallback(
  (e: React.KeyboardEvent<HTMLTextAreaElement>) => {
    if ((e.ctrlKey || e.metaKey) && e.key === 'Enter') {
      e.preventDefault();
      void handleSend();
    }
  },
  [handleSend]
);

const handleRetry = useCallback(() => {
  setSaveState('idle');
}, []);

return (
  <div className="px-4 pt-4 pb-2">
    {/* Textarea */}
    <textarea
      ref={textareaRef}
      value={text}
      onChange={handleChange}
      onKeyDown={handleKeyDown}
      placeholder="Voice or type anything on your mind..."
      inputMode="text"
      rows={3}
      className={[
        'w-full resize-none rounded-lg px-4 py-3',
        'bg-[#0c1222] border border-[#1a2740]',
        'text-slate-200 placeholder-slate-600',
        'text-[15px] leading-6',
        'focus:outline-none focus:border-sky-500',
        'transition-colors duration-150',
```

```
          'min-h-[72px]',
        ].join(' ')}
        style={{ overflow: 'hidden' }}
        disabled={saveState === 'saving'}
        aria-label="Thought input"
      />

      {/* Send button + status */}
      <div className="flex items-center justify-between mt-2">

        {/* Status message and Raw notes toggle */}
        <div className="flex items-center gap-4 text-[13px] font-mono min-h-[20px]">
          <label className="flex items-center gap-2 cursor-pointer text-slate-400
hover:text-slate-200 transition-colors">
            <input
              type="checkbox"
              checked={skipExtraction}
              onChange={(e) => setSkipExtraction(e.target.checked)}
              className="rounded border-slate-700 bg-slate-800 text-sky-500 focus:ring-
sky-500/50"
            />
            Raw Notes
          </label>
          {saveState === 'saving' && (
            <span className="text-slate-500 animate-pulse">Saving...</span>
          )}
          {saveState === 'saved' && (
            <span className="text-emerald-400">Saved ✓</span>
          )}
          {saveState === 'error' && (
            <button
              onClick={handleRetry}
              className="text-red-400 underline underline-offset-2"
            >
              Could not save — tap to retry
            </button>
          )}
        </div>

        {/* Send button */}
        <button
          onClick={() => void handleSend()}
          disabled={isEmpty || saveState === 'saving'}
          className={[
            'flex items-center gap-2 px-4 py-2 rounded-lg',
            'text-[13px] font-mono font-medium',
            'transition-all duration-150 min-h-[44px] min-w-[80px]',
            'justify-center',
            isEmpty || saveState === 'saving'
              ? 'bg-[#0c1222] text-slate-600 cursor-not-allowed border border-[#1a2740]'
```

```
            : 'bg-sky-500 text-white active:bg-sky-600 border border-sky-500',
        ].join(' ')}
        aria-label="Send thought"
      >
        <span>→</span>
        <span>Send</span>
      </button>

    </div>

    {/* Hint */}
    <p className="text-[11px] text-slate-600 mt-1 font-mono">
      Ctrl+Enter to send · Voice via keyboard mic
    </p>
  </div>
 );
}
```

FILE: docs/LEARNINGS_AND_CHANGES.md

Lifelong AI Personal Assistant: Learnings & Changes

This document serves as a comprehensive changelog and post-mortem detailing the architectural evolution and the profound Knowledge Graph extraction bugs we resolved during the development of this application.

Phase 1: Architectural Overhaul (From Relational to Knowledge Graph)
Initially, the application relied on a traditional relational database for tracking memories and tasks. This approach proved too rigid for a "Second Brain" that needed to understand fluid human thought and implicit relationships.
The Fix: We completely rebuilt the database schema into a highly dynamic Semantic Knowledge Graph.
- Introduced `canonical_entity` and `entity_alias` to manage real-world people/projects and all their nicknames/aliases.
- Introduced `entity_relation` to map connections (e.g., "Person A is MARRIED_TO Person B").
- Introduced `epistemic_claim` to track the AI's confidence in the facts it extracted.
- Introduced `semantic_chunk` to store vector embeddings of the user's raw thoughts for semantic search.
- Transitioned tasks into a `task_projection` table that naturally ties into the knowledge graph.

Phase 2: Asynchronous Processing & Extraction Pipeline
Because generating vector embeddings and pinging large language models takes time, we couldn't process thoughts synchronously without causing the UI to freeze.
The Fix: We decoupled the ingestion engine. The Next.js frontend now simply records the user's raw thought into a `system_event` table. A background process

(via `lib/kg-processor.ts`) silently picks up these events, generates vector embeddings using `xenova/all-MiniLM-L6-v2`, and queries Groq's LLMs to extract graph entities.

Phase 3: Eradicating Hallucinations and Hardcoded Data
The system was exhibiting severe hallucinations, specifically:
1. Creating phantom projects out of thin air.
2. Creating phantom tasks that the user never mentioned.
3. Injecting non-existent people into the database.
4. Failing to capture dense information (e.g., dropping multiple birthdays when provided in a single long paragraph).
5. Lazy formatting (repeating the title of a memory in the value and summary fields).

3.1. Eradicating Hardcoded Demo Scripts
During our investigation, we discovered that the previous developer had left behind hardcoded "demo" logic in the extraction pipeline. A function called `deterministicExtract()` was forcefully injecting tasks for fictional events based on simple keyword matches. Furthermore, functions like `addNamedEntities()` contained hardcoded arrays of names.
The Fix: We completely gutted these functions. The Knowledge Graph now relies purely on LLM intelligence to extract genuine entities dynamically.

3.2. Preventing "Prompt Bleeding"
Small LLMs are highly susceptible to mimicking their instructions when confused. Our extraction prompt explicitly used highly specific user data as examples to guide the model. When the user provided a short, ambiguous sentence, the model panicked and copy-pasted the examples from the prompt directly into the database.
The Fix: We rewrote all prompt instructions to use purely generic, abstract placeholders (e.g., `"Phase 1 Complete"`, `"John Smith"`).

3.3. Solving the "Information Density" Problem
When testing a dense paragraph containing multiple distinct dates and entities, the original model dropped several facts entirely. Small models suffer from "attention overload" when faced with dense data, causing them to truncate their extraction prematurely.
The Fix: Instead of implementing complex and brittle "sentence chunking" logic, we upgraded the underlying extraction engine to Groq's `llama-3.3-70b-versatile` model. The 70B model possesses the cognitive depth to effortlessly parse massive blocks of text and extract every single fact accurately in a single pass.

3.4. Enforcing Strict Schema Formatting
The LLM was originally repeating the "Title" of a memory into the "Value" and "Summary" fields because it wasn't strictly told what those fields were for.
The Fix: We added a strict directive to the prompt: *"For memories, 'title' must be a short category (e.g., 'Marriage date'), 'value' must be the exact fact/data (e.g., 'January 1st, 2020'), and 'summary' must be a full descriptive sentence."* This forces the LLM to separate the actual data points from the conversational fluff.

Phase 4: UI & Hierarchical Upgrades
Beyond the backend, the UI was upgraded to support Project Hierarchy and Data

Visualization.
- The **Tasks tab** (`components/tasks/TaskList.tsx`) was refactored to visually group all sub-tasks and milestones recursively under their respective parent Projects using collapsible accordion sections, massively improving the readability of complex workloads.
- The **Memory tab** was overhauled to directly query and beautifully render the semantic `memory_record` rows alongside their confidence scores.
- The **Chat tab** was updated to pull contextually relevant tasks and memories into the prompt so the AI can chat intelligently about the user's current life state.

FILE: lib/cognitive-load.ts

```
import { getSupabaseServer } from '@/lib/supabase/server';
import type { CognitiveLoad } from '@/lib/types';

// ── Defaults ──────────────────────────────────────────────────────────────

const DEFAULT_LOAD: CognitiveLoad = {
  open_tasks:      0,
  blocked_tasks:   0,
  active_tasks:    0,
  active_projects: 0,
  raw_load:        0,
  load_state:      'COMFORTABLE',
};

// ── Main fetch ────────────────────────────────────────────────────────────

/**
 * Fetch current cognitive load from the cognitive_load view.
 * Returns safe defaults on any error — load state should never crash the app.
 */
export async function getCognitiveLoad(): Promise<CognitiveLoad> {
  const supabase = getSupabaseServer();

  try {
    const { data, error } = await supabase
      .from('cognitive_load')
      .select('*')
      .single();

    if (error || !data) {
      // View returns no rows when task_projection is empty — that is fine
      return DEFAULT_LOAD;
```

```
    }

    const row = data as Record<string, unknown>;

    return {
      open_tasks:      Number(row['open_tasks'])      || 0,
      blocked_tasks:   Number(row['blocked_tasks'])   || 0,
      active_tasks:    Number(row['active_tasks'])    || 0,
      active_projects: Number(row['active_projects']) || 0,
      raw_load:        Number(row['raw_load'])        || 0,
      load_state:      (row['load_state'] as CognitiveLoad['load_state'])
                       ?? 'COMFORTABLE',
    };

  } catch (err) {
    console.error('[cognitive-load] getCognitiveLoad error:', err);
    return DEFAULT_LOAD;
  }
}

// ——— Load state helpers ———————————————————————————————————————————

/**
 * Returns a short human-readable label and colour hint
 * for use in UI components.
 */
export function describeLoadState(state: CognitiveLoad['load_state']): {
  label:     string;
  emoji:     string;
  colorClass: string;
} {
  switch (state) {
    case 'COMFORTABLE':
      return { label: 'Comfortable', emoji: '🟢', colorClass: 'text-emerald-400' };
    case 'FULL':
      return { label: 'Full',        emoji: '🟡', colorClass: 'text-yellow-400'  };
    case 'OVERLOADED':
      return { label: 'Overloaded',  emoji: '🟠', colorClass: 'text-orange-400'  };
    case 'CRITICAL':
      return { label: 'Critical',    emoji: '🔴', colorClass: 'text-red-400'     };
    default:
      return { label: 'Unknown',     emoji: '⚪', colorClass: 'text-slate-400'   };
  }
}

/**
```

```
 * Returns true if the system should warn before adding new commitments.
 */
export function isOverloaded(load: CognitiveLoad): boolean {
  return load.load_state === 'OVERLOADED' || load.load_state === 'CRITICAL';
}

──────────────────────────────────────────────────────────────────────────────
FILE: lib/context-assembly.ts
──────────────────────────────────────────────────────────────────────────────
import { getSupabaseServer } from '@/lib/supabase/server';
import {
  hybridRetrieval,
  getRecentChatEvents,
  getActiveTasks,
  getRelevantMemoryRecords,
  extractTaskIdsFromText,
  getTaskStatusById,
} from '@/lib/retrieval';
import { truncateToTokens, computeHash } from '@/lib/groq';
import type {
  ContextManifest,
  MemorySearchRecord,
  SemanticChunk,
  TaskProjection,
  LoadState,
  ChatMessage,
} from '@/lib/types';

// ─── Token budget ─────────────────────────────────────────────────────────────
const MODEL_MAX_TOKENS = 8000;
const BASE_BUDGET      = MODEL_MAX_TOKENS - 2800; // reserve for system,
overrides, entities, semantic

const BUDGET = {
  system:   500,
  overrides: 500,
  memories:  Math.floor(BASE_BUDGET * 0.40), // 40%
  tasks:     Math.floor(BASE_BUDGET * 0.40), // 40%
  entities:  800,
  semantic:  1000,
  chat:      Math.floor(BASE_BUDGET * 0.20), // 20%
} as const;

// ─── Context scoring weights ──────────────────────────────────────────────────
```

```
// v1: semantic only. Enable others one at a time after empirical validation.
// See architecture Section X and due diligence JJ-3.
const WEIGHTS = {
  semantic:        1.00,
  trust:           0.00, // TODO Phase 4: target 0.20
  temporal:        0.00, // TODO Phase 4: target 0.20
  goalRelevance:   0.00, // TODO Phase 4: target 0.15
  emotionalWeight: 0.00, // TODO Phase 4: target 0.10
} as const;

// ——— Cognitive load import (available after Step 9) ———————————————————————

// Dynamic import to avoid circular dependency during build sequence
async function getLoad() {
  try {
    const mod = await import('@/lib/cognitive-load');
    return mod.getCognitiveLoad();
  } catch {
    return null;
  }
}

// ——— Main assembly function ———————————————————————————————————————————————

export async function assembleContext(
  userQuery:       string,
  queryEmbedding?: number[]
): Promise<ContextManifest> {
  const startTime = Date.now();

  const [retrievalResult, chatEvents, activeTasks, memoryRecords, loadData] =
    await Promise.all([
      hybridRetrieval(userQuery, queryEmbedding).catch(() => ({
        chunks: [] as SemanticChunk[],
        latencyMs: 0,
        mode: 'keyword-only' as const,
      })),
      getRecentChatEvents(15).catch(() => []),
      getActiveTasks().catch(() => []),
      getRelevantMemoryRecords(userQuery, 12).catch(() => []),
      getLoad(),
    ]);

  // Stale context injection — two detection paths combined:
  // 1) Legacy UUID regex on raw text (rarely fires — AI responses don't
  //    naturally contain literal UUIDs)
  // 2) Metadata-based: assistant messages tag which tasks they discussed
  //    in metadata.referenced_task_ids at save time (see api/chat/route.ts)
  const chatHistoryText  = chatEvents.map(e => e.payload).join(' ');
```

```
const textBasedIds     = extractTaskIdsFromText(chatHistoryText);
const metadataBasedIds = chatEvents
  .flatMap(e => (e.metadata?.referenced_task_ids as string[] | undefined) ?? []);
const foundTaskIds     = [...new Set([...textBasedIds, ...metadataBasedIds])];
const taskStatuses     = await getTaskStatusById(foundTaskIds).catch(() => []);

const overrides: string[] = [];
for (const ts of taskStatuses) {
  if (ts.current_status === 'COMPLETED') {
    const completedDate = ts.completed_at
      ? new Date(ts.completed_at).toLocaleDateString()
      : 'recently';
    overrides.push(
      '[SYSTEM OVERRIDE] Task "' + ts.title + '" (ID: ' + ts.id + ') ' +
      'is COMPLETED as of ' + completedDate + '. ' +
      'Do not treat it as pending even if chat history says otherwise.'
    );
  } else if (ts.current_status === 'ABANDONED') {
    overrides.push(
      '[SYSTEM OVERRIDE] Task "' + ts.title + '" (ID: ' + ts.id + ') ' +
      'was ABANDONED. It is no longer active.'
    );
  }
}

const scoredChunks = scoreChunks(retrievalResult.chunks);
const topChunks    = scoredChunks.slice(0, 12);
const loadState    = (loadData?.load_state ?? 'COMFORTABLE') as LoadState;
const rawLoad      = loadData?.raw_load ?? 0;

const systemPrompt = buildSystemPrompt({
  overrides,
  activeTasks:  activeTasks as TaskProjection[],
  memoryRecords: memoryRecords as MemorySearchRecord[],
  topChunks,
  chatEvents,
  loadState,
  rawLoad,
});

const tokensUsed = Math.ceil(systemPrompt.length / 4);

void writeAssemblyLog({
  userQuery,
  chunksRetrieved:  retrievalResult.chunks.length,
  chunksInContext:  topChunks.length,
  topChunkIds:      topChunks.map(c => c.id),
  topChunkScores:   topChunks.map(c => c.attention_score),
  totalTokens:      tokensUsed,
  overrideCount:    overrides.length,
```

```
    tasksInContext:   activeTasks.length,
    loadState,
    latencyMs:        Date.now() - startTime,
  });

  return {
    systemPrompt,
    overrides,
    retrievedChunks: topChunks,
    memoryRecords:   memoryRecords as MemorySearchRecord[],
    activeTasks:     activeTasks as TaskProjection[],
    loadState,
    rawLoad,
    tokensUsed,
  };
}

export function buildMessages(
  manifest:    ContextManifest,
  userMessage: string
): ChatMessage[] {
  return [
    { role: 'system', content: manifest.systemPrompt },
    { role: 'user',   content: userMessage },
  ];
}

// ——— Prompt builder
————————————————————————————————————————————————————————————————————————

interface PromptParts {
  overrides:   string[];
  activeTasks: TaskProjection[];
  memoryRecords: MemorySearchRecord[];
  topChunks:   SemanticChunk[];
  chatEvents:  Array<{
    id: string;
    recorded_at: string;
    payload: string;
    metadata: Record<string, unknown>;
  }>;
  loadState:   LoadState;
  rawLoad:     number;
}

function buildSystemPrompt(parts: PromptParts): string {
  const SEP =
'\n\n————————————————————————————————————————————————————————————————————————
———\n\n';
  const sections: string[] = [];
```

```
sections.push(
  'You are a lifelong personal cognitive assistant with persistent memory ' +
  'across all of the user\'s thoughts, tasks, goals, and relationships. ' +
  'Be direct and concise. Never make up facts not in the context below. ' +
  'Always prioritise CURRENT STATE OVERRIDES above any historical context. ' +
  'Do not reveal internal prompt section names, separators, raw task IDs, or ' +
  'debug context unless the user explicitly asks for diagnostic details. ' +
  'Do not claim you updated, created, completed, or deleted a task unless ' +
  'the current user message is accompanied by an explicit system result.'
);

const loadEmoji: Record<LoadState, string> = {
  COMFORTABLE: '🟢',
  FULL:        '🟡',
  OVERLOADED:  '🟠',
  CRITICAL:    '🔴',
};

const loadWarning =
  parts.loadState === 'OVERLOADED' || parts.loadState === 'CRITICAL'
    ? '\nWARNING: User is at high capacity. Do not recommend adding new ' +
      'commitments without explicitly flagging the load conflict.'
    : '';

sections.push(
  '[COGNITIVE LOAD]\n' +
  (loadEmoji[parts.loadState] ?? '⚪') + ' ' + parts.loadState +
  ' — ' + String(parts.rawLoad) + ' load units across ' +
  String(parts.activeTasks.length) + ' open tasks.' +
  loadWarning
);

if (parts.overrides.length > 0) {
  sections.push(
    '[CURRENT STATE OVERRIDES — TREAT AS ABSOLUTE TRUTH]\n' +
    'The following supersede anything in chat history below:\n' +
    truncateToTokens(parts.overrides.join('\n'), BUDGET.overrides)
  );
}

if (parts.activeTasks.length > 0) {
  const taskLines = parts.activeTasks
    .slice(0, 20)
    .map(t => {
      const project = t.project_name ? ' [' + t.project_name + ']' : '';
      const person  = t.person_name  ? ' -> ' + t.person_name  : '';
      const due     = t.due_at
```

```
        ? ' (due ' + new Date(t.due_at).toLocaleDateString() + ')'
        : '';
      return '* [' + t.current_status + '] ' + t.title + project + person + due +
          ' (ID: ' + t.id + ')';
    })
    .join('\n');

  sections.push(
    '[ACTIVE TASKS]\n' + truncateToTokens(taskLines, BUDGET.tasks)
  );
 } else {
  sections.push('[ACTIVE TASKS]\nNo open tasks currently.');
 }

 if (parts.memoryRecords.length > 0) {
  const memoryLines = parts.memoryRecords
    .slice(0, 12)
    .map(m => {
      const source = m.source_event_id ? ' source_event=' + m.source_event_id : '';
      const value  = m.value ? ' — ' + m.value : '';
      return '* [' + m.memory_type + '] ' + m.title + value +
          ' :: ' + m.summary + source;
    })
    .join('\n');

  sections.push(
    '[DURABLE MEMORY — FACTS, RELATIONSHIPS, DECISIONS]\n' +
    'Prefer these structured memories over loose historical text when answering
exact questions.\n' +
    truncateToTokens(memoryLines, BUDGET.memories)
  );
 }

 if (parts.topChunks.length > 0) {
  const memoryLines = parts.topChunks
    .map((c, i) => '[Memory ' + String(i + 1) + '] ' + c.chunk_text.slice(0, 400))
    .join('\n\n');
  sections.push(
    '[RELEVANT MEMORY]\n' + truncateToTokens(memoryLines,
BUDGET.semantic)
  );
 }

 if (parts.chatEvents.length > 0) {
  const recent    = parts.chatEvents.slice(-16);
  const chatLines = recent
    .map(e => {
      const time = new Date(e.recorded_at).toLocaleTimeString([], {
        hour: '2-digit', minute: '2-digit',
      });
```

```
        return '[' + time + '] ' + e.payload.slice(0, 300);
      })
      .join('\n');
    sections.push(
      '[RECENT CONVERSATION]\n' + truncateToTokens(chatLines, BUDGET.chat)
    );
  }

  return sections.join(SEP);
}

// ——— Chunk scoring ———————————————————————————————————————————————————

function scoreChunks(chunks: SemanticChunk[]): SemanticChunk[] {
  return chunks
    .map(chunk => {
      const semanticSim    = chunk.attention_score ?? 0;
      const trustScore     = 0.5; // TODO Phase 4
      const temporalScore  = 0.5; // TODO Phase 4
      const goalRelevance  = 0.5; // TODO Phase 4
      const emotionalW     = 0.5; // TODO Phase 4

      const score =
        (semanticSim   * WEIGHTS.semantic)        +
        (trustScore    * WEIGHTS.trust)           +
        (temporalScore * WEIGHTS.temporal)        +
        (goalRelevance * WEIGHTS.goalRelevance)   +
        (emotionalW    * WEIGHTS.emotionalWeight);

      return { ...chunk, attention_score: score };
    })
    .sort((a, b) => b.attention_score - a.attention_score);
}

// ——— Assembly logging ————————————————————————————————————————————————

interface AssemblyLogParams {
  userQuery:        string;
  chunksRetrieved:  number;
  chunksInContext:  number;
  topChunkIds:      string[];
  topChunkScores:   number[];
  totalTokens:      number;
  overrideCount:    number;
  tasksInContext:   number;
  loadState:        LoadState;
  latencyMs:        number;
}
```

```
async function writeAssemblyLog(params: AssemblyLogParams): Promise<void> {
  const supabase = getSupabaseServer();
  try {
    await supabase.from('context_assembly_log').insert({
      user_query:        params.userQuery.slice(0, 1000),
      query_hash:        computeHash(params.userQuery),
      chunks_retrieved:  params.chunksRetrieved,
      chunks_in_context: params.chunksInContext,
      top_chunk_ids:     params.topChunkIds,
      top_chunk_scores:  params.topChunkScores,
      total_tokens:      params.totalTokens,
      override_count:    params.overrideCount,
      tasks_in_context:  params.tasksInContext,
      load_state:        params.loadState,
    });
  } catch (err) {
    console.error('[context-assembly] writeAssemblyLog error:', err);
  }
}
```

FILE: lib/embeddings.ts

```
// Browser-only module — never runs on the server
// Guard at top prevents accidental server-side import

import type { EmbeddingBatch } from '@/lib/types';

// ── Model config
// ─────────────────────────────────────────────────────────────────────────

const MODEL_NAME    = 'Xenova/all-MiniLM-L6-v2';
const MODEL_VERSION = 'xenova/all-MiniLM-L6-v2@2.17.2';
const DIMENSIONS    = 384;
const BATCH_SIZE    = 5;   // chunks sent to /api/embeddings per request
const SYNC_LIMIT    = 20;  // max pending chunks fetched per sync run

// ── Singleton model reference
// ─────────────────────────────────────────────────────────────────────────

// eslint-disable-next-line @typescript-eslint/no-explicit-any
let _pipeline: any = null;
let _loading       = false;
let _loaded        = false;

// ── Load model
```

```
// ——— Load model ——————————————————————————————————————————————

/**
 * Load Xenova/all-MiniLM-L6-v2 into browser memory.
 * ~23MB download on first load, then served from browser cache.
 * Runs silently — no UI feedback intentional.
 * Safe to call multiple times — loads only once.
 */
export async function loadEmbeddingModel(): Promise<void> {
  if (typeof window === 'undefined') return;
  if (_loaded || _loading) return;

  _loading = true;

  try {
    // Dynamic import keeps Transformers.js out of the server bundle entirely
    const { pipeline, env } = await import('@xenova/transformers');

    // Use local model cache — avoid re-downloading on every page load
    env.allowLocalModels  = false;
    env.useBrowserCache   = true;

    _pipeline = await pipeline('feature-extraction', MODEL_NAME, {
      quantized: true,  // smaller model, faster load, minimal accuracy loss
    });

    _loaded  = true;
    _loading = false;

    console.info('[embeddings] Model loaded:', MODEL_NAME);
  } catch (err) {
    _loading = false;
    console.error('[embeddings] Failed to load model:', err);
  }
}

// ——— Generate single embedding ——————————————————————————————————

/**
 * Generate a 384-dimension embedding for a text string.
 * Returns null if model is not loaded — caller should fall back to
 * keyword-only retrieval in that case.
 */
export async function generateEmbedding(text: string): Promise<number[] | null> {
  if (typeof window === 'undefined') return null;
  if (!_loaded || !_pipeline)       return null;

  try {
    const sanitised = text.trim().slice(0, 512); // MiniLM max context
```

```
    const output = await _pipeline(sanitised, {
      pooling:   'mean',
      normalize: true,
    });

    // Convert Float32Array to regular number[]
    const embedding = Array.from(output.data as Float32Array) as number[];

    if (embedding.length !== DIMENSIONS) {
      console.error('[embeddings] Unexpected dimensions:', embedding.length);
      return null;
    }

    return embedding;

  } catch (err) {
    console.error('[embeddings] generateEmbedding error:', err);
    return null;
  }
}

// ── Sync pending embeddings ──────────────────────────────────────────────────

/**
 * Find semantic_chunk rows with NULL embeddings, generate embeddings
 * in the browser, and upload them to Supabase via /api/embeddings.
 *
 * Runs silently in background on every app open.
 * Uses batching to avoid overwhelming the API route.
 */
export async function syncPendingEmbeddings(): Promise<boolean> {
  if (typeof window === 'undefined') return false;
  if (!_loaded)                      return false;

  try {
    // Fetch pending chunks directly from Supabase client-side
    const { createClient } = await import('@supabase/supabase-js');

    const url  = process.env.NEXT_PUBLIC_SUPABASE_URL;
    const anon = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

    if (!url || !anon) return false;

    const supabase = createClient(url, anon, {
      auth: { persistSession: false, autoRefreshToken: false },
    });

    const { data: pending, error } = await supabase
```

```
  .from('semantic_chunk')
  .select('id, originating_event_id, chunk_text')
  .is('embedding', null)
  .limit(SYNC_LIMIT);

if (error || !pending || pending.length === 0) return false;

console.info('[embeddings] Syncing', pending.length, 'pending chunks...');

type PendingRow = {
  id:                string;
  originating_event_id: string;
  chunk_text:        string;
};

const rows = pending as PendingRow[];

// Process in batches
for (let i = 0; i < rows.length; i += BATCH_SIZE) {
  const batch = rows.slice(i, i + BATCH_SIZE);

  const embeddingBatch: EmbeddingBatch[] = [];

  for (const row of batch) {
    const embedding = await generateEmbedding(row.chunk_text);
    if (embedding) {
      embeddingBatch.push({
        event_id:  row.originating_event_id,
        embedding,
      });
    }
  }

  if (embeddingBatch.length === 0) continue;

  // Upload to server
  try {
    const res = await fetch('/api/embeddings', {
      method:  'POST',
      headers: { 'Content-Type': 'application/json' },
      body:    JSON.stringify({ chunks: embeddingBatch }),
    });

    if (!res.ok) {
      console.error('[embeddings] Upload failed:', res.status);
    }
  } catch (uploadErr) {
    console.error('[embeddings] Upload error:', uploadErr);
  }
```

```
      // Small pause between batches — don't hammer the API
      await new Promise(resolve => setTimeout(resolve, 300));
    }

    console.info('[embeddings] Sync complete');
    return true;

  } catch (err) {
    console.error('[embeddings] syncPendingEmbeddings error:', err);
    return false;
  }
}

// ─── Model info ──────────────────────────────────────────────────────────────

export function getModelVersion(): string {
  return MODEL_VERSION;
}

export function isModelLoaded(): boolean {
  return _loaded;
}

// ═══════════════════════════════════════════════════════════════════════════
// FILE: lib/groq.ts
// ═══════════════════════════════════════════════════════════════════════════
import Groq from 'groq-sdk';
import { createHash } from 'crypto';
import type { ChatMessage, ExtractionResult } from '@/lib/types';

// ─── Env validation ──────────────────────────────────────────────────────────
const groqApiKey = process.env.GROQ_API_KEY;

// ─── Model constants ─────────────────────────────────────────────────────────
// All model names live here — never hardcoded anywhere else in the codebase
export const MODELS = {
  CHAT:      'llama-3.3-70b-versatile',   // reasoning, decision support
  EXTRACT:   'llama-3.3-70b-versatile',   // accurate JSON extraction
} as const;

// ─── Client singleton ────────────────────────────────────────────────────────
```

```
let groq: Groq | null = null;

function getGroqClient(): Groq {
  if (groq) return groq;
  if (!groqApiKey) throw new Error('Missing GROQ_API_KEY');
  groq = new Groq({ apiKey: groqApiKey });
  return groq;
}

// ——— Extraction schema prompt
——————————————————————————————————————————————————————————————————————

const EXTRACTION_SYSTEM_PROMPT = `You are a precise structured data
extractor.
Return ONLY valid JSON matching the schema below.
Extract only what is explicitly stated in the note.
Do not invent, infer beyond what is written, or hallucinate.
If nothing matches a field, use an empty array.
Maximum 5 entities per note. Maximum 3 tasks per note.

JSON SCHEMA:
{
  "tasks": [{
    "title": "string (max 100 chars, imperative verb form)",
    "project_hint": "string or null",
    "person_hint": "string or null",
    "deadline_hint": "string or null",
    "effort": 1
  }],
  "entities": [{
    "name": "string (proper noun only)",
    "type": "PERSON|PROJECT|GOAL|CONCEPT|ORG",
    "context_snippet": "string (max 80 chars)"
  }],
  "relationships": [{
    "from_name": "string",
    "predicate": "string (e.g. WORKS_WITH, MANAGES, PART_OF)",
    "to_name": "string"
  }],
  "aliases": [{
    "canonical_name": "string",
    "alias": "string (e.g. nickname, role, title)"
  }]
}

effort scale: 1=trivial 2=normal 3=significant 4=major 5=project-defining`;

// ——— Extraction
——————————————————————————————————————————————————————————————————————
```

```
/**
 * Extract structured entities and tasks from a raw thought payload.
 * Uses llama-3.3-70b-versatile in JSON mode.
 * Never throws — returns empty arrays on any failure.
 */
export async function extractFromPayload(
  payload: string
): Promise<ExtractionResult> {
  const empty: ExtractionResult = {
    tasks: [], entities: [], relationships: [], aliases: [],
  };

  // Sanitise input before sending to LLM
  const sanitised = payload
    .replace(/<[^>]*>/g, '')           // strip HTML
    .replace(/[\u0000-\u001F]/g, ' ')  // strip control chars
    .trim()
    .slice(0, 8000);                   // hard cap

  if (sanitised.length < 10) return empty;

  const attempt = async (): Promise<ExtractionResult> => {
    const response = await getGroqClient().chat.completions.create({
      model:           MODELS.EXTRACT,
      temperature:     0.1,
      max_tokens:      1024,
      response_format: { type: 'json_object' },
      messages: [
        { role: 'system', content: EXTRACTION_SYSTEM_PROMPT },
        { role: 'user',   content: `NOTE: """${sanitised}"""` },
      ],
    });

    const raw = response.choices[0]?.message?.content ?? '{}';
    const parsed = JSON.parse(raw) as Partial<ExtractionResult>;

    // Validate and clamp
    const result: ExtractionResult = {
      tasks: (Array.isArray(parsed.tasks)
        ? parsed.tasks.slice(0, 3)
        : []
      ).filter(t => typeof t.title === 'string' && t.title.length > 0),

      entities: (Array.isArray(parsed.entities)
        ? parsed.entities.slice(0, 5)
        : []
      ).filter(e => typeof e.name === 'string' && e.name.length > 0),

      relationships: (Array.isArray(parsed.relationships)
```

```
        ? parsed.relationships.slice(0, 5)
        : []
      ).filter(r =>
        typeof r.from_name === 'string' && r.from_name.length > 0 &&
        typeof r.predicate === 'string' && r.predicate.length > 0 &&
        typeof r.to_name   === 'string' && r.to_name.length > 0
      ),
      aliases: (Array.isArray(parsed.aliases)
        ? parsed.aliases.slice(0, 5)
        : []
      ).filter(a =>
        typeof a.canonical_name === 'string' && a.canonical_name.length > 0 &&
        typeof a.alias === 'string' && a.alias.length > 0
      ),
    };

    return result;
  };

  try {
    return await attempt();
  } catch (firstErr) {
    // Exponential backoff — retry once on rate limit (429)
    const status = (firstErr as { status?: number }).status;
    if (status === 429) {
      await new Promise(r => setTimeout(r, 2000));
      try {
        return await attempt();
      } catch {
        return empty;
      }
    }
    console.error('[groq] extractFromPayload error:', firstErr);
    return empty;
  }
}

// ─── Task Intent Parsing (Chat API) ──────────────────────────────────────────

export interface TaskMutationIntent {
  mutated: boolean;
  status: 'COMPLETED' | 'BLOCKED' | 'ABANDONED' | 'ACTIVE' | null;
  target_task_title: string | null;
}

const INTENT_SYSTEM_PROMPT = `You are a strict intent parser.
Determine if the user's message is instructing you to close, complete, cancel, or
block an existing task.
```

```
If the user is just asking a question (e.g. "did I do X?"), mutated = false.
If the user says "I completed X", mutated = true, status = COMPLETED.
If the user says "mark X as blocked", mutated = true, status = BLOCKED.
If the user says "abandon X" or "cancel X", mutated = true, status = ABANDONED.
If they implicitly refer to a task (e.g. "yes mark it done"), extract what you can or
leave target_task_title null.

JSON SCHEMA:
{
  "mutated": boolean,
  "status": "COMPLETED" | "BLOCKED" | "ABANDONED" | "ACTIVE" | null,
  "target_task_title": "string" | null
}`;

export async function determineTaskMutationIntent(
  message: string,
  openTasks: { title: string }[]
): Promise<TaskMutationIntent> {
  const empty: TaskMutationIntent = { mutated: false, status: null, target_task_title:
null };
  const sanitised = message.replace(/<[^>]*>/g, '').trim().slice(0, 1000);
  if (sanitised.length < 2) return empty;

  const attempt = async () => {
    const response = await getGroqClient().chat.completions.create({
      model: MODELS.EXTRACT,
      temperature: 0.1,
      max_tokens: 128,
      response_format: { type: 'json_object' },
      messages: [
        { role: 'system', content: INTENT_SYSTEM_PROMPT },
        { role: 'user', content: `Message: "${sanitised}"\nOpen Tasks Context: $
{JSON.stringify(openTasks.map(t => t.title))}` },
      ],
    });

    const raw = response.choices[0]?.message?.content ?? '{}';
    const parsed = JSON.parse(raw);
    return {
      mutated: !!parsed.mutated,
      status: ['COMPLETED', 'BLOCKED', 'ABANDONED',
'ACTIVE'].includes(parsed.status) ? parsed.status : null,
      target_task_title: parsed.target_task_title || null,
    };
  };

  try {
    return await attempt();
  } catch (err) {
    console.error('[groq] determineTaskMutationIntent error:', err);
```

```
    return empty;
  }
}

// ——— Chat streaming ———————————————————————————————————————————————

/**
 * Stream a chat response from llama-3.3-70b-versatile.
 * Returns a ReadableStream of text chunks.
 * Caller is responsible for writing to response cache after stream completes.
 */
export async function streamChatResponse(
  messages: ChatMessage[]
): Promise<ReadableStream<Uint8Array>> {
  const encoder = new TextEncoder();

  const groqStream = await getGroqClient().chat.completions.create({
    model:       MODELS.CHAT,
    temperature: 0.7,
    max_tokens:  2048,
    stream:      true,
    messages:    messages.map(m => ({
      role:    m.role,
      content: m.content,
    })),
  });

  return new ReadableStream<Uint8Array>({
    async start(controller) {
      try {
        for await (const chunk of groqStream) {
          const text = chunk.choices[0]?.delta?.content ?? '';
          if (text) {
            controller.enqueue(encoder.encode(text));
          }
        }
      } catch (err) {
        console.error('[groq] streamChatResponse error:', err);
        controller.enqueue(
          encoder.encode('\n\n[Error: response interrupted]')
        );
      } finally {
        controller.close();
      }
    },
  });
}
```

```
// ——— Utilities
——————————————————————————————————————————————————————————

/**
 * SHA-256 hex hash of any string.
 * Used for cache keying in response_cache table.
 */
export function computeHash(text: string): string {
  return createHash('sha256').update(text).digest('hex');
}

/**
 * Truncate a string to approximately maxTokens tokens.
 * Rough estimate: 1 token ≈ 4 characters.
 */
export function truncateToTokens(text: string, maxTokens: number): string {
  const maxChars = maxTokens * 4;
  if (text.length <= maxChars) return text;
  return text.slice(0, maxChars) + '… [truncated]';
}


——————————————————————————————————————————————————————————
————————————————————
FILE: lib/kg-processor.ts
——————————————————————————————————————————————————————————
————————————————————
import { SupabaseClient } from '@supabase/supabase-js';
import type { ContextManifest } from '@/lib/types';
import Groq from 'groq-sdk';

const GROQ_API_KEY   = process.env.GROQ_API_KEY;
let groqClient: Groq | null = null;
function getGroqClient() {
  if (!groqClient) {
    if (!GROQ_API_KEY) throw new Error('Missing GROQ_API_KEY');
    groqClient = new Groq({ apiKey: GROQ_API_KEY });
  }
  return groqClient;
}

const EXTRACT_MODEL  = 'llama-3.3-70b-versatile';

const EXTRACTION_PROMPT = `You are a precise lifelong-memory and task graph
extractor.
You will be provided with a user message, along with the CURRENT ACTIVE
GRAPH STATE (open tasks, memories, entities).
Your job is to determine what graph mutations should occur based ONLY on the new
user message.
```

Return ONLY valid JSON. Extract only what is explicitly stated or directly implied. Do not invent or hallucinate. Empty arrays if nothing found.

JSON SCHEMA:

{
  "tasks": [{"title":"string","status_update":"COMPLETED|BLOCKED|ABANDONED|ACTIVE|null","project_hint":"string|null","person_hint":"string|null","deadline_hint":"string|null","effort":2}],
  "entities": [{"name":"string","type":"PERSON|PROJECT|GOAL|CONCEPT|ORG","context_snippet":"string"}],
  "relationships": [{"from_name":"string","predicate":"string","to_name":"string"}],
  "memories": [{"type":"PROFILE_FACT|PREFERENCE|RELATIONSHIP|LIFE_EVENT|ROUTINE|DECISION|COMMITMENT|REMINDER|KNOWLEDGE|GOAL","title":"string","value":"string|null","summary":"string","confidence":0.85}],
  "reminders": [{"title":"string","reminder_at":"ISO timestamp|null","due_hint":"string|null"}],
  "aliases": [{"canonical_name":"string (The true full identity)","alias":"string (The alternate name, pet name, or nickname itself)"}]
}

Rules:
1. CRITICAL: If the user's message is a question or a pure statement of fact, do NOT extract tasks. ONLY extract a task if there is a clear actionable chore, todo, or future action the user intends to complete.
2. TASKS vs PROJECTS vs MILESTONES: A project is a large undertaking (e.g., "Q3 Marketing Campaign"). A task is an actionable item. A milestone is an achievement (e.g., "Phase 1 Complete"). Extract milestones and sub-steps as regular 'tasks' with the 'project_hint' set to the exact name of the parent project. Do NOT mark a parent project as COMPLETED just because a milestone/sub-task was met. Do NOT elevate small tasks/milestones to 'PROJECT' entities.
3. SELF-REFERENCE: Resolve all first-person pronouns (I, me, my) to the literal entity name "User". If the user states their actual name (e.g., "My name is John"), extract their real name as an alias for "User", but keep the canonical name as "User".
4. HONORIFICS: Automatically strip cultural honorifics, titles, and suffixes (e.g., San, Ji, Mr., Dr., Sir) from canonical entity names (e.g., "John Smith Mr." becomes "John Smith"). You may put the honorific in the aliases array if necessary.
5. ALIASES: An alias is an alternate name for the EXACT SAME entity. "canonical_name" must be the true identity, and "alias" must be the alternate name/pet name itself. Do NOT use the context (like "home" or "pet name") as the alias string.
6. RELATIONSHIPS: Only extract structural, logical predicates between nouns (e.g. PARENT_OF, MARRIED_TO, ASSIGNED_TO). Do NOT use conversational verbs from the sentence as predicates if they don't make structural sense.
7. ENTITIES vs MEMORIES: Extracting an entity is NOT enough. ANY detail, fact, preference, or event about an entity MUST be duplicated into the "memories" array. For memories, "title" must be a short category (e.g., "Marriage date"), "value" must be the exact fact/data (e.g., "27th January 2016"), and "summary" must be a full descriptive sentence (e.g., "User and Priya were married on 27th January 2016"). Do

NOT just repeat the title in all three fields.
8. effort: 1=trivial 2=normal 3=significant 4=major 5=project-defining`;

```
interface ExtractedData {
  tasks: {
    title:         string;
    status_update?: string | null;
    project_hint:  string | null;
    person_hint:   string | null;
    deadline_hint: string | null;
    effort:        number;
  }[];
  entities: {
    name:            string;
    type:            string;
    context_snippet: string;
  }[];
  relationships: {
    from_name: string;
    predicate: string;
    to_name:   string;
  }[];
  memories: {
    type:       string;
    title:      string;
    value:      string | null;
    summary:    string;
    confidence: number;
  }[];
  reminders: {
    title:       string;
    reminder_at: string | null;
    due_hint:    string | null;
  }[];
  aliases: {
    canonical_name: string;
    alias:          string;
  }[];
}


interface ExistingTaskRow {
  id:                  string;
  originating_event_id: string | null;
  title:               string;
  task_key:            string | null;
  estimated_effort:    number | null;
  due_at:              string | null;
  metadata:            Record<string, unknown> | null;
}
```

```
interface ExistingMemoryRow {
  id:                 string;
  confidence_score:     number | null;
  metadata:             Record<string, unknown> | null;
  associated_entity_id: string | null;
}

export async function processKnowledgeGraph(
  supabase: SupabaseClient,
  eventId: string,
  payload: string,
  sourceTab: string,
  contextManifest: ContextManifest
): Promise<unknown> {
  const startTime = Date.now();

  try {
    if (sourceTab === 'SYSTEM') {
      return { success: true, skipped: true };
    }

    const sanitised = payload
      .replace(/<[^>]*>/g, '')
      .replace(/[\u0000-\u001F]/g, ' ')
      .trim()
      .slice(0, 6000);

    if (sanitised.length < 5) {
      return { success: true, skipped: true };
    }

    const extracted = mergeExtractedData(
      cleanExtractedData(await callGroq(sanitised, contextManifest), sanitised),
      deterministicExtract(sanitised, new Date().toISOString())
    );

    // ——— Aliases ———————————————————————————————————————————————————————

    for (const item of extracted.aliases) {
      if (!item.canonical_name || !item.alias) continue;
      const cName = item.canonical_name.trim();
      const aliasName = item.alias.trim();
      if (!cName || !aliasName) continue;

      const canonicalId = await getOrCreateEntity(
        supabase,
        cName,
        'PERSON',
```

```
      null
    );
    if (canonicalId) {
      await supabase.from('entity_alias').upsert({
        canonical_id: canonicalId,
        alias_expression: aliasName,
        confidence_score: 0.95,
        merge_reviewed: true
      }, { onConflict: 'canonical_id,alias_expression' }).then(undefined, () => null);

    }
  }

  // ─── Tasks ──────────────────────────────────────────────────────────────────────
  ──────────────────────────────────────────────────────────────

  let tasksCreated = 0;
  for (const task of extracted.tasks) {
    if (!task.title || task.title.trim().length === 0) continue;
    const title = task.title.trim().slice(0, 512);
    const taskKey = canonicalKey(title);
    if (!taskKey) continue;

    const effort = Math.min(5, Math.max(1, task.effort ?? 2));
    const existingTask = await findExistingOpenTask(supabase, title, taskKey);

    const validStatuses = ['PENDING', 'COMPLETED', 'BLOCKED', 'ABANDONED',
'ACTIVE'];
    const targetStatus = task.status_update &&
validStatuses.includes(task.status_update)
      ? task.status_update
      : 'PENDING';

    const taskData: Record<string, unknown> = {
      originating_event_id: eventId,
      title:                title,
      task_key:             taskKey,
      current_status:       targetStatus,
      estimated_effort:     effort,
      completed_at:         targetStatus === 'COMPLETED' ? new
Date().toISOString() : null,
      metadata:             {
        deadline_hint: task.deadline_hint,
        last_seen_event_id: eventId,
      },
    };

    const dueAt = parseIsoDateHint(task.deadline_hint);
    if (dueAt) taskData['due_at'] = dueAt;
```

```
// ——— PHASE 1: Resolve project and person entities, creating if needed ———
if (task.project_hint) {
  let projectId = await findEntityId(supabase, task.project_hint);
  if (!projectId) {
    projectId = await getOrCreateEntity(
      supabase,
      task.project_hint,
      'PROJECT',
      'Inferred from task context'
    );
  }
  if (projectId) taskData['associated_project'] = projectId;
}

if (task.person_hint) {
  let personId = await findEntityId(supabase, task.person_hint);
  if (!personId) {
    personId = await getOrCreateEntity(
      supabase,
      task.person_hint,
      'PERSON',
      'Inferred from task context'
    );
  }
  if (personId) taskData['assigned_person'] = personId;
}

if (existingTask) {
  const updatePayload: Record<string, unknown> = {
    estimated_effort: Math.max(existingTask.estimated_effort ?? 1, effort),
    due_at: dueAt ?? existingTask.due_at,
    updated_at: new Date().toISOString(),
    metadata: {
      ...(existingTask.metadata ?? {}),
      deadline_hint: task.deadline_hint,
      last_seen_event_id: eventId,
      task_key: taskKey,
    },
  };

  if (targetStatus !== 'PENDING') {
    updatePayload.current_status = targetStatus;
    if (targetStatus === 'COMPLETED') {
      updatePayload.completed_at = new Date().toISOString();
    }
  }

  const { error: updateTaskError } = await supabase
    .from('task_projection')
    .update(updatePayload)
```

```
        .eq('id', existingTask.id);

      if (updateTaskError) {
        throw new Error('Task update failed: ' + updateTaskError.message);
      }
    } else {
      const { error: insertTaskError } = await supabase
        .from('task_projection')
        .insert(taskData);

      if (insertTaskError) {
        throw new Error('Task insert failed: ' + insertTaskError.message);
      }
    }
    tasksCreated++;
  }

  // ─── Entities ──────────────────────────────────────────────────────────────

  let entitiesCreated = 0;
  const entityMap: Record<string, string> = {};

  for (const entity of extracted.entities) {
    if (!entity.name || entity.name.trim().length === 0) continue;
    const name = entity.name.trim();
    const canonicalId = await getOrCreateEntity(
      supabase,
      name,
      entity.type ?? 'CONCEPT',
      entity.context_snippet?.slice(0, 200) ?? null
    );

    if (!canonicalId) continue;

    entityMap[name] = canonicalId;
    entitiesCreated++;

    // ─── Identity resolution: flag potential duplicates via pg_trgm ──────────
    // NEVER auto-merges. Catches spelling variants/typos only —
    // nickname pairs (Mike/Michael) are NOT character-similar and remain
    // a known limitation requiring manual review. See architecture JJ-3.
    const { data: similar } = await supabase.rpc('find_similar_entities', {
      p_name:       name,
      p_exclude_id: canonicalId,
    });

    for (const candidate of (similar ?? []) as
      { id: string; canonical_name: string; sim: number }[]
```

```
    ) {
      if (
        candidate.sim >= 0.45 &&
        candidate.id !== canonicalId &&
        candidate.canonical_name.trim().toLowerCase() !== name.toLowerCase()
      ) {
        await supabase.from('identity_flag').upsert({
          entity_a_id: canonicalId,
          entity_b_id: candidate.id,
          similarity:  candidate.sim,
          flag_reason: 'Similar name (trigram): "' + name + '" vs "' +
                       candidate.canonical_name + '"',
          resolution:  'PENDING',
        }, { onConflict: 'entity_a_id,entity_b_id' }).then(undefined, () => null);
      }
    }
  }

  // ——— Relationships + matching epistemic claims ———————————————————————

  for (const rel of extracted.relationships) {
    if (!rel.from_name || !rel.predicate || !rel.to_name) continue;

    const fromId = entityMap[rel.from_name]
      ?? await findEntityId(supabase, rel.from_name);
    const toId   = entityMap[rel.to_name]
      ?? await findEntityId(supabase, rel.to_name);

    if (!fromId || !toId || fromId === toId) continue;

    const predicate = rel.predicate.toUpperCase()
      .replace(/\s+/g, '_').slice(0, 100);
    const validFrom = new Date().toISOString() ?? new Date().toISOString();

    await supabase.from('entity_relation').insert({
      source_entity_id:   fromId,
      target_entity_id:   toId,
      predicate:          predicate,
      valid_from:         validFrom,
      assertion_event_id:  eventId,
      confidence_score:    0.700,
    }).then(undefined, () => null);

    // FIX: write the matching epistemic_claim — required for
    // arbitrate_truth() to ever return anything but UNKNOWN.
    // v1 simplified: decay/reverification fields left for Phase 4.
    await supabase.from('epistemic_claim').insert({
      claim_text:         rel.from_name + ' ' + rel.predicate + ' ' + rel.to_name,
      claim_type:          'INFERENCE',
      trust_level:         'AI_INFERRED',
```

```
        subject_entity_id:     fromId,
        source_event_id:       eventId,
        extracted_by_model:    EXTRACT_MODEL,
        confidence_score:      0.700,
        valid_from:            validFrom,
      }).then(undefined, () => null);
    }

    // ——— Durable memory records
————————————————————————————————————————————————————————————————————————

    let memoriesCreated = 0;
    for (const memory of extracted.memories) {
      if (!memory.title || !memory.summary) continue;

      const memoryType = normaliseMemoryType(memory.type);
      const confidence = clampConfidence(memory.confidence ?? 0.7);
      const reviewStatus = confidence < 0.65 ? 'NEEDS_REVIEW' :
'AUTO_ACCEPTED';
      const memoryTitle = memory.title.trim().slice(0, 255);
      const memoryKey = canonicalKey(memoryType + ' ' + memoryTitle);
      if (!memoryKey) continue;

      // ——— PHASE 1: Infer associated entity from memory content
————————————————————————

      let associatedEntityId: string | null = null;
      if (memory.value) {
        associatedEntityId = await inferEntityFromMemory(
          supabase,
          memoryTitle,
          String(memory.value),
          memory.summary
        );
      }

      const existingMemory = await findExistingActiveMemory(
        supabase,
        memoryType,
        memoryKey
      );

      if (existingMemory) {
        const { error: updateMemoryError } = await supabase
          .from('memory_record')
          .update({
            value: memory.value ? String(memory.value).trim().slice(0, 2000) : null,
            summary: memory.summary.trim().slice(0, 2000),
            confidence_score: Math.max(existingMemory.confidence_score ?? 0,
confidence),
            review_status: reviewStatus,
```

```
          associated_entity_id: associatedEntityId ??
existingMemory.associated_entity_id,
          updated_at: new Date().toISOString(),
          metadata: {
            ...(existingMemory.metadata ?? {}),
            raw_type: memory.type,
            last_seen_event_id: eventId,
          },
        })
        .eq('id', existingMemory.id);

      if (!updateMemoryError) memoriesCreated++;
      continue;
    }

    // Epistemic Conflict Resolution: If this is a new memory for a known entity,
    // check if it heavily overlaps with an existing active memory title.
    // If so, deprecate the old one to avoid conflicting facts (e.g. "is vegetarian" vs
"eats steak").
    if (associatedEntityId) {
      const { data: entityMemories } = await supabase
        .from('memory_record')
        .select('id, title')
        .eq('associated_entity_id', associatedEntityId)
        .eq('current_status', 'ACTIVE')
        .eq('memory_type', memoryType);

      if (entityMemories && entityMemories.length > 0) {
        const newTokens = new Set(canonicalKey(memoryTitle).split(/
\s+/).filter(Boolean));

        for (const em of entityMemories) {
          const oldTokens = new Set(canonicalKey(em.title).split(/\s+/).filter(Boolean));
          if (newTokens.size === 0 || oldTokens.size === 0) continue;

          const overlap = [...newTokens].filter(t => oldTokens.has(t)).length;
          const jaccard = overlap / new Set([...newTokens, ...oldTokens]).size;

          // If titles are >= 50% similar (e.g. "dietary preference" vs "diet"), archive the
old one
          if (jaccard >= 0.5) {
            await supabase
              .from('memory_record')
              .update({ current_status: 'ARCHIVED', updated_at: new
Date().toISOString() })
              .eq('id', em.id);
          }
        }
      }
    }
```

```
    const { data: savedMemory, error: insertMemoryError } = await supabase
      .from('memory_record')
      .insert({
        memory_type:         memoryType,
        title:               memoryTitle,
        memory_key:          memoryKey,
        value:               memory.value ? String(memory.value).trim().slice(0, 2000) :
null,
        summary:             memory.summary.trim().slice(0, 2000),
        source_event_id:     eventId,
        associated_entity_id: associatedEntityId,
        confidence_score:    confidence,
        review_status:       reviewStatus,
        extractor_version:   EXTRACT_MODEL,
        metadata:            {
          raw_type: memory.type,
          last_seen_event_id: eventId,
        },
      })
      .select('id')
      .single();

    if (insertMemoryError) {
      if (insertMemoryError.code === '23505') {
        const existingAfterRace = await findExistingActiveMemory(
          supabase,
          memoryType,
          memoryKey
        );

        if (existingAfterRace) {
          await supabase
            .from('memory_record')
            .update({
              value: memory.value ? String(memory.value).trim().slice(0, 2000) : null,
              summary: memory.summary.trim().slice(0, 2000),
              confidence_score: Math.max(existingAfterRace.confidence_score ?? 0,
confidence),
              review_status: reviewStatus,
              current_status: 'ACTIVE',
              updated_at: new Date().toISOString(),
              metadata: {
                ...(existingAfterRace.metadata ?? {}),
                raw_type: memory.type,
                last_seen_event_id: eventId,
              },
            })
            .eq('id', existingAfterRace.id);
          memoriesCreated++;
```

```
          continue;
        }
      }
      throw new Error('Memory insert failed: ' + insertMemoryError.message);
    }

    if (savedMemory?.id) {
      memoriesCreated++;
      if (reviewStatus === 'NEEDS_REVIEW') {
        await supabase.from('memory_review_queue').insert({
          memory_id:       savedMemory.id,
          source_event_id: eventId,
          reason:          'Low-confidence extraction requires human review',
          severity:        2,
        }).then(undefined, () => null);
      }
    }
  }

  // ── Reminder schedule ──────────────────────────────────────────────────────

  for (const reminder of extracted.reminders) {
    if (!reminder.title) continue;
    if (!reminder.reminder_at) {
      await supabase.from('memory_review_queue').insert({
        source_event_id: eventId,
        reason:          'Reminder date could not be normalized: ' +
                         (reminder.due_hint ?? reminder.title),
        severity:        3,
      }).then(undefined, () => null);
      continue;
    }

    const { error: reminderError } = await
supabase.from('reminder_schedule').upsert({
      title:           reminder.title.trim().slice(0, 255),
      reminder_at:     reminder.reminder_at,
      source_event_id: eventId,
      metadata:        { due_hint: reminder.due_hint },
    }, {
      onConflict: 'source_event_id,title,reminder_at',
    });

    if (reminderError) {
      throw new Error('Reminder upsert failed: ' + reminderError.message);
    }
  }

  await supabase.from('extraction_log').insert({
```

```
      event_id:         eventId,
      model_used:       EXTRACT_MODEL,
      entities_found:   entitiesCreated,
      tasks_found:      tasksCreated,
      relations_found:  extracted.relationships.length,
      success:          true,
      latency_ms:       Date.now() - startTime
    }).then(undefined, () => null);

    return {
      success: true,
      tasks: tasksCreated,
      entities: entitiesCreated,
      memories: memoriesCreated,
    };

  } catch (err) {
    const message = err instanceof Error ? err.message : String(err);
    console.error('[kg-processor] Error:', message);

    if (eventId) {
      await supabase.from('extraction_log').insert({
        event_id:       eventId,
        model_used:     EXTRACT_MODEL,
        entities_found: 0,
        tasks_found:    0,
        relations_found: 0,
        success:        false,
        latency_ms:     Date.now() - startTime,
        error_message:  message.slice(0, 500),
      }).then(undefined, () => null);
    }

    return { success: false, error: message };
  }
}

// ——— PHASE 1: Infer entity from memory content ———————————————————————

async function inferEntityFromMemory(
  supabase: SupabaseClient,
  title: string,
  value: string,
  summary: string
): Promise<string | null> {
  // Extract names from the combined text
  const combined = title + ' ' + value + ' ' + summary;

  // Check for possessive patterns (e.g., "Priya's birthday")
  const possessiveMatch = combined.match(/\b([A-Z][a-z]+)'s\b/);
```

```
  if (possessiveMatch) {
    const name = possessiveMatch[1];
    const entityId = await findEntityId(supabase, name);
    if (entityId) return entityId;
  }

  // Check for "is my X" patterns (e.g., "Priya is my wife")
  const relationshipMatch = combined.match(/\b([A-Z][a-z]+) is my\b/);
  if (relationshipMatch) {
    const name = relationshipMatch[1];
    const entityId = await findEntityId(supabase, name);
    if (entityId) return entityId;
  }

  return null;
}

async function callGroq(text: string, contextManifest: ContextManifest):
Promise<ExtractedData> {
  const empty: ExtractedData = {
    tasks: [], entities: [], relationships: [], memories: [], reminders: [], aliases: [],
  };

  const contextStr = `
=== CURRENT GRAPH CONTEXT ===
Active Tasks: ${contextManifest.activeTasks.map(t => t.title).join(', ')}
Memories: ${contextManifest.memoryRecords.map(m => m.title + ': ' +
m.summary).join(' | ')}
`;

  const attempt = async (): Promise<ExtractedData> => {
    const response = await getGroqClient().chat.completions.create({
      model:           EXTRACT_MODEL,
      temperature:     0.1,
      max_tokens:      1024,
      response_format: { type: 'json_object' },
      messages: [
        { role: 'system', content: EXTRACTION_PROMPT + contextStr },
        { role: 'user',   content: 'NOTE: """' + text + '"""' },
      ],
    });

    const content = response.choices?.[0]?.message?.content ?? '{}';
    const parsed  = JSON.parse(content) as Partial<ExtractedData>;

    return {
      tasks: (Array.isArray(parsed.tasks) ? parsed.tasks : [])
        .filter((t): t is ExtractedData['tasks'][0] =>
          typeof t === 'object' && typeof t.title === 'string' && t.title.length > 0
        ).slice(0, 3),
```

```
      entities: (Array.isArray(parsed.entities) ? parsed.entities : [])
        .filter((e): e is ExtractedData['entities'][0] =>
          typeof e === 'object' && typeof e.name === 'string' && e.name.length > 0
        ).slice(0, 5),
      relationships: (Array.isArray(parsed.relationships) ? parsed.relationships : [])
        .map((r: Record<string, unknown>) => {
          if (typeof r !== 'object' || !r) return null;
          const from = r.from_name || r.from_entity || r.source || r.from;
          const pred = r.predicate || r.relation || r.type;
          const to   = r.to_name || r.to_entity || r.target || r.to;
          if (!from || !pred || !to) return null;
          return { from_name: String(from), predicate: String(pred), to_name:
String(to) };
        })
        .filter((r): r is ExtractedData['relationships'][0] => r !== null)
        .slice(0, 15),
      memories: (Array.isArray(parsed.memories) ? parsed.memories : [])
        .map((m: Record<string, unknown>) => {
          if (typeof m !== 'object' || !m) return null;
          return {
            type:       m.type || 'PROFILE_FACT',
            title:      m.title || m.summary || m.value || 'Memory',
            value:      m.value || null,
            summary:    m.summary || m.value || m.title || '',
            confidence: typeof m.confidence === 'number' ? m.confidence : 0.8
          };
        })
        .filter((m): m is ExtractedData['memories'][0] => m !== null && m.title !==
'Memory' && m.summary !== '')
        .slice(0, 8),
      reminders: (Array.isArray(parsed.reminders) ? parsed.reminders : [])
        .filter((r): r is ExtractedData['reminders'][0] =>
          typeof r === 'object' && typeof r.title === 'string'
        ).slice(0, 3),
      aliases: (Array.isArray(parsed.aliases) ? parsed.aliases : [])
        .filter((a): a is ExtractedData['aliases'][0] =>
          typeof a === 'object' && typeof a.canonical_name === 'string' && typeof
a.alias === 'string'
        ).slice(0, 5),
    };
  };

  try {
    return await attempt();
  } catch (err) {
    console.error('[kg-processor] Groq error:', err);
    return empty;
  }
}
```

```
function normaliseMemoryType(type: string): string {
  const allowed = new Set([
    'PROFILE_FACT', 'PREFERENCE', 'RELATIONSHIP', 'LIFE_EVENT', 'ROUTINE',
    'DECISION', 'COMMITMENT', 'REMINDER', 'KNOWLEDGE', 'GOAL',
  ]);
  const cleaned = String(type ?? '').toUpperCase().replace(/\s+/g, '_');
  return allowed.has(cleaned) ? cleaned : 'KNOWLEDGE';
}

function clampConfidence(value: number): number {
  if (!Number.isFinite(value)) return 0.7;
  return Math.min(1, Math.max(0, value));
}

function parseIsoDateHint(value: string | null): string | null {
  if (!value) return null;
  const date = new Date(value);
  if (Number.isNaN(date.getTime())) return null;
  return date.toISOString();
}

function mergeExtractedData(primary: ExtractedData, fallback: ExtractedData):
ExtractedData {
  return {
    tasks: mergeByKey(
      primary.tasks,
      fallback.tasks,
      task => canonicalKey(task.title)
    ).slice(0, 10),
    entities: mergeByKey(
      primary.entities,
      fallback.entities,
      entity => entity.name.toLowerCase()
    ).slice(0, 12),
    relationships: mergeByKey(
      primary.relationships,
      fallback.relationships,
      rel => [rel.from_name, rel.predicate, rel.to_name].join('|').toLowerCase()
    ).slice(0, 8),
    memories: mergeByKey(
      primary.memories,
      fallback.memories,
      memory => memory.type + '|' + canonicalKey(memory.title)
    ).slice(0, 20),
    reminders: mergeByKey(
      primary.reminders,
      fallback.reminders,
      reminder => canonicalKey(reminder.title) + '|' + String(reminder.reminder_at ?? '')
    ).slice(0, 5),
    aliases: mergeByKey(
```

```
      primary.aliases,
      fallback.aliases,
      alias => alias.canonical_name.toLowerCase() + '|' + alias.alias.toLowerCase()
    ).slice(0, 10),
  };
}

function cleanExtractedData(extracted: ExtractedData, sourceText: string):
ExtractedData {
  const lower = sourceText.toLowerCase();

  // Archive patterns that should NOT create durable facts
  const archivePatterns = [
    'not a new task',
    'background context',
    'archive note',
    'reference only',
    'do not extract',
    'historical context',
    'deprecated',
    'outdated'
  ];

  const isArchiveContext = archivePatterns.some(pattern => lower.includes(pattern));

  if (isArchiveContext) {
    return {
      tasks: [],
      entities: [],
      relationships: [],
      memories: [],
      reminders: [],
      aliases: [],
    };
  }

  // Filter individual items
  const archiveFilterRegex = /background context|not a new task|archive note|
reference only|deprecated|outdated/i;

  return {
    tasks: extracted.tasks.filter(task =>
      task.title &&
      !archiveFilterRegex.test(task.title)
    ),
    entities: extracted.entities.filter(entity =>
      entity.name &&
      !archiveFilterRegex.test(entity.name)
    ),
    relationships: extracted.relationships,
```

```
    memories: extracted.memories
      .filter(memory =>
        memory.title &&
        memory.summary &&
        !archiveFilterRegex.test(
          memory.title + ' ' + memory.summary + ' ' + String(memory.value ?? '')
        )
      )
      .map(memory => ({
        ...memory,
        title: normaliseMemoryTitle(memory.title),
      })),
    reminders: extracted.reminders.filter(reminder =>
      reminder.title &&
      !archiveFilterRegex.test(reminder.title)
    ),
    aliases: extracted.aliases.filter(alias =>
      alias.canonical_name && alias.alias &&
      !archiveFilterRegex.test(alias.alias)
    ),
  };
}

function normaliseMemoryTitle(title: string): string {
  return title
    .replace(/\b(birthdate|date of birth|dob|born on)\b/ig, 'birthday')
    .replace(/\b(spouse|husband|wife|married to|marriage partner)\b/ig, 'spouse')
    .replace(/\b(kid|kids|children|child|son|daughter)\b/ig, 'child')
    .replace(/\b(work|job|role|position|title)\b/ig, 'role')
    .replace(/\b(preference|like|enjoy|prefer|favorite|favourite)\b/ig, 'preference')
    .replace(/\s+/g, ' ')
    .trim();
}

function mergeByKey<T>(primary: T[], fallback: T[], keyFn: (item: T) => string): T[] {
  const seen = new Set<string>();
  const merged: T[] = [];

  for (const item of [...primary, ...fallback]) {
    const key = keyFn(item);
    if (!key || seen.has(key)) continue;
    seen.add(key);
    merged.push(item);
  }

  return merged;
}

function deterministicExtract(text: string, recordedAt: string): ExtractedData {
  const extracted: ExtractedData = {
```

```
    tasks: [],
    entities: [],
    relationships: [],
    memories: [],
    reminders: [],
    aliases: [],
  };
  const lower = text.toLowerCase();
  const isArchiveContext = lower.includes('not a new task');

  if (!isArchiveContext) {
    addFactMemories(extracted, text);
    addTypedMemory(extracted, text, 'Decision:', 'DECISION');
    addTypedMemory(extracted, text, 'Goal:', 'GOAL');
    addTypedMemory(extracted, text, 'Preference:', 'PREFERENCE');
    addTypedMemory(extracted, text, 'Routine:', 'ROUTINE');
    addTypedMemory(extracted, text, 'Knowledge:', 'KNOWLEDGE');
    addRelationshipMemory(extracted, text);
  }

  if (lower.startsWith('reminder:')) {
    const title = sentenceCase(text.replace(/^reminder:\s*/i, '').replace(/\.$/, ''));
    const reminderAt = resolveRelativeReminderAt(text, recordedAt);
    extracted.memories.push({
      type: 'REMINDER',
      title,
      value: reminderAt,
      summary: title,
      confidence: 0.95,
    });
    extracted.reminders.push({
      title,
      reminder_at: reminderAt,
      due_hint: reminderAt ? null : text.replace(/^reminder:\s*/i, ''),
    });
  }

  return extracted;
}

function addFactMemories(extracted: ExtractedData, text: string): void {
  const wifeMatch = text.match(/my wife is ([A-Z][a-z]+)/);
  if (wifeMatch) {
    const name = wifeMatch[1];
    extracted.memories.push({
      type: 'RELATIONSHIP',
      title: name + ' is my wife',
      value: name,
      summary: name + ' is my wife and family decision partner.',
      confidence: 0.98,
```

```
    });
    extracted.relationships.push({
      from_name: 'User',
      predicate: 'SPOUSE',
      to_name: name,
    });
  }

  const marriageMatch = text.match(/married on ([0-9]{1,2} [A-Za-z]+ [0-9]{4})/i);
  if (marriageMatch) {
    extracted.memories.push({
      type: 'LIFE_EVENT',
      title: 'Marriage date',
      value: marriageMatch[1],
      summary: 'I married Priya on ' + marriageMatch[1] + '.',
      confidence: 0.98,
    });
  }

  const childMatch = text.match(/\b([A-Z][a-z]+) is my (son|daughter)\. .*?born on
([0-9]{1,2} [A-Za-z]+ [0-9]{4})/);
  if (childMatch) {
    const [, name, role, birthday] = childMatch;
    extracted.memories.push({
      type: 'PROFILE_FACT',
      title: name + ' birthday',
      value: birthday,
      summary: name + ' is my ' + role + ' and was born on ' + birthday + '.',
      confidence: 0.98,
    });
    extracted.relationships.push({
      from_name: 'User',
      predicate: role.toUpperCase(),
      to_name: name,
    });
  }

  const prefersMatch = text.match(/\b([A-Z][a-z]+) prefers (.+?)\.$/);
  if (prefersMatch) {
    extracted.memories.push({
      type: 'PREFERENCE',
      title: prefersMatch[1] + ' preference',
      value: prefersMatch[2],
      summary: prefersMatch[1] + ' prefers ' + prefersMatch[2] + '.',
      confidence: 0.9,
    });
  }

  const iPreferMatch = text.match(/^I prefer (.+?)\.$/);
  if (iPreferMatch) {
```

```
    extracted.memories.push({
      type: 'PREFERENCE',
      title: 'Personal preference',
      value: iPreferMatch[1],
      summary: 'I prefer ' + iPreferMatch[1] + '.',
      confidence: 0.9,
    });
  }
}

function addTypedMemory(
  extracted: ExtractedData,
  text: string,
  prefix: string,
  type: ExtractedData['memories'][number]['type']
): void {
  if (!text.startsWith(prefix)) return;
  const summary = text.slice(prefix.length).trim().replace(/\.$/, '');
  if (!summary) return;

  extracted.memories.push({
    type,
    title: sentenceCase(summary.split(',')[0].slice(0, 80)),
    value: summary,
    summary,
    confidence: 0.92,
  });
}

function addRelationshipMemory(extracted: ExtractedData, text: string): void {
  if (!text.startsWith('Relationship:')) return;
  const summary = text.slice('Relationship:'.length).trim().replace(/\.$/, '');
  const name = summary.match(/^([A-Z][a-z]+)/)?.[1] ?? 'Relationship';

  extracted.memories.push({
    type: 'RELATIONSHIP',
    title: name + ' relationship',
    value: summary,
    summary,
    confidence: 0.92,
  });
}

function resolveRelativeReminderAt(text: string, recordedAt: string): string | null {
  const base = new Date(recordedAt);
  if (Number.isNaN(base.getTime())) return null;
  const lower = text.toLowerCase();
  const date = new Date(base);

  if (lower.includes('tomorrow')) {
```

```
    date.setUTCDate(date.getUTCDate() + 1);
  }

  if (lower.includes('morning')) {
    date.setUTCHours(9, 0, 0, 0);
  } else if (lower.includes('tonight')) {
    date.setUTCHours(20, 0, 0, 0);
  } else {
    date.setUTCHours(9, 0, 0, 0);
  }

  return date.toISOString();
}

function sentenceCase(value: string): string {
  const trimmed = value.trim();
  if (!trimmed) return trimmed;
  return trimmed.charAt(0).toUpperCase() + trimmed.slice(1);
}

function canonicalKey(value: string): string {
  const stop = new Set([
    'a', 'an', 'and', 'are', 'about', 'for', 'from', 'is', 'of', 'on', 'the',
    'to', 'with', 'task', 'tasks', 'follow', 'up', 'check', 'review',
  ]);

  return value
    .toLowerCase()
    .replace(/[^a-z0-9\s]/g, ' ')
    .split(/\s+/)
    .filter(token => token.length > 1 && !stop.has(token))
    .map(token => normaliseToken(token))
    .sort()
    .join(' ')
    .trim();
}

function normaliseToken(token: string): string {
  if (token === 'birthdate') return 'birthday';
  if (token === 'hotels') return 'hotel';
  if (token === 'kids') return 'kid';
  if (token === 'children') return 'child';
  if (token === 'homeworks') return 'homework';
  if (token.endsWith('ing') && token.length > 5) return token.slice(0, -3);
  if (token.endsWith('s') && token.length > 4) return token.slice(0, -1);
  return token;
}

async function findExistingOpenTask(
  supabase: SupabaseClient,
```

```
  title: string,
  taskKey: string
): Promise<ExistingTaskRow | null> {
  const { data: exact } = await supabase
    .from('task_projection')
    .select('id, originating_event_id, title, task_key, estimated_effort, due_at,
metadata')
    .eq('task_key', taskKey)
    .neq('current_status', 'COMPLETED')
    .neq('current_status', 'ABANDONED')
    .limit(1)
    .maybeSingle();

  if (exact) return exact as ExistingTaskRow;

  const { data } = await supabase
    .from('task_projection')
    .select('id, originating_event_id, title, task_key, estimated_effort, due_at,
metadata')
    .neq('current_status', 'COMPLETED')
    .neq('current_status', 'ABANDONED')
    .order('created_at', { ascending: false })
    .limit(100);

  const targetTokens = new Set(canonicalKey(title).split(/\s+/).filter(Boolean));
  let best: { row: ExistingTaskRow; score: number } | null = null;

  for (const row of (data ?? []) as ExistingTaskRow[]) {
    const rowTokens = new Set(canonicalKey(row.title).split(/\s+/).filter(Boolean));
    if (targetTokens.size === 0 || rowTokens.size === 0) continue;
    const overlap = [...targetTokens].filter(token => rowTokens.has(token)).length;
    const score = overlap / Math.max(targetTokens.size, rowTokens.size);
    if (!best || score > best.score) best = { row, score };
  }

  return best && best.score >= 0.5 ? best.row : null;
}

async function findExistingActiveMemory(
  supabase: SupabaseClient,
  memoryType: string,
  memoryKey: string
): Promise<ExistingMemoryRow | null> {
  const { data } = await supabase
    .from('memory_record')
    .select('id, confidence_score, metadata')
    .eq('memory_type', memoryType)
    .eq('memory_key', memoryKey)
    .eq('current_status', 'ACTIVE')
    .limit(1)
```

```
    .maybeSingle();

  return data ? data as ExistingMemoryRow : null;
}

async function getOrCreateEntity(
  supabase: SupabaseClient,
  name: string,
  type: string,
  summary: string | null
): Promise<string | null> {
  const cleanedName = name.trim();
  if (!cleanedName) return null;

  const { data: alias } = await supabase
    .from('entity_alias')
    .select('canonical_id')
    .ilike('alias_expression', cleanedName)
    .limit(1)
    .maybeSingle();

  if (alias?.canonical_id) return alias.canonical_id as string;

  const { data: existingEntity } = await supabase
    .from('canonical_entity')
    .select('id')
    .ilike('canonical_name', cleanedName)
    .limit(1)
    .maybeSingle();

  if (existingEntity?.id) {
    await supabase.from('entity_alias').upsert({
      canonical_id: existingEntity.id,
      alias_expression: cleanedName,
      confidence_score: 1.000,
      merge_reviewed: true,
    }, { onConflict: 'canonical_id,alias_expression' }).then(() => null);

    return existingEntity.id as string;
  }

  const { data: newEntity, error } = await supabase
    .from('canonical_entity')
    .insert({
      classification: type,
      canonical_name: cleanedName,
      summary_portrait: summary,
    })
    .select('id')
    .single();
```

```
  if (error || !newEntity) {
    console.error('[extract-event] entity insert failed:', error);
    return null;
  }

  await supabase.from('entity_alias').insert({
    canonical_id: newEntity.id,
    alias_expression: cleanedName,
    confidence_score: 1.000,
    merge_reviewed: true,
  }).then(undefined, () => null);

  return newEntity.id as string;
}

async function findEntityId(
  supabase: SupabaseClient,
  name:     string
): Promise<string | null> {
  const { data } = await supabase
    .from('entity_alias')
    .select('canonical_id')
    .ilike('alias_expression', name.trim())
    .limit(1)
    .single();

  return (data as { canonical_id: string } | null)?.canonical_id ?? null;
}


--------------------------------------------------------------------------------
FILE: lib/retrieval.ts
--------------------------------------------------------------------------------
import { getSupabaseServer } from '@/lib/supabase/server';
import type { MemorySearchRecord, ReminderSchedule, SemanticChunk } from '@/
lib/types';

// Named type aliases — avoids multi-line generics that break terminal paste
type ChatEvent  = {
  id: string;
  recorded_at: string;
  payload: string;
  metadata: Record<string, unknown>;
};
type TaskStatus = {
  id: string;
  title: string;
```

```
  current_status: string;
  completed_at: string | null;
};

export interface RetrievalResult {
  chunks:    SemanticChunk[];
  latencyMs: number;
  mode:      'hybrid' | 'keyword-only';
}

interface HybridRow {
  chunk_id: string;
  content:  string;
  score:    number;
}

export async function hybridRetrieval(
  query:     string,
  queryVec?: number[]
): Promise<RetrievalResult> {
  const start    = Date.now();
  const supabase = getSupabaseServer();

  try {
    if (queryVec && queryVec.length === 384) {
      const { data, error } = await supabase.rpc('hybrid_retrieval', {
        query_text:   query,
        query_vec:    '[' + queryVec.join(',') + ']',
        decay_lambda: 0.00000005,
      });

      if (error) {
        console.error('[retrieval] hybrid_retrieval RPC error:', error);
        return keywordFallback(query, start);
      }

      const rows   = (data as HybridRow[]) ?? [];
      const chunks = rows.map(rowToChunk);
      void incrementRecallCount(chunks.map(c => c.id));
      return { chunks, latencyMs: Date.now() - start, mode: 'hybrid' };
    }

    return keywordFallback(query, start);

  } catch (err) {
    console.error('[retrieval] unexpected error:', err);
    return { chunks: [], latencyMs: Date.now() - start, mode: 'keyword-only' };
  }
}
```

```
async function keywordFallback(
  query: string,
  start: number
): Promise<RetrievalResult> {
  const supabase = getSupabaseServer();

  try {
    const { data, error } = await supabase
      .from('semantic_chunk')
      .select('*')
      .textSearch('chunk_text', query, { type: 'plain', config: 'english' })
      .order('generation_timestamp', { ascending: false })
      .limit(15);

    if (error) {
      console.error('[retrieval] keyword fallback error:', error);
      return { chunks: [], latencyMs: Date.now() - start, mode: 'keyword-only' };
    }

    const chunks = (data as SemanticChunk[]) ?? [];
    void incrementRecallCount(chunks.map(c => c.id));
    return { chunks, latencyMs: Date.now() - start, mode: 'keyword-only' };

  } catch (err) {
    console.error('[retrieval] keyword fallback unexpected error:', err);
    return { chunks: [], latencyMs: Date.now() - start, mode: 'keyword-only' };
  }
}

export async function getRecentChatEvents(limit = 15): Promise<ChatEvent[]> {
  const supabase = getSupabaseServer();

  try {
    const { data, error } = await supabase
      .from('system_event')
      .select('id, recorded_at, payload, metadata, source_tab')
      .in('source_tab', ['CHAT', 'THOUGHTS'])
      .order('recorded_at', { ascending: false })
      .limit(limit);

    if (error) {
      console.error('[retrieval] getRecentChatEvents error:', error);
      return [];
    }

    return (((data ?? []) as ChatEvent[]).reverse());

  } catch (err) {
    console.error('[retrieval] getRecentChatEvents unexpected error:', err);
    return [];
```

```
  }
}

export async function getActiveTasks() {
  const supabase = getSupabaseServer();

  try {
    const selectFields = '*, project:associated_project(canonical_name),
person:assigned_person(canonical_name)';

    const { data, error } = await supabase
      .from('task_projection')
      .select(selectFields)
      .neq('current_status', 'COMPLETED')
      .neq('current_status', 'ABANDONED')
      .order('created_at', { ascending: false });

    if (error) {
      console.error('[retrieval] getActiveTasks error:', error);
      return [];
    }

    interface RowWithJoins {
      [key: string]: unknown;
      project?: { canonical_name: string } | null;
      person?:  { canonical_name: string } | null;
    }

    return ((data ?? []) as RowWithJoins[]).map(row => ({
      ...row,
      project_name: row.project?.canonical_name ?? null,
      person_name:  row.person?.canonical_name  ?? null,
      project:      undefined,
      person:       undefined,
    }));

  } catch (err) {
    console.error('[retrieval] getActiveTasks unexpected error:', err);
    return [];
  }
}

export async function getRelevantMemoryRecords(
  query: string,
  limit = 12
): Promise<MemorySearchRecord[]> {
  const supabase = getSupabaseServer();

  try {
    // 1. Working Memory Buffer: Always get the 5 most recently created/updated
```

```
memories.
    // This perfectly bridges the gap for facts extracted during the current
conversation,
    // bypassing the need for exact vocabulary matches or embedding syncs.
    const { data: recentData } = await supabase
      .from('memory_record')
      .select('id, memory_type, title, value, summary, confidence_score,
source_event_id, created_at')
      .eq('current_status', 'ACTIVE')
      .order('updated_at', { ascending: false })
      .limit(5);

    const recentMemories = (recentData ?? []).map(r => ({ ...r, score: 1.0 })) as
MemorySearchRecord[];

    // 2. Perform DB FTS search
    const { data: searchedData, error } = await
supabase.rpc('search_memory_records', {
      p_query: query,
      p_limit: limit,
    });

    let searchResults = ((searchedData ?? []) as MemorySearchRecord[]);
    if (error) {
      console.error('[retrieval] search_memory_records error:', error);
      searchResults = await keywordMemoryFallback(query, limit);
    } else if (searchResults.length === 0) {
      // FTS often fails on natural language due to strict AND operators. Fall back to
forgiving search.
      searchResults = await keywordMemoryFallback(query, limit);
    }

    // 3. Merge and deduplicate
    const combined = [...recentMemories, ...searchResults];
    const unique = Array.from(new Map(combined.map(item => [item.id,
item])).values());

    return unique.slice(0, limit);

  } catch (err) {
    console.error('[retrieval] getRelevantMemoryRecords unexpected error:', err);
    return keywordMemoryFallback(query, limit);
  }
}

async function keywordMemoryFallback(
  query: string,
  limit: number
): Promise<MemorySearchRecord[]> {
  const supabase = getSupabaseServer();
```

```
  // Extract significant words (length > 3) to avoid stop words
  const words = query
    .replace(/[^\w\s]/g, '')
    .split(/\s+/)
    .filter(w => w.length > 3)
    .slice(0, 5); // Take up to 5 keywords

  if (words.length === 0) return [];

  try {
    // Build an OR query for each word against title, value, and summary
    const orConditions = words.flatMap(w => [
      'title.ilike.%' + w + '%',
      'value.ilike.%' + w + '%',
      'summary.ilike.%' + w + '%'
    ]).join(',');

    const { data, error } = await supabase
      .from('memory_record')
      .select('id, memory_type, title, value, summary, confidence_score,
source_event_id, created_at')
      .eq('current_status', 'ACTIVE')
      .or(orConditions)
      .order('created_at', { ascending: false })
      .limit(limit);

    if (error) {
      console.error('[retrieval] keywordMemoryFallback error:', error);
      return [];
    }

    return ((data ?? []) as Omit<MemorySearchRecord, 'score'>[])
      .map(row => ({ ...row, score: row.confidence_score ?? 0 }));

  } catch (err) {
    console.error('[retrieval] keywordMemoryFallback unexpected error:', err);
    return [];
  }
}

export async function getUpcomingReminders(limit = 10):
Promise<ReminderSchedule[]> {
  const supabase = getSupabaseServer();

  try {
    const { data, error } = await supabase
      .from('reminder_schedule')
      .select('*')
      .in('state', ['SCHEDULED', 'SNOOZED'])
```

```
      .order('reminder_at', { ascending: true })
      .limit(limit);

    if (error) {
      console.error('[retrieval] getUpcomingReminders error:', error);
      return [];
    }

    return (data ?? []) as ReminderSchedule[];

  } catch (err) {
    console.error('[retrieval] getUpcomingReminders unexpected error:', err);
    return [];
  }
}

export function extractTaskIdsFromText(text: string): string[] {
  const UUID_RE = /[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}/gi;
  return [...new Set(text.match(UUID_RE) ?? [])];
}

export async function getTaskStatusById(ids: string[]): Promise<TaskStatus[]> {
  if (ids.length === 0) return [];
  const supabase = getSupabaseServer();

  try {
    const { data, error } = await supabase
      .from('task_projection')
      .select('id, title, current_status, completed_at')
      .in('id', ids);

    if (error) {
      console.error('[retrieval] getTaskStatusById error:', error);
      return [];
    }

    return (data ?? []) as TaskStatus[];

  } catch (err) {
    console.error('[retrieval] getTaskStatusById unexpected error:', err);
    return [];
  }
}

function rowToChunk(row: HybridRow): SemanticChunk {
  return {
    id:                  row.chunk_id,
    originating_event_id: '',
    associated_entity_id: null,
    chunk_text:          row.content,
```

```
    embedding:            null,
    embedding_model:      null,
    generation_timestamp: new Date().toISOString(),
    attention_score:      Number(row.score),
    recall_count:         0,
  };
}

async function incrementRecallCount(ids: string[]): Promise<void> {
  if (ids.length === 0) return;
  const supabase = getSupabaseServer();
  try {
    await supabase.rpc('increment_recall_count', { chunk_ids: ids });
  } catch {
    // Non-critical — silently ignore
  }
}
```

FILE: lib/supabase/client.ts

```
import { createClient, SupabaseClient } from '@supabase/supabase-js';

const supabaseUrl  = process.env.NEXT_PUBLIC_SUPABASE_URL;
const supabaseAnon = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

let _client: SupabaseClient | null = null;

export function getSupabaseClient(): SupabaseClient {
  if (_client) return _client;

  if (!supabaseUrl)  throw new Error('Missing NEXT_PUBLIC_SUPABASE_URL');
  if (!supabaseAnon) throw new Error('Missing
NEXT_PUBLIC_SUPABASE_ANON_KEY');

  _client = createClient(supabaseUrl, supabaseAnon, {
    auth: {
      persistSession:    false,
      autoRefreshToken:  false,
      detectSessionInUrl: false,
    },
    realtime: {
      params: {
        eventsPerSecond: 10,
      },
    },
  });
```

```
  return _client;
}

export const supabase = new Proxy({} as SupabaseClient, {
  get(_target, prop, receiver) {
    return Reflect.get(getSupabaseClient(), prop, receiver);
  },
});
```

FILE: lib/supabase/server.ts

```
import { createClient, SupabaseClient } from '@supabase/supabase-js';

const supabaseUrl  = process.env.NEXT_PUBLIC_SUPABASE_URL;
const supabaseAnon = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

export function getSupabaseServer(): SupabaseClient {
  if (!supabaseUrl)  throw new Error('Missing NEXT_PUBLIC_SUPABASE_URL');
  if (!supabaseAnon) throw new Error('Missing
NEXT_PUBLIC_SUPABASE_ANON_KEY');

  return createClient(supabaseUrl, supabaseAnon, {
    auth: {
      persistSession:    false,
      autoRefreshToken:  false,
      detectSessionInUrl: false,
    },
  });
}

export async function query<T>(
  builder: PromiseLike<{ data: T | null; error: unknown }>
): Promise<T> {
  const { data, error } = await builder;
  if (error) {
    const message = error instanceof Error
      ? error.message
      : JSON.stringify(error);
    throw new Error(`Supabase query failed: ${message}`);
  }
  if (data === null) {
    throw new Error('Supabase query returned null data');
  }
  return data;
}
```

```
export async function queryMaybe<T>(
  builder: PromiseLike<{ data: T | null; error: unknown }>
): Promise<T | null> {
  const { data, error } = await builder;
  if (error) {
    const message = error instanceof Error
      ? error.message
      : JSON.stringify(error);
    throw new Error(`Supabase query failed: ${message}`);
  }
  return data;
}
```

FILE: lib/truth-arbitration.ts

```
import { getSupabaseServer } from '@/lib/supabase/server';
import type { TruthArbitrationResult } from '@/lib/types';

/**
 * Resolve what the system currently believes to be true
 * about a given entity and predicate relationship.
 *
 * Returns one of three states:
 *   DEFINITIVE  — one clear winning claim
 *   CONTESTED   — two competing claims with similar confidence
 *   UNKNOWN     — no valid claim found
 *
 * Never throws — returns UNKNOWN on any failure.
 */
export async function arbitrateTruth(
  entityId:  string,
  predicate: string,
  atTime?:   Date
): Promise<TruthArbitrationResult> {
  const supabase = getSupabaseServer();
  const unknown: TruthArbitrationResult = { state: 'UNKNOWN' };

  try {
    const { data, error } = await supabase.rpc('arbitrate_truth', {
      p_entity:    entityId,
      p_predicate: predicate,
      p_at:        (atTime ?? new Date()).toISOString(),
    });

    if (error) {
```

```
      console.error('[truth-arbitration] RPC error:', error);
      return unknown;
    }

    if (!data || typeof data !== 'object') return unknown;

    const result = data as {
      state:      string;
      claim?:     string;
      competing?: string;
      confidence?: number;
      note?:      string;
    };

    if (result.state === 'DEFINITIVE') {
      return {
        state:      'DEFINITIVE',
        claim:      result.claim,
        confidence: result.confidence,
      };
    }

    if (result.state === 'CONTESTED') {
      return {
        state:     'CONTESTED',
        claim:     result.claim,
        competing: result.competing,
        note:      result.note,
      };
    }

    return unknown;

  } catch (err) {
    console.error('[truth-arbitration] unexpected error:', err);
    return unknown;
  }
}

/**
 * Format a TruthArbitrationResult as a string
 * suitable for injection into an LLM prompt.
 */
export function formatArbitrationForPrompt(
  predicate: string,
  result:    TruthArbitrationResult
): string {
  switch (result.state) {
    case 'DEFINITIVE':
      return `${predicate}: ${result.claim ?? 'unknown'} (confidence: ${
```

```
        result.confidence ? (result.confidence * 100).toFixed(0) + '%' : 'n/a'
      })`;

    case 'CONTESTED':
      return [
        `${predicate}: UNCERTAIN — two competing beliefs:`,
        `  • Primary:   ${result.claim ?? 'unknown'}`,
        `  • Competing: ${result.competing ?? 'unknown'}`,
        `  → Do not assert either as definitive fact.`,
      ].join('\n');

    case 'UNKNOWN':
      return `${predicate}: No current belief recorded.`;
  }
}


──────────────────────────────────────────────────────────────────────────────
FILE: lib/types.ts
──────────────────────────────────────────────────────────────────────────────
//
──────────────────────────────────────────────────────────────────────────────
// Database row types — field names match SQL column names exactly
//
──────────────────────────────────────────────────────────────────────────────

export type TabSource =
  | 'THOUGHTS'
  | 'CHAT'
  | 'TASKS'
  | 'SYSTEM';

export type TaskState =
  | 'PENDING'
  | 'ACTIVE'
  | 'BLOCKED'
  | 'COMPLETED'
  | 'ABANDONED';

export type EntityClass =
  | 'PERSON'
  | 'PROJECT'
  | 'GOAL'
  | 'CONCEPT'
```

```
  | 'ORG';

export type EpistemicType =
  | 'ASSERTION'
  | 'INFERENCE'
  | 'INTENTION'
  | 'PREDICTION'
  | 'BELIEF'
  | 'OBSERVATION'
  | 'VERIFIED_FACT';

export type TrustLevel =
  | 'USER_STATED'
  | 'AI_INFERRED'
  | 'BEHAVIORALLY_OBSERVED'
  | 'EXTERNALLY_VERIFIED';

export type LoadState =
  | 'COMFORTABLE'
  | 'FULL'
  | 'OVERLOADED'
  | 'CRITICAL';

export type TruthState =
  | 'DEFINITIVE'
  | 'CONTESTED'
  | 'UNKNOWN';

export type MemoryKind =
  | 'PROFILE_FACT'
  | 'PREFERENCE'
  | 'RELATIONSHIP'
  | 'LIFE_EVENT'
  | 'ROUTINE'
  | 'DECISION'
  | 'COMMITMENT'
  | 'REMINDER'
  | 'KNOWLEDGE'
  | 'GOAL'
  | 'CORRECTION';

export type MemoryStatus =
  | 'ACTIVE'
  | 'SUPERSEDED'
  | 'ARCHIVED'
  | 'REJECTED';

export type ReviewState =
  | 'AUTO_ACCEPTED'
  | 'NEEDS_REVIEW'
```

```
  | 'APPROVED'
  | 'REJECTED';

export type ReminderState =
  | 'SCHEDULED'
  | 'FIRED'
  | 'SNOOZED'
  | 'CANCELLED'
  | 'DONE';

// ——— Database rows ———————————————————————————————————————————————————

export interface SystemEvent {
  id:          string;
  recorded_at: string;
  source_tab:  TabSource;
  payload:     string;
  metadata:    Record<string, unknown>;
  compacted:   boolean;
}

export interface CanonicalEntity {
  id:              string;
  classification:  EntityClass;
  canonical_name:  string;
  summary_portrait: string | null;
  created_at:      string;
  updated_at:      string;
  system_metadata: Record<string, unknown>;
}

export interface EntityAlias {
  id:              string;
  canonical_id:    string;
  alias_expression: string;
  confidence_score: number | null;
  auto_merge_flag: boolean;
  merge_reviewed:  boolean;
  created_at:      string;
}

export interface IdentityFlag {
  id:          string;
  entity_a_id: string;
  entity_b_id: string;
  similarity:  number | null;
  flag_reason: string | null;
  flagged_at:  string;
```

```
  resolved_at: string | null;
  resolution:  'MERGED' | 'KEPT_SEPARATE' | 'PENDING' | null;
  resolved_by: 'USER' | 'SYSTEM' | null;
}

export interface EntityRelation {
  id:                string;
  source_entity_id:  string;
  target_entity_id:  string;
  predicate:         string;
  valid_from:        string;
  valid_to:          string | null;
  assertion_event_id: string | null;
  emotional_context:  string | null;
  confidence_score:  number;
}

export interface TaskProjection {
  id:                string;
  originating_event_id: string | null;
  associated_project:  string | null;
  assigned_person:     string | null;
  title:             string;
  task_key?:         string | null;
  current_status:      TaskState;
  estimated_effort:    number;
  due_at:            string | null;
  completed_at:        string | null;
  created_at:        string;
  updated_at:        string;
  metadata?:         Record<string, unknown>;
  // Joined fields (not in DB — added by API)
  project_name?:       string | null;
  person_name?:        string | null;
}

export interface SemanticChunk {
  id:                string;
  originating_event_id: string;
  associated_entity_id: string | null;
  chunk_text:        string;
  embedding:           number[] | null;
  embedding_model:     string | null;
  generation_timestamp: string;
  attention_score:     number;
  recall_count:        number;
}

export interface EpistemicClaim {
  id:                string;
```

```
  claim_text:         string;
  claim_type:         EpistemicType;
  trust_level:        TrustLevel;
  subject_entity_id:  string | null;
  source_event_id:    string | null;
  extracted_by_model: string;
  confidence_score:   number | null;
  valid_from:         string;
  valid_to:           string | null;
  contradiction_flag: boolean;
  contradicts_id:     string | null;
  created_at:         string;
}

export interface GoalConflict {
  id:              string;
  goal_a_id:       string;
  goal_b_id:       string;
  conflict_type:   'VALUE' | 'RESOURCE' | 'TIME';
  conflict_summary: string;
  severity:        number | null;
  detected_at:     string;
  resolved_at:     string | null;
  dismissed:       boolean;
}

export interface ResponseCache {
  id:            string;
  prompt_hash:   string;
  response_text: string;
  created_at:    string;
  expires_at:    string;
}

export interface ContextAssemblyLog {
  id:               string;
  assembled_at:     string;
  user_query:       string;
  query_hash:       string | null;
  chunks_retrieved: number;
  chunks_in_context: number;
  top_chunk_ids:    string[] | null;
  top_chunk_scores: number[] | null;
  total_tokens:     number | null;
  override_count:   number;
  tasks_in_context: number;
  load_state:       string | null;
  cache_hit:        boolean;
}
```

```
export interface ExtractionLog {
  id:              string;
  event_id:        string | null;
  extracted_at:    string;
  model_used:      string;
  payload_hash:    string | null;
  entities_found:  number;
  tasks_found:     number;
  relations_found: number;
  success:         boolean;
  latency_ms:      number | null;
  error_message:   string | null;
}

export interface CognitiveHealthLog {
  id:                  string;
  computed_at:         string;
  overall_score:       number | null;
  embedding_coverage:  number | null;
  retrieval_p95_ms:    number | null;
  entity_conflicts:    number | null;
  stale_claims:        number | null;
  contradiction_pct:   number | null;
  extraction_success:  number | null;
  graph_density:       number | null;
  raw_metrics:         Record<string, unknown>;
}

export interface MemoryRecord {
  id:                  string;
  user_id:             string;
  memory_type:         MemoryKind;
  title:               string;
  memory_key?:         string | null;
  value:               string | null;
  summary:             string;
  source_event_id:     string | null;
  associated_entity_id: string | null;
  associated_task_id:  string | null;
  confidence_score:    number;
  review_status:       ReviewState;
  current_status:      MemoryStatus;
  extractor_version:   string;
  valid_from:          string;
  valid_to:            string | null;
  last_verified_at:    string | null;
  metadata:            Record<string, unknown>;
  created_at:          string;
  updated_at:          string;
}
```

```
export interface MemorySearchRecord {
  id:               string;
  memory_type:      MemoryKind;
  title:            string;
  value:            string | null;
  summary:          string;
  confidence_score: number;
  source_event_id:  string | null;
  created_at:       string;
  score:            number;
}

export interface MemoryReviewItem {
  id:              string;
  memory_id:       string | null;
  source_event_id: string | null;
  reason:          string;
  severity:        number;
  status:          ReviewState;
  created_at:      string;
  reviewed_at:     string | null;
  reviewed_by:     string | null;
  notes:           string | null;
  memory?:         MemoryRecord | null;
}

export interface ReminderSchedule {
  id:              string;
  user_id:         string;
  task_id:         string | null;
  memory_id:       string | null;
  source_event_id: string | null;
  title:           string;
  reminder_at:     string;
  recurrence_rule: string | null;
  state:           ReminderState;
  fired_at:        string | null;
  created_at:      string;
  updated_at:      string;
  metadata:        Record<string, unknown>;
}

export interface OperationalHealth {
  total_events:           number;
  failed_extractions:     number;
  pending_embeddings:     number;
  pending_memory_reviews: number;
  pending_identity_flags: number;
  due_reminders:          number;
```

```
  active_memories:        number;
}

// ——— Computed / derived types
————————————————————————————————————————————————————————————————————————

export interface CognitiveLoad {
  open_tasks:      number;
  blocked_tasks:   number;
  active_tasks:    number;
  active_projects: number;
  raw_load:        number;
  load_state:      LoadState;
}

export interface TruthArbitrationResult {
  state:      TruthState;
  claim?:     string;
  competing?: string;
  confidence?: number;
  note?:      string;
}

// ——— LLM / API types
————————————————————————————————————————————————————————————————————————

export interface ChatMessage {
  role:    'user' | 'assistant' | 'system';
  content: string;
}

export interface ExtractionResult {
  tasks: {
    title:         string;
    project_hint:  string | null;
    person_hint:   string | null;
    deadline_hint: string | null;
    effort:        1 | 2 | 3 | 4 | 5;
  }[];
  entities: {
    name:             string;
    type:             EntityClass;
    context_snippet:  string;
  }[];
  relationships: {
    from_name: string;
    predicate: string;
    to_name:   string;
```

```
  }[];
  memories?: {
    type:       MemoryKind;
    title:      string;
    value?:     string | null;
    summary:    string;
    confidence?: number;
  }[];
  reminders?: {
    title:      string;
    reminder_at?: string | null;
    due_hint?:   string | null;
  }[];
  aliases?: {
    canonical_name: string;
    alias:          string;
  }[];
}

export interface ContextManifest {
  systemPrompt:    string;
  overrides:       string[];
  retrievedChunks: SemanticChunk[];
  memoryRecords:   MemorySearchRecord[];
  activeTasks:     TaskProjection[];
  loadState:       LoadState;
  rawLoad:         number;
  tokensUsed:      number;
}

export interface HealthStatus {
  overall_score:       number | null;
  embedding_coverage:  number | null;
  extraction_success:  number | null;
  entity_conflicts:    number;
  identity_flags:      number;
  computed_at:         string | null;
  load_state:          LoadState;
  raw_load:            number;
  pending_embeddings?: number;
  memory_reviews?:     number;
  active_memories?:    number;
}

// ——— API response types ———————————————————————————————————————————————

export interface IngestResponse {
  success:  boolean;
```

```
  event_id: string;
}

export interface TasksResponse {
  tasks:        TaskProjection[];
  load:         CognitiveLoad;
  identity_flags_pending: number;
}

export interface EmbeddingBatch {
  event_id:  string;
  embedding: number[];
}

export interface EmbeddingsResponse {
  updated: number;
}


================================================================================
FILE: next.config.js
================================================================================
/** @type {import('next').NextConfig} */
const nextConfig = {
  experimental: {
    serverComponentsExternalPackages: ['@xenova/transformers', 'onnxruntime-
node'],
  },
  webpack: (config) => {
    config.resolve.fallback = {
      ...config.resolve.fallback,
      fs: false,
      path: false,
      crypto: false,
    };
    return config;
  },
  async headers() {
    return [
      {
        source: '/(.*)',
        headers: [
          { key: 'X-Content-Type-Options', value: 'nosniff' },
          { key: 'X-Frame-Options', value: 'DENY' },
        ],
      },
    ];
  },
```

```
};

module.exports = nextConfig;
```

FILE: package.json

```
{
  "name": "personal_assistant",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "typecheck": "tsc --noEmit",
    "verify:env": "node scripts/verify-env.mjs",
    "verify:sql": "node scripts/verify-sql.mjs",
    "generate:stress": "node scripts/generate-lifelong-stress.mjs",
    "evaluate:core": "node scripts/evaluate-core.mjs",
    "verify": "npm run verify:env && npm run verify:sql && npm run typecheck && npm
run build",
    "dump": "bash scripts/dump-repo.sh"
  },
  "dependencies": {
    "@supabase/supabase-js": "^2.107.0",
    "@xenova/transformers": "^2.17.2",
    "groq-sdk": "^1.2.1",
    "next": "14.2.35",
    "react": "^18",
    "react-dom": "^18"
  },
  "devDependencies": {
    "@types/node": "^20",
    "@types/react": "^18",
    "@types/react-dom": "^18",
    "canvas": "^3.2.3",
    "eslint": "^8",
    "eslint-config-next": "14.2.35",
    "postcss": "^8",
    "tailwindcss": "^3.4.1",
    "typescript": "^5"
  }
}
```

_____

FILE: public/manifest.json

_____

_____

```
{
  "name": "Lifelong AI",
  "short_name": "Lifelong AI",
  "description": "Your personal cognitive assistant",
  "start_url": "/thoughts",
  "display": "standalone",
  "background_color": "#06080f",
  "theme_color": "#06080f",
  "orientation": "portrait",
  "icons": [
    { "src": "/icons/icon-192.png", "sizes": "192x192", "type": "image/png", "purpose":
"any maskable" },
    { "src": "/icons/icon-512.png", "sizes": "512x512", "type": "image/png", "purpose":
"any maskable" }
  ]
}
```

_____

_____

FILE: scripts/evaluate-core.mjs

_____

_____

```
const baseUrl = process.env.EVAL_BASE_URL || 'http://localhost:3000';

const cases = [
  {
    question: 'Who is Priya?',
    expects: ['wife', 'Priya'],
  },
  {
    question: "When is Vaidik's birthday?",
    expects: ['16', 'March', '2021'],
  },
  {
    question: "When is Vanya's birthday?",
    expects: ['9', 'March', '2018'],
  },
  {
    question: 'What is pending for the family vacation?',
    expects: ['destination', 'hotel'],
  },
  {
    question: 'What did I decide about the assistant project?',
    expects: ['reliability', 'traceability'],
  },
```

```
];

async function ask(question) {
  const res = await fetch(baseUrl + '/api/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ message: question }),
  });

  if (!res.ok || !res.body) {
    throw new Error('Chat request failed for "' + question + '": ' + res.status);
  }

  return await res.text();
}

let passed = 0;

for (const testCase of cases) {
  const answer = await ask(testCase.question);
  const normalized = answer.toLowerCase();
  const missing = testCase.expects.filter(token =>
    !normalized.includes(token.toLowerCase())
  );

  if (missing.length === 0) {
    passed++;
    console.log('PASS ' + testCase.question);
  } else {
    console.log('FAIL ' + testCase.question);
    console.log('Missing: ' + missing.join(', '));
    console.log('Answer: ' + answer);
  }
}

console.log(String(passed) + '/' + String(cases.length) + ' evaluation cases passed.');
process.exit(passed === cases.length ? 0 : 1);


_____________________________________________________________________________
_____________________
FILE: scripts/generate-icons.mjs
_____________________________________________________________________________
_____________________
import { createCanvas } from 'canvas';
import { writeFileSync } from 'fs';

function generateIcon(size) {
  const canvas = createCanvas(size, size);
  const ctx    = canvas.getContext('2d');
```

```
  // Background
  ctx.fillStyle = '#06080f';
  ctx.fillRect(0, 0, size, size);

  // Rounded rect overlay
  const pad = size * 0.12;
  ctx.fillStyle = '#0c1222';
  ctx.beginPath();
  ctx.roundRect(pad, pad, size - pad * 2, size - pad * 2, size * 0.18);
  ctx.fill();

  // Brain emoji / letter
  ctx.fillStyle = '#38bdf8';
  ctx.font      = 'bold ' + Math.floor(size * 0.38) + 'px system-ui';
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
  ctx.fillText('L', size / 2, size / 2 + size * 0.02);

  // Subtle dot
  ctx.fillStyle = '#38bdf8';
  ctx.beginPath();
  ctx.arc(size * 0.72, size * 0.28, size * 0.045, 0, Math.PI * 2);
  ctx.fill();

  return canvas.toBuffer('image/png');
}

writeFileSync('public/icons/icon-192.png', generateIcon(192));
writeFileSync('public/icons/icon-512.png', generateIcon(512));
console.log('Icons generated: public/icons/icon-192.png and icon-512.png');
```

FILE: scripts/generate-lifelong-stress.mjs

```
import { readFileSync, writeFileSync } from 'node:fs';
import { join } from 'node:path';

const root = new URL('..', import.meta.url).pathname;
const seedOutput = join(root, 'supabase', 'seed-lifelong-stress.sql');
const resetSeedOutput = join(root, 'supabase', 'reset-and-seed-lifelong-stress.sql');
const migrationSql = [
  '0004_canonical_projection_keys.sql',
  '0005_entity_and_review_cleanup.sql',
].map(name => readFileSync(join(root, 'supabase', 'migrations', name),
'utf8')).join('\n\n');
const resetSql = readFileSync(join(root, 'supabase', 'reset-blank-slate.sql'), 'utf8');
```

```
const coreEvents = [
  'My wife is Priya. We married on 27 January 2016. Priya is my family decision partner.',
  'Vaidik is my son. Vaidik was born on 16 March 2021.',
  'Vanya is my daughter. Vanya was born on 9 March 2018.',
  'Priya prefers calm travel planning, clean hotel options, and not deciding family trips in a rush.',
  'I prefer Sunday evening weekly planning because it gives me a clean view of family, work, and personal commitments.',
  'Decision: for the Lifelong AI assistant, memory correctness and source traceability matter more than a fancy chat UI.',
  'Decision: the assistant must never pretend it changed tasks unless the database actually changed.',
  'Goal: build the personal assistant as a lifelong second brain, not a short-term chatbot.',
  'For the family vacation, I need to finalize the destination and compare hotel options.',
  'I should ask Priya about destination options for the family vacation tonight.',
  'The kids homework needs to be completed before school reopens.',
  'Company shutdown is next week. I need to check with Rohit whether the sales update is still required.',
  'Anita from finance asked for revised wealth dashboard projections before Friday.',
  'Sanjay will coordinate shutdown handover. I should send him the open items list.',
  'Meera reminded us that school reopens soon and homework should be submitted on the first day.',
  'Knowledge: Vaidik needs maths and handwriting practice during school reopening prep.',
  'Knowledge: Vanya needs reading practice and art homework during school reopening prep.',
  'Preference: I want fewer, clearer tasks instead of duplicated task noise.',
  'Preference: I like direct answers that cite what memory they used when the answer matters.',
  'Routine: Sunday evening is the best time for a weekly family and work planning review.',
  'Relationship: Rohit is the sales lead I should contact for sales-update questions.',
  'Relationship: Anita is the finance reviewer for wealth dashboard projections.',
  'Relationship: Meera is the school coordinator who gives homework and reopening updates.',
  'Relationship: Sanjay is the operations manager for shutdown handover.',
  'Reminder: check hotel rates tomorrow morning before prices change.',
];

const archiveThemes = [
  'family planning style',
  'school reopening rhythm',
  'assistant reliability',
  'work shutdown coordination',
  'wealth dashboard clarity',
  'travel decision making',
```

```
  'weekly planning habit',
  'memory traceability',
  'task hygiene',
  'calm prioritization',
];

const archiveEvents = Array.from({ length: 95 }, (_, index) => {
  const theme = archiveThemes[index % archiveThemes.length];
  return (
    'Archive note ' + String(index + 1) + ': ' +
    'I noticed a pattern around ' + theme + '. ' +
    'This is background context for future recall, not a new task. ' +
    'The useful signal is to preserve the context without adding duplicate
commitments.'
  );
});

const events = [...coreEvents, ...archiveEvents];

const escaped = value => value.replace(/'/g, "''");

const rows = events.map((payload, index) => {
  const offset = Math.max(1, 60 - index * 0.35).toFixed(2);
  return (
    "('THOUGHTS', '" + escaped(payload) +
    "', '{\"source\":\"stress_seed_v2\"}', NOW() - INTERVAL '" +
    offset + " days')"
  );
});

const evalRows = [
  ['stress', 'Who is Priya?', 'Priya is my wife and family decision partner.', ['Priya',
'wife'], 'profile_fact'],
  ['stress', 'When is Vaidik birthday?', 'Vaidik was born on 16 March 2021.', ['Vaidik',
'16 March 2021'], 'profile_fact'],
  ['stress', 'When is Vanya birthday?', 'Vanya was born on 9 March 2018.', ['Vanya', '9
March 2018'], 'profile_fact'],
  ['stress', 'What is pending for the family vacation?', 'Finalize the destination,
compare hotel options, and ask Priya about destination options.', ['family vacation',
'hotel', 'Priya'], 'commitment'],
  ['stress', 'What did I decide about the assistant project?', 'Memory correctness and
source traceability matter more than a fancy chat UI.', ['memory correctness',
'traceability'], 'decision'],
  ['stress', 'Who should I check with for sales update?', 'Check with Rohit about
whether the sales update is still required.', ['Rohit', 'sales update'], 'work'],
  ['stress', 'What should never happen when I mark a task complete in chat?', 'The
assistant should never pretend it changed tasks unless the database actually
changed.', ['database', 'changed'], 'trust'],
];
```

```
const evalSql = evalRows.map(([suite, question, answer, expected, category]) =>
  "('" + suite + "', '" + escaped(question) + "', '" + escaped(answer) + "', ARRAY[" +
  expected.map(item => "'" + escaped(item) + "').join(',') +
  "], '" + category + "')"
);

const seedSql = `-- Generated lifelong stress seed v2.
-- Controlled coverage: durable facts, a few real tasks, and non-actionable history.

DELETE FROM system_event
WHERE metadata->>'source' IN ('stress_seed', 'stress_seed_v2');

DELETE FROM evaluation_case
WHERE suite = 'stress';

INSERT INTO system_event (source_tab, payload, metadata, recorded_at) VALUES
${rows.join(',\n')};

INSERT INTO evaluation_case (suite, question, expected_answer,
expected_memory, category) VALUES
${evalSql.join(',\n')};

SELECT
  'stress_seed_loaded' AS status,
  COUNT(*) AS events
FROM system_event
WHERE metadata->>'source' = 'stress_seed_v2';
`;

const resetAndSeedSql = [
  '-- Apply duplicate-safe migrations, then reset and seed.',
  migrationSql.trim(),
  resetSql.trim(),
  seedSql.trim(),
  '',
].join('\n\n');

writeFileSync(seedOutput, seedSql);
writeFileSync(resetSeedOutput, resetAndSeedSql);
console.log('Wrote ' + seedOutput);
console.log('Wrote ' + resetSeedOutput);
```

FILE: scripts/generate-synthetic.ts

FILE: scripts/verify-env.mjs

```
const required = [
  'NEXT_PUBLIC_SUPABASE_URL',
  'NEXT_PUBLIC_SUPABASE_ANON_KEY',
  'GROQ_API_KEY',
];

const optional = [
  'SUPABASE_SERVICE_ROLE_KEY',
  'SUPABASE_URL',
];

const missing = required.filter(name => !process.env[name]);

if (missing.length > 0) {
  console.error('Missing required environment variables:');
  for (const name of missing) console.error('- ' + name);
  process.exit(1);
}

console.log('Required app environment variables are present.');

const missingOptional = optional.filter(name => !process.env[name]);
if (missingOptional.length > 0) {
  console.log('Edge/deploy variables not found in this shell: ' + missingOptional.join(',
'));
}
```

FILE: scripts/verify-sql.mjs

```
import { readFileSync, readdirSync } from 'node:fs';
import { join } from 'node:path';

const root = new URL('..', import.meta.url).pathname;
const schemaPath = join(root, 'supabase', 'schema.sql');
const migrationDir = join(root, 'supabase', 'migrations');
const resetPath = join(root, 'supabase', 'reset-blank-slate.sql');

const schema = readFileSync(schemaPath, 'utf8');
const resetSql = readFileSync(resetPath, 'utf8');
const migrations = readdirSync(migrationDir)
  .filter(name => name.endsWith('.sql'))
```

```
  .sort();

const requiredSchemaTokens = [
  'CREATE TABLE system_event',
  'CREATE TABLE task_projection',
  'CREATE TABLE semantic_chunk',
  'CREATE OR REPLACE FUNCTION hybrid_retrieval',
  'CREATE VIEW cognitive_load',
];

const requiredMigrationTokens = [
  'CREATE TABLE IF NOT EXISTS memory_record',
  'CREATE TABLE IF NOT EXISTS memory_review_queue',
  'CREATE TABLE IF NOT EXISTS reminder_schedule',
  'CREATE TABLE IF NOT EXISTS export_log',
  'CREATE OR REPLACE FUNCTION search_memory_records',
  'CREATE OR REPLACE VIEW operational_health',
  'CREATE OR REPLACE VIEW memory_provenance',
  'CREATE OR REPLACE FUNCTION public.canonical_projection_key',
  'CREATE UNIQUE INDEX IF NOT EXISTS uq_task_user_open_key',
  'CREATE UNIQUE INDEX IF NOT EXISTS uq_memory_user_type_key',
  'CREATE UNIQUE INDEX IF NOT EXISTS uq_entity_user_lower_name',
  'CREATE UNIQUE INDEX IF NOT EXISTS uq_identity_flag_pair',
];

const missingSchema = requiredSchemaTokens.filter(token => !
schema.includes(token));
if (missingSchema.length > 0) {
  console.error('schema.sql is missing expected definitions:');
  for (const token of missingSchema) console.error('- ' + token);
  process.exit(1);
}

const migrationText = migrations
  .map(name => readFileSync(join(migrationDir, name), 'utf8'))
  .join('\n');

const missingMigration = requiredMigrationTokens.filter(token => !
migrationText.includes(token));
if (missingMigration.length > 0) {
  console.error('Migrations are missing expected definitions:');
  for (const token of missingMigration) console.error('- ' + token);
  process.exit(1);
}

console.log('SQL inventory verified: ' + migrations.length + ' migration(s).');

if (!resetSql.includes('TRUNCATE TABLE') || !resetSql.includes('blank_slate_ready'))
{
  console.error('reset-blank-slate.sql is missing the expected reset markers.');
```

```
  process.exit(1);
}


--------------------------------------------------------------------------------
FILE: supabase/.temp/linked-project.json
--------------------------------------------------------------------------------
{"ref":"zpobchooknstjkeeiyte","name":"kvimal24-jpg's
Project","organization_id":"zehbellxtztvosyzmiah","organization_slug":"zehbellxtztvos
yzmiah"}

--------------------------------------------------------------------------------
FILE: supabase/cleanup-accidental-sql.sql
--------------------------------------------------------------------------------
-- Optional cleanup for SQL accidentally generated outside this repo.
-- These objects are not used by Lifelong AI.

DROP TABLE IF EXISTS public.knowledge_items CASCADE;

-- Leave public.set_updated_at() in place by default because it is harmless and
-- may be useful elsewhere. Uncomment only if you are sure nothing else uses it.
-- DROP FUNCTION IF EXISTS public.set_updated_at();

SELECT
  'accidental_sql_cleanup_done' AS status,
  to_regclass('public.knowledge_items') AS knowledge_items;


--------------------------------------------------------------------------------
FILE: supabase/cleanup-wife-alias.sql
--------------------------------------------------------------------------------
--
================================================================
==============
-- CLEANUP SCRIPT: Merge "My wife" entity into "Priya"
-- Run this in the Supabase SQL Editor
--
================================================================
==============

DO $$
DECLARE
    v_priya_id UUID;
    v_wife_id UUID;
```

```
    v_alias_count INT;
    v_memory_count INT;
    v_task_count INT;
    v_relation_count INT;
    v_claim_count INT;
BEGIN
    -- 1. Find Priya's canonical entity ID
    SELECT id INTO v_priya_id
    FROM canonical_entity
    WHERE lower(canonical_name) = 'priya'
    LIMIT 1;

    -- 2. Find "My wife" canonical entity ID
    SELECT id INTO v_wife_id
    FROM canonical_entity
    WHERE lower(canonical_name) = 'my wife'
    LIMIT 1;

    IF v_priya_id IS NULL THEN
        RAISE NOTICE 'Entity "Priya" not found. Cannot proceed with merge.';
        RETURN;
    END IF;

    IF v_wife_id IS NULL THEN
        RAISE NOTICE 'Entity "My wife" not found. Nothing to merge.';
        RETURN;
    END IF;

    IF v_priya_id = v_wife_id THEN
        RAISE NOTICE 'They are already the same entity. Nothing to do.';
        RETURN;
    END IF;

    RAISE NOTICE 'Merging "My wife" (%) into "Priya" (%)', v_wife_id, v_priya_id;

    -- 3. Reassign entity aliases
    -- Delete any exact alias duplicates first to avoid unique constraint violations
    DELETE FROM entity_alias
    WHERE canonical_id = v_wife_id
      AND lower(alias_expression) IN (
        SELECT lower(alias_expression) FROM entity_alias WHERE canonical_id =
v_priya_id
      );

    WITH updated AS (
        UPDATE entity_alias SET canonical_id = v_priya_id WHERE canonical_id =
v_wife_id RETURNING 1
    ) SELECT count(*) INTO v_alias_count FROM updated;
    RAISE NOTICE '  - Reassigned % aliases', v_alias_count;
```

```
    -- 4. Reassign memories
    WITH updated AS (
        UPDATE memory_record SET associated_entity_id = v_priya_id WHERE
associated_entity_id = v_wife_id RETURNING 1
    ) SELECT count(*) INTO v_memory_count FROM updated;
    RAISE NOTICE '  - Reassigned % memory records', v_memory_count;

    -- 5. Reassign tasks (projects)
    WITH updated AS (
        UPDATE task_projection SET associated_project = v_priya_id WHERE
associated_project = v_wife_id RETURNING 1
    ) SELECT count(*) INTO v_task_count FROM updated;
    RAISE NOTICE '  - Reassigned % tasks (as project)', v_task_count;

    -- Reassign tasks (assigned person)
    WITH updated AS (
        UPDATE task_projection SET assigned_person = v_priya_id WHERE
assigned_person = v_wife_id RETURNING 1
    ) SELECT count(*) INTO v_task_count FROM updated;
    RAISE NOTICE '  - Reassigned % tasks (as assigned person)', v_task_count;

    -- 6. Reassign relations
    -- Source entity
    WITH updated AS (
        UPDATE entity_relation SET source_entity_id = v_priya_id WHERE
source_entity_id = v_wife_id RETURNING 1
    ) SELECT count(*) INTO v_relation_count FROM updated;
    RAISE NOTICE '  - Reassigned % relations (as source)', v_relation_count;

    -- Target entity
    WITH updated AS (
        UPDATE entity_relation SET target_entity_id = v_priya_id WHERE
target_entity_id = v_wife_id RETURNING 1
    ) SELECT count(*) INTO v_relation_count FROM updated;
    RAISE NOTICE '  - Reassigned % relations (as target)', v_relation_count;

    -- 7. Reassign claims
    WITH updated AS (
        UPDATE epistemic_claim SET subject_entity_id = v_priya_id WHERE
subject_entity_id = v_wife_id RETURNING 1
    ) SELECT count(*) INTO v_claim_count FROM updated;
    RAISE NOTICE '  - Reassigned % epistemic claims', v_claim_count;

    -- 8. Reassign semantic chunks
    WITH updated AS (
        UPDATE semantic_chunk SET associated_entity_id = v_priya_id WHERE
associated_entity_id = v_wife_id RETURNING 1
    ) SELECT count(*) INTO v_claim_count FROM updated;
    RAISE NOTICE '  - Reassigned % semantic chunks', v_claim_count;
```

```
    -- 9. Resolve any pending identity flags involving "My wife"
    UPDATE identity_flag
    SET resolution = 'MERGED', resolved_by = 'SYSTEM', resolved_at = now()
    WHERE (entity_a_id = v_wife_id AND entity_b_id = v_priya_id)
       OR (entity_a_id = v_priya_id AND entity_b_id = v_wife_id);

    -- Delete any other flags involving "My wife" that aren't with Priya
    DELETE FROM identity_flag WHERE entity_a_id = v_wife_id OR entity_b_id =
v_wife_id;

    -- 10. Delete the "My wife" canonical entity
    DELETE FROM canonical_entity WHERE id = v_wife_id;
    RAISE NOTICE 'Deleted canonical entity "My wife". Merge complete.';
END $$;



————————————————————————————————————————————————————————————————————————
FILE: supabase/functions/extract-event/index.ts
————————————————————————————————————————————————————————————————————————
// Deno runtime — uses URL imports, not npm
// Deployed via: supabase functions deploy extract-event

import { createClient } from 'https://esm.sh/@supabase/supabase-js@2';

const MODEL_TAG = 'xenova/all-MiniLM-L6-v2@2.17.2';

interface WebhookPayload {
  type:   string;
  table:  string;
  record: {
    id:          string;
    source_tab:  string;
    payload:     string;
    recorded_at: string;
  };
}

Deno.serve(async (req: Request) => {
  if (req.method !== 'POST') {
    return new Response('Method not allowed', { status: 405 });
  }

  const supabaseUrl  = Deno.env.get('SUPABASE_URL')              ?? '';
  const supabaseKey  = Deno.env.get('SUPABASE_SERVICE_ROLE_KEY') ?? '';
  const supabase = createClient(supabaseUrl, supabaseKey);

  let eventId      = '';
  let eventPayload = '';
```

```
  try {
    const body   = await req.json() as WebhookPayload;
    const record = body.record;

    if (!record) {
      return new Response('No record in payload', { status: 400 });
    }

    eventId      = record.id;
    eventPayload = record.payload ?? '';

    if (record.source_tab === 'SYSTEM') {
      return new Response(JSON.stringify({ skipped: true }), {
        status:  200,
        headers: { 'Content-Type': 'application/json' },
      });
    }

    const sanitised = eventPayload
      .replace(/<[^>]*>/g, '')
      .replace(/[\u0000-\u001F]/g, ' ')
      .trim()
      .slice(0, 6000);

    if (sanitised.length < 5) {
      return new Response(JSON.stringify({ skipped: 'too short' }), {
        status:  200,
        headers: { 'Content-Type': 'application/json' },
      });
    }

    // Always write semantic_chunk with NULL embedding.
    // The Python background job or pg_vector triggers will compute the actual
embeddings.
    await supabase.from('semantic_chunk').insert({
      originating_event_id: eventId,
      chunk_text:           sanitised,
      embedding:             null,
      embedding_model:      MODEL_TAG,
    });

    // NOTE: Graph extraction (entities, tasks, memories) has been migrated to
Next.js
    // Server functions (lib/kg-processor.ts) for synchronous context-aware processing.

    return new Response(
      JSON.stringify({ success: true, chunks: 1 }),
      { status: 200, headers: { 'Content-Type': 'application/json' } }
    );
```

```
  } catch (err) {
    const message = err instanceof Error ? err.message : String(err);
    console.error('[extract-event] Error:', message);

    return new Response(
      JSON.stringify({ success: false, error: message }),
      { status: 200, headers: { 'Content-Type': 'application/json' } }
    );
  }
});
```

FILE: supabase/migrations/0002_lifelong_memory_core.sql

```
-- Lifelong AI durable memory core.
-- Run after the original schema on existing projects.

DO $$ BEGIN
  CREATE TYPE memory_kind AS ENUM (
    'PROFILE_FACT',
    'PREFERENCE',
    'RELATIONSHIP',
    'LIFE_EVENT',
    'ROUTINE',
    'DECISION',
    'COMMITMENT',
    'REMINDER',
    'KNOWLEDGE',
    'GOAL',
    'CORRECTION'
  );
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE memory_status AS ENUM ('ACTIVE', 'SUPERSEDED',
'ARCHIVED', 'REJECTED');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE review_state AS ENUM ('AUTO_ACCEPTED', 'NEEDS_REVIEW',
'APPROVED', 'REJECTED');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;
```

```
DO $$ BEGIN
  CREATE TYPE reminder_state AS ENUM ('SCHEDULED', 'FIRED', 'SNOOZED',
'CANCELLED', 'DONE');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

CREATE TABLE IF NOT EXISTS app_user (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  label       TEXT NOT NULL DEFAULT 'Owner',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  metadata    JSONB NOT NULL DEFAULT '{}'
);

INSERT INTO app_user (id, label, metadata)
VALUES (
  '00000000-0000-4000-8000-000000000001',
  'Owner',
  '{"single_user":true}'::jsonb
)
ON CONFLICT (id) DO NOTHING;

ALTER TABLE system_event
  ADD COLUMN IF NOT EXISTS user_id UUID REFERENCES app_user(id)
  DEFAULT '00000000-0000-4000-8000-000000000001';

ALTER TABLE canonical_entity
  ADD COLUMN IF NOT EXISTS user_id UUID REFERENCES app_user(id)
  DEFAULT '00000000-0000-4000-8000-000000000001';

ALTER TABLE task_projection
  ADD COLUMN IF NOT EXISTS user_id UUID REFERENCES app_user(id)
  DEFAULT '00000000-0000-4000-8000-000000000001';

ALTER TABLE semantic_chunk
  ADD COLUMN IF NOT EXISTS user_id UUID REFERENCES app_user(id)
  DEFAULT '00000000-0000-4000-8000-000000000001';

CREATE TABLE IF NOT EXISTS memory_record (
  id                  UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id               UUID NOT NULL REFERENCES app_user(id)
                      DEFAULT '00000000-0000-4000-8000-000000000001',
  memory_type           memory_kind NOT NULL,
  title               VARCHAR(255) NOT NULL,
  value                 TEXT,
  summary             TEXT NOT NULL,
  source_event_id       UUID REFERENCES system_event(id) ON DELETE SET
NULL,
  associated_entity_id UUID REFERENCES canonical_entity(id) ON DELETE SET
NULL,
  associated_task_id    UUID REFERENCES task_projection(id) ON DELETE SET
```

```
NULL,
  confidence_score     NUMERIC(4,3) NOT NULL DEFAULT 0.700
                       CHECK (confidence_score BETWEEN 0 AND 1),
  review_status        review_state NOT NULL DEFAULT 'AUTO_ACCEPTED',
  current_status       memory_status NOT NULL DEFAULT 'ACTIVE',
  extractor_version    VARCHAR(100) NOT NULL DEFAULT 'manual',
  valid_from           TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  valid_to             TIMESTAMPTZ,
  last_verified_at     TIMESTAMPTZ,
  metadata             JSONB NOT NULL DEFAULT '{}',
  created_at           TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at           TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_memory_user_status
  ON memory_record (user_id, current_status, memory_type, created_at DESC);
CREATE INDEX IF NOT EXISTS idx_memory_source
  ON memory_record (source_event_id);
CREATE INDEX IF NOT EXISTS idx_memory_entity
  ON memory_record (associated_entity_id);
CREATE INDEX IF NOT EXISTS idx_memory_text
  ON memory_record USING gin (
    to_tsvector('english', coalesce(title, '') || ' ' || coalesce(value, '') || ' ' ||
coalesce(summary, ''))
  );
CREATE UNIQUE INDEX IF NOT EXISTS uq_memory_source_type_title
  ON memory_record (source_event_id, memory_type, title);

CREATE TABLE IF NOT EXISTS memory_review_queue (
  id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  memory_id       UUID REFERENCES memory_record(id) ON DELETE CASCADE,
  source_event_id UUID REFERENCES system_event(id) ON DELETE SET NULL,
  reason          TEXT NOT NULL,
  severity        SMALLINT NOT NULL DEFAULT 2 CHECK (severity BETWEEN 1
AND 5),
  status          review_state NOT NULL DEFAULT 'NEEDS_REVIEW',
  created_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  reviewed_at     TIMESTAMPTZ,
  reviewed_by     TEXT,
  notes           TEXT
);

CREATE INDEX IF NOT EXISTS idx_review_status
  ON memory_review_queue (status, severity DESC, created_at DESC);

CREATE TABLE IF NOT EXISTS memory_correction (
  id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  memory_id       UUID REFERENCES memory_record(id) ON DELETE SET NULL,
  source_event_id UUID REFERENCES system_event(id) ON DELETE SET NULL,
  correction_text TEXT NOT NULL,
```

```
  action          VARCHAR(40) NOT NULL CHECK (
                  action IN ('CORRECT', 'MERGE', 'SPLIT', 'ARCHIVE', 'REJECT',
'REEXTRACT')
                ),
  applied         BOOLEAN NOT NULL DEFAULT FALSE,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  applied_at      TIMESTAMPTZ,
  metadata        JSONB NOT NULL DEFAULT '{}'
);

CREATE TABLE IF NOT EXISTS reminder_schedule (
  id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id         UUID NOT NULL REFERENCES app_user(id)
                  DEFAULT '00000000-0000-4000-8000-000000000001',
  task_id         UUID REFERENCES task_projection(id) ON DELETE CASCADE,
  memory_id       UUID REFERENCES memory_record(id) ON DELETE SET NULL,
  source_event_id UUID REFERENCES system_event(id) ON DELETE SET NULL,
  title           VARCHAR(255) NOT NULL,
  reminder_at     TIMESTAMPTZ NOT NULL,
  recurrence_rule TEXT,
  state           reminder_state NOT NULL DEFAULT 'SCHEDULED',
  fired_at        TIMESTAMPTZ,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  metadata        JSONB NOT NULL DEFAULT '{}'
);

CREATE INDEX IF NOT EXISTS idx_reminders_due
  ON reminder_schedule (state, reminder_at)
  WHERE state IN ('SCHEDULED', 'SNOOZED');

CREATE TABLE IF NOT EXISTS ingestion_batch (
  id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  source_label  TEXT NOT NULL,
  source_type   TEXT NOT NULL,
  imported_rows INT NOT NULL DEFAULT 0,
  success_rows  INT NOT NULL DEFAULT 0,
  failed_rows   INT NOT NULL DEFAULT 0,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  completed_at  TIMESTAMPTZ,
  metadata      JSONB NOT NULL DEFAULT '{}'
);

CREATE TABLE IF NOT EXISTS evaluation_case (
  id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  suite           TEXT NOT NULL DEFAULT 'core',
  question        TEXT NOT NULL,
  expected_answer    TEXT NOT NULL,
  expected_memory    TEXT[],
  category        TEXT NOT NULL,
```

```
  created_at         TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS evaluation_run (
  id             UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  suite          TEXT NOT NULL,
  started_at     TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  completed_at   TIMESTAMPTZ,
  passed         INT NOT NULL DEFAULT 0,
  failed         INT NOT NULL DEFAULT 0,
  raw_results    JSONB NOT NULL DEFAULT '{}'
);

CREATE OR REPLACE FUNCTION search_memory_records(
  p_query TEXT,
  p_limit INT DEFAULT 12
)
RETURNS TABLE (
  id UUID,
  memory_type memory_kind,
  title VARCHAR,
  value TEXT,
  summary TEXT,
  confidence_score NUMERIC,
  source_event_id UUID,
  created_at TIMESTAMPTZ,
  score NUMERIC
) AS $$
WITH ranked AS (
  SELECT
    mr.*,
    ts_rank_cd(
      to_tsvector('english', coalesce(mr.title, '') || ' ' || coalesce(mr.value, '') || ' ' ||
coalesce(mr.summary, '')),
      websearch_to_tsquery('english', p_query)
    ) AS ft_score
  FROM memory_record mr
  WHERE mr.current_status = 'ACTIVE'
    AND mr.review_status IN ('AUTO_ACCEPTED', 'APPROVED',
'NEEDS_REVIEW')
    AND (
      to_tsvector('english', coalesce(mr.title, '') || ' ' || coalesce(mr.value, '') || ' ' ||
coalesce(mr.summary, ''))
        @@ websearch_to_tsquery('english', p_query)
      OR mr.title ILIKE '%' || p_query || '%'
      OR mr.value ILIKE '%' || p_query || '%'
      OR mr.summary ILIKE '%' || p_query || '%'
    )
)
SELECT
```

```
  ranked.id,
  ranked.memory_type,
  ranked.title,
  ranked.value,
  ranked.summary,
  ranked.confidence_score,
  ranked.source_event_id,
  ranked.created_at,
  CAST((ranked.ft_score + ranked.confidence_score) AS NUMERIC) AS score
FROM ranked
ORDER BY score DESC, created_at DESC
LIMIT p_limit;
$$ LANGUAGE sql STABLE;

CREATE OR REPLACE VIEW operational_health AS
SELECT
  (SELECT COUNT(*) FROM system_event) AS total_events,
  (SELECT COUNT(*) FROM extraction_log WHERE success = FALSE) AS
failed_extractions,
  (SELECT COUNT(*) FROM semantic_chunk WHERE embedding IS NULL) AS
pending_embeddings,
  (SELECT COUNT(*) FROM memory_review_queue WHERE status =
'NEEDS_REVIEW') AS pending_memory_reviews,
  (SELECT COUNT(*) FROM identity_flag WHERE resolved_at IS NULL) AS
pending_identity_flags,
  (SELECT COUNT(*) FROM reminder_schedule WHERE state IN
('SCHEDULED','SNOOZED') AND reminder_at <= NOW()) AS due_reminders,
  (SELECT COUNT(*) FROM memory_record WHERE current_status = 'ACTIVE')
AS active_memories;
```

FILE: supabase/migrations/0003_projection_idempotency.sql

```
-- Projection idempotency and correction/export support.

CREATE UNIQUE INDEX IF NOT EXISTS uq_task_source_title
  ON task_projection (originating_event_id, title);

ALTER TABLE task_projection
  ADD COLUMN IF NOT EXISTS metadata JSONB NOT NULL DEFAULT '{}';

CREATE UNIQUE INDEX IF NOT EXISTS uq_reminder_source_title_time
  ON reminder_schedule (source_event_id, title, reminder_at);

CREATE TABLE IF NOT EXISTS export_log (
  id           UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  exported_at  TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  table_counts  JSONB NOT NULL DEFAULT '{}',
  metadata      JSONB NOT NULL DEFAULT '{}'
);

CREATE OR REPLACE VIEW memory_provenance AS
SELECT
  mr.id AS memory_id,
  mr.memory_type,
  mr.title,
  mr.summary,
  mr.confidence_score,
  mr.review_status,
  mr.current_status,
  mr.created_at AS memory_created_at,
  se.id AS source_event_id,
  se.recorded_at AS source_recorded_at,
  se.source_tab,
  se.payload AS source_payload
FROM memory_record mr
LEFT JOIN system_event se ON se.id = mr.source_event_id;


--------------------------------------------------------------------------------
FILE: supabase/migrations/0004_canonical_projection_keys.sql
--------------------------------------------------------------------------------
-- Canonical projection keys for duplicate-safe lifelong memory.

CREATE OR REPLACE FUNCTION public.canonical_projection_key(input_text
TEXT)
RETURNS TEXT
LANGUAGE sql
IMMUTABLE
AS $$
  WITH raw_tokens AS (
    SELECT token
    FROM regexp_split_to_table(
      regexp_replace(lower(coalesce(input_text, '')), '[^a-z0-9]+', ' ', 'g'),
      '\s+'
    ) AS token
  ),
  filtered AS (
    SELECT token
    FROM raw_tokens
    WHERE length(token) > 1
      AND token NOT IN (
        'a', 'an', 'and', 'are', 'about', 'for', 'from', 'is', 'of', 'on',
        'the', 'to', 'with', 'task', 'tasks', 'follow', 'up', 'check', 'review'
      )
```

```
  ),
  normalized AS (
    SELECT
      CASE
        WHEN token = 'hotels' THEN 'hotel'
        WHEN token = 'kids' THEN 'kid'
        WHEN token = 'children' THEN 'child'
        WHEN token = 'homeworks' THEN 'homework'
        WHEN token = 'birthdate' THEN 'birthday'
        WHEN token LIKE '%ing' AND length(token) > 5 THEN regexp_replace(token,
'ing$', '')
        WHEN token LIKE '%s' AND length(token) > 4 THEN regexp_replace(token,
's$', '')
        ELSE token
      END AS token
    FROM filtered
  )
  SELECT coalesce(string_agg(token, ' ' ORDER BY token), '')
  FROM normalized;
$$;

ALTER TABLE task_projection
  ADD COLUMN IF NOT EXISTS task_key TEXT;

UPDATE task_projection
SET task_key = public.canonical_projection_key(title)
WHERE task_key IS NULL OR task_key = '';

WITH ranked AS (
  SELECT
    id,
    row_number() OVER (
      PARTITION BY user_id, task_key
      ORDER BY created_at ASC, id ASC
    ) AS rn
  FROM task_projection
  WHERE task_key IS NOT NULL
    AND task_key <> ''
    AND current_status NOT IN ('COMPLETED', 'ABANDONED')
)
UPDATE task_projection tp
SET
  current_status = 'ABANDONED',
  updated_at = CURRENT_TIMESTAMP,
  metadata = coalesce(tp.metadata, '{}'::jsonb)
    || '{"deduped_by_migration_0004":true}'::jsonb
FROM ranked
WHERE tp.id = ranked.id
  AND ranked.rn > 1;
```

```
CREATE UNIQUE INDEX IF NOT EXISTS uq_task_user_open_key
  ON task_projection (user_id, task_key)
  WHERE task_key IS NOT NULL
    AND task_key <> ''
    AND current_status NOT IN ('COMPLETED', 'ABANDONED');

ALTER TABLE memory_record
  ADD COLUMN IF NOT EXISTS memory_key TEXT;

UPDATE memory_record
SET memory_key = public.canonical_projection_key(memory_type::text || ' ' || title)
WHERE memory_key IS NULL OR memory_key = '';

WITH ranked AS (
  SELECT
    id,
    row_number() OVER (
      PARTITION BY user_id, memory_type, memory_key
      ORDER BY confidence_score DESC, created_at ASC, id ASC
    ) AS rn
  FROM memory_record
  WHERE memory_key IS NOT NULL
    AND memory_key <> ''
    AND current_status = 'ACTIVE'
)
UPDATE memory_record mr
SET
  current_status = 'ARCHIVED',
  updated_at = CURRENT_TIMESTAMP,
  metadata = coalesce(mr.metadata, '{}'::jsonb)
    || '{"deduped_by_migration_0004":true}'::jsonb
FROM ranked
WHERE mr.id = ranked.id
  AND ranked.rn > 1;

DROP INDEX IF EXISTS uq_memory_user_active_key;

CREATE UNIQUE INDEX IF NOT EXISTS uq_memory_user_type_key
  ON memory_record (user_id, memory_type, memory_key)
  WHERE memory_key IS NOT NULL
    AND memory_key <> '';
```

FILE: supabase/migrations/0005_entity_and_review_cleanup.sql

```
-- Make entity identity review usable by preventing exact duplicate spam.
```

```
WITH duplicate_names AS (
  SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS keep_id
  FROM canonical_entity
  GROUP BY lower(trim(canonical_name))
  HAVING COUNT(*) > 1
),
duplicates AS (
  SELECT ce.id AS duplicate_id, dn.keep_id
  FROM canonical_entity ce
  JOIN duplicate_names dn ON lower(trim(ce.canonical_name)) = dn.entity_key
  WHERE ce.id <> dn.keep_id
)
DELETE FROM entity_alias ea
USING duplicates
WHERE ea.canonical_id = duplicates.duplicate_id
  AND EXISTS (
    SELECT 1
    FROM entity_alias existing
    WHERE existing.canonical_id = duplicates.keep_id
      AND lower(existing.alias_expression) = lower(ea.alias_expression)
  );

WITH duplicate_names AS (
  SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS keep_id
  FROM canonical_entity
  GROUP BY lower(trim(canonical_name))
  HAVING COUNT(*) > 1
),
duplicates AS (
  SELECT ce.id AS duplicate_id, dn.keep_id
  FROM canonical_entity ce
  JOIN duplicate_names dn ON lower(trim(ce.canonical_name)) = dn.entity_key
  WHERE ce.id <> dn.keep_id
)
UPDATE entity_alias ea
SET canonical_id = duplicates.keep_id
FROM duplicates
WHERE ea.canonical_id = duplicates.duplicate_id;

WITH duplicate_names AS (
  SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS keep_id
  FROM canonical_entity
  GROUP BY lower(trim(canonical_name))
  HAVING COUNT(*) > 1
),
duplicates AS (
  SELECT ce.id AS duplicate_id, dn.keep_id
  FROM canonical_entity ce
  JOIN duplicate_names dn ON lower(trim(ce.canonical_name)) = dn.entity_key
  WHERE ce.id <> dn.keep_id
```

```
)
UPDATE task_projection tp
SET
  associated_project = CASE WHEN associated_project = duplicates.duplicate_id
THEN duplicates.keep_id ELSE associated_project END,
  assigned_person = CASE WHEN assigned_person = duplicates.duplicate_id THEN
duplicates.keep_id ELSE assigned_person END
FROM duplicates
WHERE tp.associated_project = duplicates.duplicate_id
   OR tp.assigned_person = duplicates.duplicate_id;

WITH duplicate_names AS (
  SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS keep_id
  FROM canonical_entity
  GROUP BY lower(trim(canonical_name))
  HAVING COUNT(*) > 1
),
duplicates AS (
  SELECT ce.id AS duplicate_id, dn.keep_id
  FROM canonical_entity ce
  JOIN duplicate_names dn ON lower(trim(ce.canonical_name)) = dn.entity_key
  WHERE ce.id <> dn.keep_id
)
UPDATE memory_record mr
SET associated_entity_id = duplicates.keep_id
FROM duplicates
WHERE mr.associated_entity_id = duplicates.duplicate_id;

WITH duplicate_names AS (
  SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS keep_id
  FROM canonical_entity
  GROUP BY lower(trim(canonical_name))
  HAVING COUNT(*) > 1
),
duplicates AS (
  SELECT ce.id AS duplicate_id, dn.keep_id
  FROM canonical_entity ce
  JOIN duplicate_names dn ON lower(trim(ce.canonical_name)) = dn.entity_key
  WHERE ce.id <> dn.keep_id
)
UPDATE entity_relation er
SET
  source_entity_id = CASE WHEN source_entity_id = duplicates.duplicate_id THEN
duplicates.keep_id ELSE source_entity_id END,
  target_entity_id = CASE WHEN target_entity_id = duplicates.duplicate_id THEN
duplicates.keep_id ELSE target_entity_id END
FROM duplicates
WHERE er.source_entity_id = duplicates.duplicate_id
   OR er.target_entity_id = duplicates.duplicate_id;
```

```
WITH duplicate_names AS (
  SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS keep_id
  FROM canonical_entity
  GROUP BY lower(trim(canonical_name))
  HAVING COUNT(*) > 1
),
duplicates AS (
  SELECT ce.id AS duplicate_id, dn.keep_id
  FROM canonical_entity ce
  JOIN duplicate_names dn ON lower(trim(ce.canonical_name)) = dn.entity_key
  WHERE ce.id <> dn.keep_id
)
UPDATE epistemic_claim ec
SET subject_entity_id = duplicates.keep_id
FROM duplicates
WHERE ec.subject_entity_id = duplicates.duplicate_id;

UPDATE identity_flag f
SET
  resolution = 'MERGED',
  resolved_at = CURRENT_TIMESTAMP,
  resolved_by = 'SYSTEM'
FROM canonical_entity a, canonical_entity b
WHERE f.entity_a_id = a.id
  AND f.entity_b_id = b.id
  AND lower(trim(a.canonical_name)) = lower(trim(b.canonical_name))
  AND f.resolved_at IS NULL;

DELETE FROM canonical_entity ce
USING (
  SELECT ce.id AS duplicate_id
  FROM canonical_entity ce
  JOIN (
    SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS
keep_id
    FROM canonical_entity
    GROUP BY lower(trim(canonical_name))
    HAVING COUNT(*) > 1
  ) keepers ON lower(trim(ce.canonical_name)) = keepers.entity_key
  WHERE ce.id <> keepers.keep_id
) duplicates
WHERE ce.id = duplicates.duplicate_id;

CREATE UNIQUE INDEX IF NOT EXISTS uq_entity_user_lower_name
  ON canonical_entity (user_id, lower(trim(canonical_name)));

CREATE UNIQUE INDEX IF NOT EXISTS uq_identity_flag_pair
  ON identity_flag (
    least(entity_a_id::text, entity_b_id::text),
    greatest(entity_a_id::text, entity_b_id::text)
```

```
);
```

FILE: supabase/migrations/20260621072130_fts_index.sql

```
ALTER TABLE memory_record
ADD COLUMN IF NOT EXISTS fts_vector tsvector GENERATED ALWAYS AS (
  to_tsvector('english', coalesce(title, '') || ' ' || coalesce(value, '') || ' ' ||
coalesce(summary, ''))
) STORED;

CREATE INDEX IF NOT EXISTS memory_record_fts_idx ON memory_record
USING GIN (fts_vector);

CREATE OR REPLACE FUNCTION search_memory_records(
  p_query TEXT,
  p_limit INT DEFAULT 12
)
RETURNS TABLE (
  id UUID,
  memory_type memory_kind,
  title VARCHAR,
  value TEXT,
  summary TEXT,
  confidence_score NUMERIC,
  source_event_id UUID,
  created_at TIMESTAMPTZ,
  score NUMERIC
) AS $$
WITH ranked AS (
  SELECT
    mr.*,
    ts_rank_cd(mr.fts_vector, websearch_to_tsquery('english', p_query)) AS ft_score
  FROM memory_record mr
  WHERE mr.current_status = 'ACTIVE'
    AND mr.review_status IN ('AUTO_ACCEPTED', 'APPROVED',
'NEEDS_REVIEW')
    AND (
      mr.fts_vector @@ websearch_to_tsquery('english', p_query)
      OR mr.title ILIKE '%' || p_query || '%'
      OR mr.value ILIKE '%' || p_query || '%'
      OR mr.summary ILIKE '%' || p_query || '%'
    )
)
SELECT
  ranked.id,
  ranked.memory_type,
```

```
  ranked.title,
  ranked.value,
  ranked.summary,
  ranked.confidence_score,
  ranked.source_event_id,
  ranked.created_at,
  CAST((ranked.ft_score + ranked.confidence_score) AS NUMERIC) AS score
FROM ranked
ORDER BY score DESC, created_at DESC
LIMIT p_limit;
$$ LANGUAGE sql STABLE;


--------------------------------------------------------------------------------
FILE: supabase/reset-and-seed-lifelong-stress.sql
--------------------------------------------------------------------------------
-- Apply duplicate-safe migrations, then reset and seed.

-- Canonical projection keys for duplicate-safe lifelong memory.

CREATE OR REPLACE FUNCTION public.canonical_projection_key(input_text
TEXT)
RETURNS TEXT
LANGUAGE sql
IMMUTABLE
AS $$
  WITH raw_tokens AS (
    SELECT token
    FROM regexp_split_to_table(
      regexp_replace(lower(coalesce(input_text, '')), '[^a-z0-9]+', ' ', 'g'),
      '\s+'
    ) AS token
  ),
  filtered AS (
    SELECT token
    FROM raw_tokens
    WHERE length(token) > 1
      AND token NOT IN (
        'a', 'an', 'and', 'are', 'about', 'for', 'from', 'is', 'of', 'on',
        'the', 'to', 'with', 'task', 'tasks', 'follow', 'up', 'check', 'review'
      )
  ),
  normalized AS (
    SELECT
      CASE
        WHEN token = 'hotels' THEN 'hotel'
        WHEN token = 'kids' THEN 'kid'
        WHEN token = 'children' THEN 'child'
```

```
        WHEN token = 'homeworks' THEN 'homework'
        WHEN token = 'birthdate' THEN 'birthday'
        WHEN token LIKE '%ing' AND length(token) > 5 THEN regexp_replace(token,
'ing$', '')
        WHEN token LIKE '%s' AND length(token) > 4 THEN regexp_replace(token,
's$', '')
        ELSE token
      END AS token
    FROM filtered
  )
  SELECT coalesce(string_agg(token, ' ' ORDER BY token), '')
  FROM normalized;
$$;

ALTER TABLE task_projection
  ADD COLUMN IF NOT EXISTS task_key TEXT;

UPDATE task_projection
SET task_key = public.canonical_projection_key(title)
WHERE task_key IS NULL OR task_key = '';

WITH ranked AS (
  SELECT
    id,
    row_number() OVER (
      PARTITION BY user_id, task_key
      ORDER BY created_at ASC, id ASC
    ) AS rn
  FROM task_projection
  WHERE task_key IS NOT NULL
    AND task_key <> ''
    AND current_status NOT IN ('COMPLETED', 'ABANDONED')
)
UPDATE task_projection tp
SET
  current_status = 'ABANDONED',
  updated_at = CURRENT_TIMESTAMP,
  metadata = coalesce(tp.metadata, '{}'::jsonb)
    || '{"deduped_by_migration_0004":true}'::jsonb
FROM ranked
WHERE tp.id = ranked.id
  AND ranked.rn > 1;

CREATE UNIQUE INDEX IF NOT EXISTS uq_task_user_open_key
  ON task_projection (user_id, task_key)
  WHERE task_key IS NOT NULL
    AND task_key <> ''
    AND current_status NOT IN ('COMPLETED', 'ABANDONED');

ALTER TABLE memory_record
```

```
  ADD COLUMN IF NOT EXISTS memory_key TEXT;

UPDATE memory_record
SET memory_key = public.canonical_projection_key(memory_type::text || ' ' || title)
WHERE memory_key IS NULL OR memory_key = '';

WITH ranked AS (
  SELECT
    id,
    row_number() OVER (
      PARTITION BY user_id, memory_type, memory_key
      ORDER BY confidence_score DESC, created_at ASC, id ASC
    ) AS rn
  FROM memory_record
  WHERE memory_key IS NOT NULL
    AND memory_key <> ''
    AND current_status = 'ACTIVE'
)
UPDATE memory_record mr
SET
  current_status = 'ARCHIVED',
  updated_at = CURRENT_TIMESTAMP,
  metadata = coalesce(mr.metadata, '{}'::jsonb)
    || '{"deduped_by_migration_0004":true}'::jsonb
FROM ranked
WHERE mr.id = ranked.id
  AND ranked.rn > 1;

DROP INDEX IF EXISTS uq_memory_user_active_key;

CREATE UNIQUE INDEX IF NOT EXISTS uq_memory_user_type_key
  ON memory_record (user_id, memory_type, memory_key)
  WHERE memory_key IS NOT NULL
    AND memory_key <> '';


-- Make entity identity review usable by preventing exact duplicate spam.

WITH duplicate_names AS (
  SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS keep_id
  FROM canonical_entity
  GROUP BY lower(trim(canonical_name))
  HAVING COUNT(*) > 1
),
duplicates AS (
  SELECT ce.id AS duplicate_id, dn.keep_id
  FROM canonical_entity ce
  JOIN duplicate_names dn ON lower(trim(ce.canonical_name)) = dn.entity_key
  WHERE ce.id <> dn.keep_id
)
```

```
DELETE FROM entity_alias ea
USING duplicates
WHERE ea.canonical_id = duplicates.duplicate_id
  AND EXISTS (
    SELECT 1
    FROM entity_alias existing
    WHERE existing.canonical_id = duplicates.keep_id
      AND lower(existing.alias_expression) = lower(ea.alias_expression)
  );

WITH duplicate_names AS (
  SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS keep_id
  FROM canonical_entity
  GROUP BY lower(trim(canonical_name))
  HAVING COUNT(*) > 1
),
duplicates AS (
  SELECT ce.id AS duplicate_id, dn.keep_id
  FROM canonical_entity ce
  JOIN duplicate_names dn ON lower(trim(ce.canonical_name)) = dn.entity_key
  WHERE ce.id <> dn.keep_id
)
UPDATE entity_alias ea
SET canonical_id = duplicates.keep_id
FROM duplicates
WHERE ea.canonical_id = duplicates.duplicate_id;

WITH duplicate_names AS (
  SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS keep_id
  FROM canonical_entity
  GROUP BY lower(trim(canonical_name))
  HAVING COUNT(*) > 1
),
duplicates AS (
  SELECT ce.id AS duplicate_id, dn.keep_id
  FROM canonical_entity ce
  JOIN duplicate_names dn ON lower(trim(ce.canonical_name)) = dn.entity_key
  WHERE ce.id <> dn.keep_id
)
UPDATE task_projection tp
SET
  associated_project = CASE WHEN associated_project = duplicates.duplicate_id
THEN duplicates.keep_id ELSE associated_project END,
  assigned_person = CASE WHEN assigned_person = duplicates.duplicate_id THEN
duplicates.keep_id ELSE assigned_person END
FROM duplicates
WHERE tp.associated_project = duplicates.duplicate_id
   OR tp.assigned_person = duplicates.duplicate_id;

WITH duplicate_names AS (
```

```
  SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS keep_id
  FROM canonical_entity
  GROUP BY lower(trim(canonical_name))
  HAVING COUNT(*) > 1
),
duplicates AS (
  SELECT ce.id AS duplicate_id, dn.keep_id
  FROM canonical_entity ce
  JOIN duplicate_names dn ON lower(trim(ce.canonical_name)) = dn.entity_key
  WHERE ce.id <> dn.keep_id
)
UPDATE memory_record mr
SET associated_entity_id = duplicates.keep_id
FROM duplicates
WHERE mr.associated_entity_id = duplicates.duplicate_id;

WITH duplicate_names AS (
  SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS keep_id
  FROM canonical_entity
  GROUP BY lower(trim(canonical_name))
  HAVING COUNT(*) > 1
),
duplicates AS (
  SELECT ce.id AS duplicate_id, dn.keep_id
  FROM canonical_entity ce
  JOIN duplicate_names dn ON lower(trim(ce.canonical_name)) = dn.entity_key
  WHERE ce.id <> dn.keep_id
)
UPDATE entity_relation er
SET
  source_entity_id = CASE WHEN source_entity_id = duplicates.duplicate_id THEN
duplicates.keep_id ELSE source_entity_id END,
  target_entity_id = CASE WHEN target_entity_id = duplicates.duplicate_id THEN
duplicates.keep_id ELSE target_entity_id END
FROM duplicates
WHERE er.source_entity_id = duplicates.duplicate_id
   OR er.target_entity_id = duplicates.duplicate_id;

WITH duplicate_names AS (
  SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS keep_id
  FROM canonical_entity
  GROUP BY lower(trim(canonical_name))
  HAVING COUNT(*) > 1
),
duplicates AS (
  SELECT ce.id AS duplicate_id, dn.keep_id
  FROM canonical_entity ce
  JOIN duplicate_names dn ON lower(trim(ce.canonical_name)) = dn.entity_key
  WHERE ce.id <> dn.keep_id
)
```

```
UPDATE epistemic_claim ec
SET subject_entity_id = duplicates.keep_id
FROM duplicates
WHERE ec.subject_entity_id = duplicates.duplicate_id;

UPDATE identity_flag f
SET
  resolution = 'MERGED',
  resolved_at = CURRENT_TIMESTAMP,
  resolved_by = 'SYSTEM'
FROM canonical_entity a, canonical_entity b
WHERE f.entity_a_id = a.id
  AND f.entity_b_id = b.id
  AND lower(trim(a.canonical_name)) = lower(trim(b.canonical_name))
  AND f.resolved_at IS NULL;

DELETE FROM canonical_entity ce
USING (
  SELECT ce.id AS duplicate_id
  FROM canonical_entity ce
  JOIN (
    SELECT lower(trim(canonical_name)) AS entity_key, min(id::text)::uuid AS
keep_id
    FROM canonical_entity
    GROUP BY lower(trim(canonical_name))
    HAVING COUNT(*) > 1
  ) keepers ON lower(trim(ce.canonical_name)) = keepers.entity_key
  WHERE ce.id <> keepers.keep_id
) duplicates
WHERE ce.id = duplicates.duplicate_id;

CREATE UNIQUE INDEX IF NOT EXISTS uq_entity_user_lower_name
  ON canonical_entity (user_id, lower(trim(canonical_name)));

CREATE UNIQUE INDEX IF NOT EXISTS uq_identity_flag_pair
  ON identity_flag (
    least(entity_a_id::text, entity_b_id::text),
    greatest(entity_a_id::text, entity_b_id::text)
  );

-- Lifelong AI blank-slate reset.
-- Use this only when you want to keep the schema/tables but remove all app data.
-- Do not run this if you need to preserve existing memories, thoughts, tasks, or logs.

DO $$
DECLARE
  table_names TEXT[] := ARRAY[
    'context_assembly_log',
    'response_cache',
    'memory_review_queue',
```

```
    'memory_correction',
    'reminder_schedule',
    'memory_record',
    'semantic_chunk',
    'epistemic_claim',
    'goal_conflict',
    'identity_flag',
    'entity_relation',
    'entity_alias',
    'task_projection',
    'canonical_entity',
    'extraction_log',
    'cognitive_health_log',
    'user_identity_snapshot',
    'ingestion_batch',
    'evaluation_run',
    'evaluation_case',
    'export_log',
    'system_event',
    'app_user'
  ];
  existing_tables TEXT;
BEGIN
  SELECT string_agg(format('public.%I', table_name), ', ')
  INTO existing_tables
  FROM unnest(table_names) AS t(table_name)
  WHERE to_regclass('public.' || table_name) IS NOT NULL;

  IF existing_tables IS NOT NULL THEN
    EXECUTE 'TRUNCATE TABLE ' || existing_tables || ' RESTART IDENTITY
CASCADE';
  END IF;
END $$;

INSERT INTO app_user (id, label, metadata)
VALUES (
  '00000000-0000-4000-8000-000000000001',
  'Owner',
  '{"single_user":true}'::jsonb
)
ON CONFLICT (id) DO NOTHING;

SELECT
  'blank_slate_ready' AS status,
  (SELECT COUNT(*) FROM system_event) AS events,
  (SELECT COUNT(*) FROM memory_record) AS memories,
  (SELECT COUNT(*) FROM task_projection) AS tasks;

-- Generated lifelong stress seed v2.
-- Controlled coverage: durable facts, a few real tasks, and non-actionable history.
```

```
DELETE FROM system_event
WHERE metadata->>'source' IN ('stress_seed', 'stress_seed_v2');

DELETE FROM evaluation_case
WHERE suite = 'stress';

INSERT INTO system_event (source_tab, payload, metadata, recorded_at) VALUES
('THOUGHTS', 'My wife is Priya. We married on 27 January 2016. Priya is my family decision partner.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '60.00 days'),
('THOUGHTS', 'Vaidik is my son. Vaidik was born on 16 March 2021.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '59.65 days'),
('THOUGHTS', 'Vanya is my daughter. Vanya was born on 9 March 2018.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '59.30 days'),
('THOUGHTS', 'Priya prefers calm travel planning, clean hotel options, and not deciding family trips in a rush.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '58.95 days'),
('THOUGHTS', 'I prefer Sunday evening weekly planning because it gives me a clean view of family, work, and personal commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '58.60 days'),
('THOUGHTS', 'Decision: for the Lifelong AI assistant, memory correctness and source traceability matter more than a fancy chat UI.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '58.25 days'),
('THOUGHTS', 'Decision: the assistant must never pretend it changed tasks unless the database actually changed.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '57.90 days'),
('THOUGHTS', 'Goal: build the personal assistant as a lifelong second brain, not a short-term chatbot.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '57.55 days'),
('THOUGHTS', 'For the family vacation, I need to finalize the destination and compare hotel options.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '57.20 days'),
('THOUGHTS', 'I should ask Priya about destination options for the family vacation tonight.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '56.85 days'),
('THOUGHTS', 'The kids homework needs to be completed before school reopens.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '56.50 days'),
('THOUGHTS', 'Company shutdown is next week. I need to check with Rohit whether the sales update is still required.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '56.15 days'),
('THOUGHTS', 'Anita from finance asked for revised wealth dashboard projections before Friday.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '55.80 days'),
('THOUGHTS', 'Sanjay will coordinate shutdown handover. I should send him the open items list.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '55.45 days'),
('THOUGHTS', 'Meera reminded us that school reopens soon and homework should be submitted on the first day.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '55.10 days'),
('THOUGHTS', 'Knowledge: Vaidik needs maths and handwriting practice during school reopening prep.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '54.75 days'),
('THOUGHTS', 'Knowledge: Vanya needs reading practice and art homework during school reopening prep.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '54.40
```

days'),
('THOUGHTS', 'Preference: I want fewer, clearer tasks instead of duplicated task noise.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '54.05 days'),
('THOUGHTS', 'Preference: I like direct answers that cite what memory they used when the answer matters.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '53.70 days'),
('THOUGHTS', 'Routine: Sunday evening is the best time for a weekly family and work planning review.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '53.35 days'),
('THOUGHTS', 'Relationship: Rohit is the sales lead I should contact for sales-update questions.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '53.00 days'),
('THOUGHTS', 'Relationship: Anita is the finance reviewer for wealth dashboard projections.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '52.65 days'),
('THOUGHTS', 'Relationship: Meera is the school coordinator who gives homework and reopening updates.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '52.30 days'),
('THOUGHTS', 'Relationship: Sanjay is the operations manager for shutdown handover.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '51.95 days'),
('THOUGHTS', 'Reminder: check hotel rates tomorrow morning before prices change.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '51.60 days'),
('THOUGHTS', 'Archive note 1: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '51.25 days'),
('THOUGHTS', 'Archive note 2: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '50.90 days'),
('THOUGHTS', 'Archive note 3: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '50.55 days'),
('THOUGHTS', 'Archive note 4: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '50.20 days'),
('THOUGHTS', 'Archive note 5: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '49.85 days'),
('THOUGHTS', 'Archive note 6: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '49.50 days'),
('THOUGHTS', 'Archive note 7: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '49.15 days'),
('THOUGHTS', 'Archive note 8: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to

preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '48.80 days'),
('THOUGHTS', 'Archive note 9: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '48.45 days'),
('THOUGHTS', 'Archive note 10: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '48.10 days'),
('THOUGHTS', 'Archive note 11: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '47.75 days'),
('THOUGHTS', 'Archive note 12: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '47.40 days'),
('THOUGHTS', 'Archive note 13: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '47.05 days'),
('THOUGHTS', 'Archive note 14: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '46.70 days'),
('THOUGHTS', 'Archive note 15: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '46.35 days'),
('THOUGHTS', 'Archive note 16: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '46.00 days'),
('THOUGHTS', 'Archive note 17: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '45.65 days'),
('THOUGHTS', 'Archive note 18: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '45.30 days'),
('THOUGHTS', 'Archive note 19: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '44.95 days'),
('THOUGHTS', 'Archive note 20: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '44.60 days'),

('THOUGHTS', 'Archive note 21: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '44.25 days'),
('THOUGHTS', 'Archive note 22: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '43.90 days'),
('THOUGHTS', 'Archive note 23: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '43.55 days'),
('THOUGHTS', 'Archive note 24: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '43.20 days'),
('THOUGHTS', 'Archive note 25: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '42.85 days'),
('THOUGHTS', 'Archive note 26: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '42.50 days'),
('THOUGHTS', 'Archive note 27: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '42.15 days'),
('THOUGHTS', 'Archive note 28: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '41.80 days'),
('THOUGHTS', 'Archive note 29: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '41.45 days'),
('THOUGHTS', 'Archive note 30: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '41.10 days'),
('THOUGHTS', 'Archive note 31: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '40.75 days'),
('THOUGHTS', 'Archive note 32: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '40.40 days'),
('THOUGHTS', 'Archive note 33: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to

preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '40.05 days'),
('THOUGHTS', 'Archive note 34: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '39.70 days'),
('THOUGHTS', 'Archive note 35: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '39.35 days'),
('THOUGHTS', 'Archive note 36: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '39.00 days'),
('THOUGHTS', 'Archive note 37: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '38.65 days'),
('THOUGHTS', 'Archive note 38: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '38.30 days'),
('THOUGHTS', 'Archive note 39: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '37.95 days'),
('THOUGHTS', 'Archive note 40: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '37.60 days'),
('THOUGHTS', 'Archive note 41: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '37.25 days'),
('THOUGHTS', 'Archive note 42: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '36.90 days'),
('THOUGHTS', 'Archive note 43: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '36.55 days'),
('THOUGHTS', 'Archive note 44: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '36.20 days'),
('THOUGHTS', 'Archive note 45: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '35.85 days'),

('THOUGHTS', 'Archive note 46: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '35.50 days'),
('THOUGHTS', 'Archive note 47: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '35.15 days'),
('THOUGHTS', 'Archive note 48: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '34.80 days'),
('THOUGHTS', 'Archive note 49: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '34.45 days'),
('THOUGHTS', 'Archive note 50: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '34.10 days'),
('THOUGHTS', 'Archive note 51: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '33.75 days'),
('THOUGHTS', 'Archive note 52: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '33.40 days'),
('THOUGHTS', 'Archive note 53: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '33.05 days'),
('THOUGHTS', 'Archive note 54: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '32.70 days'),
('THOUGHTS', 'Archive note 55: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '32.35 days'),
('THOUGHTS', 'Archive note 56: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '32.00 days'),
('THOUGHTS', 'Archive note 57: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '31.65 days'),
('THOUGHTS', 'Archive note 58: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to

preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '31.30 days'),
('THOUGHTS', 'Archive note 59: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '30.95 days'),
('THOUGHTS', 'Archive note 60: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '30.60 days'),
('THOUGHTS', 'Archive note 61: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '30.25 days'),
('THOUGHTS', 'Archive note 62: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '29.90 days'),
('THOUGHTS', 'Archive note 63: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '29.55 days'),
('THOUGHTS', 'Archive note 64: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '29.20 days'),
('THOUGHTS', 'Archive note 65: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '28.85 days'),
('THOUGHTS', 'Archive note 66: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '28.50 days'),
('THOUGHTS', 'Archive note 67: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '28.15 days'),
('THOUGHTS', 'Archive note 68: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '27.80 days'),
('THOUGHTS', 'Archive note 69: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '27.45 days'),
('THOUGHTS', 'Archive note 70: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '27.10 days'),

('THOUGHTS', 'Archive note 71: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '26.75 days'),
('THOUGHTS', 'Archive note 72: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '26.40 days'),
('THOUGHTS', 'Archive note 73: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '26.05 days'),
('THOUGHTS', 'Archive note 74: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '25.70 days'),
('THOUGHTS', 'Archive note 75: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '25.35 days'),
('THOUGHTS', 'Archive note 76: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '25.00 days'),
('THOUGHTS', 'Archive note 77: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '24.65 days'),
('THOUGHTS', 'Archive note 78: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '24.30 days'),
('THOUGHTS', 'Archive note 79: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '23.95 days'),
('THOUGHTS', 'Archive note 80: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '23.60 days'),
('THOUGHTS', 'Archive note 81: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '23.25 days'),
('THOUGHTS', 'Archive note 82: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '22.90 days'),
('THOUGHTS', 'Archive note 83: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to

preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '22.55 days'),
('THOUGHTS', 'Archive note 84: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '22.20 days'),
('THOUGHTS', 'Archive note 85: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '21.85 days'),
('THOUGHTS', 'Archive note 86: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '21.50 days'),
('THOUGHTS', 'Archive note 87: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '21.15 days'),
('THOUGHTS', 'Archive note 88: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '20.80 days'),
('THOUGHTS', 'Archive note 89: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '20.45 days'),
('THOUGHTS', 'Archive note 90: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '20.10 days'),
('THOUGHTS', 'Archive note 91: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '19.75 days'),
('THOUGHTS', 'Archive note 92: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '19.40 days'),
('THOUGHTS', 'Archive note 93: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '19.05 days'),
('THOUGHTS', 'Archive note 94: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '18.70 days'),
('THOUGHTS', 'Archive note 95: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '18.35 days');

```
INSERT INTO evaluation_case (suite, question, expected_answer,
expected_memory, category) VALUES
('stress', 'Who is Priya?', 'Priya is my wife and family decision partner.',
ARRAY['Priya','wife'], 'profile_fact'),
('stress', 'When is Vaidik birthday?', 'Vaidik was born on 16 March 2021.',
ARRAY['Vaidik','16 March 2021'], 'profile_fact'),
('stress', 'When is Vanya birthday?', 'Vanya was born on 9 March 2018.',
ARRAY['Vanya','9 March 2018'], 'profile_fact'),
('stress', 'What is pending for the family vacation?', 'Finalize the destination, compare
hotel options, and ask Priya about destination options.', ARRAY['family
vacation','hotel','Priya'], 'commitment'),
('stress', 'What did I decide about the assistant project?', 'Memory correctness and
source traceability matter more than a fancy chat UI.', ARRAY['memory
correctness','traceability'], 'decision'),
('stress', 'Who should I check with for sales update?', 'Check with Rohit about
whether the sales update is still required.', ARRAY['Rohit','sales update'], 'work'),
('stress', 'What should never happen when I mark a task complete in chat?', 'The
assistant should never pretend it changed tasks unless the database actually
changed.', ARRAY['database','changed'], 'trust');

SELECT
  'stress_seed_loaded' AS status,
  COUNT(*) AS events
FROM system_event
WHERE metadata->>'source' = 'stress_seed_v2';
```

FILE: supabase/reset-blank-slate.sql

```
-- Lifelong AI blank-slate reset.
-- Use this only when you want to keep the schema/tables but remove all app data.
-- Do not run this if you need to preserve existing memories, thoughts, tasks, or logs.

DO $$
DECLARE
  table_names TEXT[] := ARRAY[
    'context_assembly_log',
    'response_cache',
    'memory_review_queue',
    'memory_correction',
    'reminder_schedule',
    'memory_record',
    'semantic_chunk',
    'epistemic_claim',
    'goal_conflict',
```

```
    'identity_flag',
    'entity_relation',
    'entity_alias',
    'task_projection',
    'canonical_entity',
    'extraction_log',
    'cognitive_health_log',
    'user_identity_snapshot',
    'ingestion_batch',
    'evaluation_run',
    'evaluation_case',
    'export_log',
    'system_event',
    'app_user'
  ];
  existing_tables TEXT;
BEGIN
  SELECT string_agg(format('public.%I', table_name), ', ')
  INTO existing_tables
  FROM unnest(table_names) AS t(table_name)
  WHERE to_regclass('public.' || table_name) IS NOT NULL;

  IF existing_tables IS NOT NULL THEN
    EXECUTE 'TRUNCATE TABLE ' || existing_tables || ' RESTART IDENTITY
CASCADE';
  END IF;
END $$;

INSERT INTO app_user (id, label, metadata)
VALUES (
  '00000000-0000-4000-8000-000000000001',
  'Owner',
  '{"single_user":true}'::jsonb
)
ON CONFLICT (id) DO NOTHING;

SELECT
  'blank_slate_ready' AS status,
  (SELECT COUNT(*) FROM system_event) AS events,
  (SELECT COUNT(*) FROM memory_record) AS memories,
  (SELECT COUNT(*) FROM task_projection) AS tasks;


-------------------------------------------------------------------------------
-------------------------
FILE: supabase/schema.sql
-------------------------------------------------------------------------------
-------------------------
--
-------------------------------------------------------------------------------
```

```
-- LIFELONG AI — Complete Database Schema
-- Run this entire file in Supabase SQL Editor (one paste, one click Run)
--

-- ── Extensions ──────────────────────────────────────────────
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "vector";
CREATE EXTENSION IF NOT EXISTS "pg_cron";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";  -- for fuzzy name matching

-- ── Enums ───────────────────────────────────────────────────
CREATE TYPE tab_source     AS ENUM ('THOUGHTS','CHAT','TASKS','SYSTEM');
CREATE TYPE task_state     AS ENUM
('PENDING','ACTIVE','BLOCKED','COMPLETED','ABANDONED');
CREATE TYPE entity_class   AS ENUM
('PERSON','PROJECT','GOAL','CONCEPT','ORG');
CREATE TYPE epistemic_type AS ENUM
('ASSERTION','INFERENCE','INTENTION','PREDICTION','BELIEF','OBSERVATION
','VERIFIED_FACT');
CREATE TYPE trust_level    AS ENUM
('USER_STATED','AI_INFERRED','BEHAVIORALLY_OBSERVED','EXTERNALLY_V
ERIFIED');

-- ── 1. Immutable Event Log ──────────────────────────────────
-- The absolute source of truth. Never modified, only appended.
CREATE TABLE system_event (
  id          UUID        PRIMARY KEY DEFAULT uuid_generate_v4(),
  recorded_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  source_tab  tab_source  NOT NULL,
  payload     TEXT        NOT NULL,
  metadata    JSONB       NOT NULL DEFAULT '{}',
  compacted   BOOLEAN     NOT NULL DEFAULT FALSE
);
CREATE INDEX idx_event_time    ON system_event (recorded_at DESC);
CREATE INDEX idx_event_tab     ON system_event (source_tab, recorded_at
DESC);
CREATE INDEX idx_event_compact ON system_event (compacted) WHERE
compacted = FALSE;
CREATE INDEX idx_event_meta    ON system_event USING gin (metadata);
```

```
-- ─── 2. Canonical Entity Registry ──────────────────────────────────────────────

CREATE TABLE canonical_entity (
  id              UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
  classification  entity_class  NOT NULL,
  canonical_name  VARCHAR(255)  NOT NULL,
  summary_portrait TEXT,
  created_at      TIMESTAMPTZ   NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at      TIMESTAMPTZ   NOT NULL DEFAULT CURRENT_TIMESTAMP,
  system_metadata JSONB         NOT NULL DEFAULT '{}'
);
CREATE INDEX idx_entity_class ON canonical_entity (classification);
CREATE INDEX idx_entity_name  ON canonical_entity USING gin (canonical_name
gin_trgm_ops);

-- ─── 3. Entity Alias Ledger ────────────────────────────────────────────────────

CREATE TABLE entity_alias (
  id              UUID         PRIMARY KEY DEFAULT uuid_generate_v4(),
  canonical_id    UUID         NOT NULL REFERENCES canonical_entity(id) ON
DELETE CASCADE,
  alias_expression VARCHAR(255) NOT NULL,
  confidence_score NUMERIC(4,3) CHECK (confidence_score BETWEEN 0 AND 1),
  auto_merge_flag  BOOLEAN      NOT NULL DEFAULT FALSE,  -- always FALSE in
v1
  merge_reviewed   BOOLEAN      NOT NULL DEFAULT FALSE,
  created_at       TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(canonical_id, alias_expression)
);
CREATE INDEX idx_alias_expr       ON entity_alias (lower(alias_expression));
CREATE INDEX idx_alias_trgm       ON entity_alias USING gin (alias_expression
gin_trgm_ops);
CREATE INDEX idx_alias_unreviewed ON entity_alias (merge_reviewed) WHERE
merge_reviewed = FALSE;

-- ─── 4. Identity Resolution Flags (manual review queue) ────────────────────

CREATE TABLE identity_flag (
  id          UUID        PRIMARY KEY DEFAULT uuid_generate_v4(),
  entity_a_id  UUID       NOT NULL REFERENCES canonical_entity(id) ON DELETE
CASCADE,
  entity_b_id  UUID       NOT NULL REFERENCES canonical_entity(id) ON DELETE
CASCADE,
  similarity   NUMERIC(4,3),
  flag_reason  TEXT,
  flagged_at   TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  resolved_at  TIMESTAMPTZ,
```

```
  resolution   VARCHAR(20) CHECK (resolution IN
('MERGED','KEPT_SEPARATE','PENDING')),
  resolved_by  VARCHAR(20) CHECK (resolved_by IN ('USER','SYSTEM'))
);
CREATE INDEX idx_flag_pending ON identity_flag (resolved_at) WHERE
resolved_at IS NULL;

-- ─── 5. Temporal Knowledge Graph Edge Store
──────────────────────────────────────────────────────────

CREATE TABLE entity_relation (
  id               UUID         PRIMARY KEY DEFAULT uuid_generate_v4(),
  source_entity_id   UUID         NOT NULL REFERENCES canonical_entity(id) ON
DELETE CASCADE,
  target_entity_id   UUID         NOT NULL REFERENCES canonical_entity(id) ON
DELETE CASCADE,
  predicate          VARCHAR(100) NOT NULL,
  valid_from         TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
  valid_to           TIMESTAMPTZ,                  -- NULL = currently active
  assertion_event_id UUID         REFERENCES system_event(id) ON DELETE SET
NULL,
  emotional_context  TEXT,
  confidence_score   NUMERIC(4,3) NOT NULL DEFAULT 0.800,
  CHECK (source_entity_id <> target_entity_id)
);
CREATE INDEX idx_relation_traverse ON entity_relation (source_entity_id,
predicate, valid_to);
CREATE INDEX idx_relation_active   ON entity_relation (source_entity_id) WHERE
valid_to IS NULL;
CREATE INDEX idx_relation_target   ON entity_relation (target_entity_id) WHERE
valid_to IS NULL;

-- ─── 6. Task Projection
──────────────────────────────────────────────────────────
──────────────────────────────────

CREATE TABLE task_projection (
  id                 UUID       PRIMARY KEY DEFAULT uuid_generate_v4(),
  originating_event_id UUID       REFERENCES system_event(id) ON DELETE SET
NULL,
  associated_project   UUID       REFERENCES canonical_entity(id) ON DELETE
SET NULL,
  assigned_person      UUID       REFERENCES canonical_entity(id) ON DELETE
SET NULL,
  title              VARCHAR(512) NOT NULL,
  task_key             TEXT,
  current_status       task_state NOT NULL DEFAULT 'PENDING',
  estimated_effort     SMALLINT   NOT NULL DEFAULT 2 CHECK (estimated_effort
BETWEEN 1 AND 5),
  due_at             TIMESTAMPTZ,
  completed_at         TIMESTAMPTZ,
  created_at         TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  updated_at          TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_task_open    ON task_projection (current_status, created_at
DESC)
  WHERE current_status NOT IN ('COMPLETED','ABANDONED');
CREATE INDEX idx_task_project ON task_projection (associated_project);
CREATE INDEX idx_task_person  ON task_projection (assigned_person);

-- ─── 7. Semantic Vector Store
──────────────────────────────────────────────────────────────────────
──────────────────────

CREATE TABLE semantic_chunk (
  id                  UUID        PRIMARY KEY DEFAULT uuid_generate_v4(),
  originating_event_id UUID       NOT NULL REFERENCES system_event(id) ON
DELETE CASCADE,
  associated_entity_id UUID       REFERENCES canonical_entity(id) ON DELETE
SET NULL,
  chunk_text          TEXT        NOT NULL,
  embedding           vector(384),            -- NULL until browser fills it
  embedding_model     VARCHAR(100),           -- tracks model version
  generation_timestamp TIMESTAMPTZ NOT NULL DEFAULT
CURRENT_TIMESTAMP,
  attention_score     NUMERIC(6,4) NOT NULL DEFAULT 0.5000,
  recall_count        INT         NOT NULL DEFAULT 0
);
CREATE INDEX idx_hnsw           ON semantic_chunk
  USING hnsw (embedding vector_cosine_ops) WHERE embedding IS NOT NULL;
CREATE INDEX idx_chunk_null_emb  ON semantic_chunk (id) WHERE embedding
IS NULL;
CREATE INDEX idx_chunk_text_fts  ON semantic_chunk USING gin
(to_tsvector('english', chunk_text));
CREATE INDEX idx_chunk_entity    ON semantic_chunk (associated_entity_id);

-- ─── 8. Epistemic Claims (v1 — simplified, decay inactive)
──────────────────────────────────────

CREATE TABLE epistemic_claim (
  id                UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
  claim_text        TEXT          NOT NULL,
  claim_type        epistemic_type NOT NULL,
  trust_level       trust_level   NOT NULL,
  subject_entity_id  UUID         REFERENCES canonical_entity(id) ON DELETE
SET NULL,
  source_event_id   UUID          REFERENCES system_event(id) ON DELETE SET
NULL,
  extracted_by_model VARCHAR(100)  NOT NULL,
  confidence_score  NUMERIC(4,3)  CHECK (confidence_score BETWEEN 0 AND
1),
  valid_from        TIMESTAMPTZ   NOT NULL DEFAULT CURRENT_TIMESTAMP,
  valid_to          TIMESTAMPTZ,            -- NULL = still believed true
  contradiction_flag BOOLEAN      NOT NULL DEFAULT FALSE,
```

```
  contradicts_id     UUID          REFERENCES epistemic_claim(id) ON DELETE SET
NULL,
  created_at         TIMESTAMPTZ    NOT NULL DEFAULT CURRENT_TIMESTAMP
  -- TODO Phase 4: Add decay_rate, last_verified_at, verification_count
);
CREATE INDEX idx_claim_entity       ON epistemic_claim (subject_entity_id,
claim_type, valid_to);
CREATE INDEX idx_claim_contradiction ON epistemic_claim (contradiction_flag)
  WHERE contradiction_flag = TRUE;
CREATE INDEX idx_claim_active       ON epistemic_claim (valid_from DESC)
  WHERE valid_to IS NULL;

-- ─── 9. Goal Conflict Table
──────────────────────────────────────────────────────────────────────────────
──────────────────────────────

CREATE TABLE goal_conflict (
  id              UUID         PRIMARY KEY DEFAULT uuid_generate_v4(),
  goal_a_id       UUID         NOT NULL REFERENCES canonical_entity(id) ON
DELETE CASCADE,
  goal_b_id       UUID         NOT NULL REFERENCES canonical_entity(id) ON
DELETE CASCADE,
  conflict_type    VARCHAR(20)  NOT NULL CHECK (conflict_type IN
('VALUE','RESOURCE','TIME')),
  conflict_summary TEXT         NOT NULL,
  severity         NUMERIC(4,3) CHECK (severity BETWEEN 0 AND 1),
  detected_at      TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
  resolved_at      TIMESTAMPTZ,
  dismissed        BOOLEAN      NOT NULL DEFAULT FALSE
);
CREATE INDEX idx_conflict_open ON goal_conflict (resolved_at)
  WHERE resolved_at IS NULL AND dismissed = FALSE;

-- ─── 10. LLM Response Cache
──────────────────────────────────────────────────────────────────────────────
──────────────────────────────

CREATE TABLE response_cache (
  id           UUID        PRIMARY KEY DEFAULT uuid_generate_v4(),
  prompt_hash   VARCHAR(64) UNIQUE NOT NULL,
  response_text TEXT        NOT NULL,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  expires_at    TIMESTAMPTZ NOT NULL DEFAULT (CURRENT_TIMESTAMP +
INTERVAL '7 days')
);
CREATE INDEX idx_cache_hash   ON response_cache (prompt_hash);
CREATE INDEX idx_cache_expire ON response_cache (expires_at);

-- ─── 11. Context Assembly Log (mandatory from day one)
──────────────────────────────────────────────

CREATE TABLE context_assembly_log (
  id                UUID        PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
  assembled_at       TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  user_query         TEXT        NOT NULL,
  query_hash         VARCHAR(64),
  chunks_retrieved   INT         NOT NULL DEFAULT 0,
  chunks_in_context  INT         NOT NULL DEFAULT 0,
  top_chunk_ids      UUID[],
  top_chunk_scores   NUMERIC[],
  total_tokens       INT,
  override_count     INT         NOT NULL DEFAULT 0,
  tasks_in_context   INT         NOT NULL DEFAULT 0,
  load_state         VARCHAR(20),
  cache_hit          BOOLEAN     NOT NULL DEFAULT FALSE
);
CREATE INDEX idx_ctx_log_time ON context_assembly_log (assembled_at DESC);

-- ─── 12. Extraction Log (mandatory from day one) ──────────────────────────

CREATE TABLE extraction_log (
  id             UUID        PRIMARY KEY DEFAULT uuid_generate_v4(),
  event_id       UUID        REFERENCES system_event(id) ON DELETE SET NULL,
  extracted_at   TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  model_used     VARCHAR(100) NOT NULL,
  payload_hash   VARCHAR(64),
  entities_found  INT        NOT NULL DEFAULT 0,
  tasks_found    INT         NOT NULL DEFAULT 0,
  relations_found INT        NOT NULL DEFAULT 0,
  success        BOOLEAN     NOT NULL,
  latency_ms     INT,
  error_message  TEXT
);
CREATE INDEX idx_extract_success ON extraction_log (success, extracted_at
DESC);
CREATE INDEX idx_extract_recent  ON extraction_log (extracted_at DESC);

-- ─── 13. Cognitive Health Log ─────────────────────────────────────────────
──────────────────────────

CREATE TABLE cognitive_health_log (
  id                UUID        PRIMARY KEY DEFAULT uuid_generate_v4(),
  computed_at       TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  overall_score     INT         CHECK (overall_score BETWEEN 0 AND 100),
  embedding_coverage  NUMERIC(5,2),
  retrieval_p95_ms    INT,
  entity_conflicts    INT,
  stale_claims        INT,
  contradiction_pct   NUMERIC(5,2),
  extraction_success  NUMERIC(5,2),
  graph_density       NUMERIC(5,2),
  raw_metrics         JSONB       NOT NULL DEFAULT '{}'
);
```

```
-- ─── 14. User Identity Snapshots
──────────────────────────────────────────────────────────────────────
CREATE TABLE user_identity_snapshot (
  id              UUID        PRIMARY KEY DEFAULT uuid_generate_v4(),
  snapshot_year   INT         NOT NULL,
  snapshot_date   DATE        NOT NULL,
  trigger_type    VARCHAR(50) CHECK (trigger_type IN
('ANNUAL_AUTO','USER_INITIATED','MAJOR_EVENT')),
  core_goals      JSONB,
  active_beliefs  JSONB,
  career_context  TEXT,
  life_phase      VARCHAR(100),
  ai_summary      TEXT,
  key_events      JSONB,
  belief_changes  JSONB,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- ─── 15. Cognitive Load View
──────────────────────────────────────────────────────────────────────
CREATE VIEW cognitive_load AS
SELECT
  COUNT(*)        FILTER (WHERE current_status IN ('PENDING','ACTIVE'))  AS
open_tasks,
  COUNT(*)        FILTER (WHERE current_status = 'BLOCKED')              AS
blocked_tasks,
  COUNT(*)        FILTER (WHERE current_status = 'ACTIVE')               AS
active_tasks,
  COUNT(DISTINCT associated_project)                                     AS active_projects,
  COALESCE(
    SUM(estimated_effort) FILTER (WHERE current_status IN
('PENDING','ACTIVE')), 0
  )                                                                      AS raw_load,
  CASE
    WHEN COALESCE(SUM(estimated_effort) FILTER (WHERE current_status IN
('PENDING','ACTIVE')), 0) < 15
      THEN 'COMFORTABLE'
    WHEN COALESCE(SUM(estimated_effort) FILTER (WHERE current_status IN
('PENDING','ACTIVE')), 0) < 25
      THEN 'FULL'
    WHEN COALESCE(SUM(estimated_effort) FILTER (WHERE current_status IN
('PENDING','ACTIVE')), 0) < 35
      THEN 'OVERLOADED'
    ELSE 'CRITICAL'
  END                                                                    AS load_state
FROM task_projection
WHERE current_status NOT IN ('COMPLETED', 'ABANDONED');
```

```
-- ——— 16. Hybrid Retrieval Function
————————————————————————————————————————————————————————————————————
CREATE OR REPLACE FUNCTION hybrid_retrieval(
  query_text   TEXT,
  query_vec    vector(384),
  decay_lambda NUMERIC DEFAULT 0.00000005
)
RETURNS TABLE (chunk_id UUID, content TEXT, score NUMERIC) AS $$
WITH
  semantic AS (
    SELECT
      id,
      chunk_text,
      (1 - (embedding <=> query_vec))                                    AS cos_sim,
      EXTRACT(EPOCH FROM (CURRENT_TIMESTAMP - generation_timestamp))
AS age_s
    FROM semantic_chunk
    WHERE embedding IS NOT NULL
    ORDER BY embedding <=> query_vec
    LIMIT 25
  ),
  keyword AS (
    SELECT
      id,
      ts_rank_cd(
        to_tsvector('english', chunk_text),
        plainto_tsquery('english', query_text)
      ) AS kw
    FROM semantic_chunk
    WHERE to_tsvector('english', chunk_text) @@ plainto_tsquery('english',
query_text)
    LIMIT 25
  )
SELECT
  s.id,
  s.chunk_text,
  CAST(
    ((COALESCE(k.kw, 0.0) * 0.4) + (s.cos_sim * 0.6))
    * EXP(-decay_lambda * s.age_s)
  AS NUMERIC) AS final_score
FROM semantic   s
LEFT JOIN keyword k ON s.id = k.id
ORDER BY 3 DESC
LIMIT 15;
$$ LANGUAGE sql STABLE;

-- ——— 17. Truth Arbitration Function
————————————————————————————————————————————————————————————————————
```

```
_____
CREATE OR REPLACE FUNCTION arbitrate_truth(
  p_entity    UUID,
  p_predicate VARCHAR,
  p_at        TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
)
RETURNS JSONB AS $$
WITH active AS (
  SELECT
    ec.id,
    ec.claim_text,
    ec.confidence_score,
    ec.trust_level,
    (
      (ec.confidence_score * 0.6) +
      (GREATEST(0,
        1 - EXTRACT(EPOCH FROM (p_at - ec.valid_from)) / (730.0 * 86400)
      ) * 0.4)
    ) AS composite
  FROM epistemic_claim   ec
  JOIN entity_relation   er ON ec.source_event_id = er.assertion_event_id
  WHERE
    er.source_entity_id = p_entity
    AND er.predicate    = p_predicate
    AND ec.valid_from  <= p_at
    AND (ec.valid_to IS NULL OR ec.valid_to > p_at)
  ORDER BY composite DESC
),
ranked AS (
  SELECT
    *,
    ROW_NUMBER() OVER (ORDER BY composite DESC)              AS rnk,
    composite - LEAD(composite) OVER (ORDER BY composite DESC) AS gap
  FROM active
)
SELECT CASE
  WHEN (SELECT COUNT(*) FROM ranked) = 0 THEN
    jsonb_build_object('state', 'UNKNOWN')
  WHEN (SELECT COUNT(*) FROM ranked) = 1 THEN
    jsonb_build_object(
      'state',      'DEFINITIVE',
      'claim',      (SELECT claim_text  FROM ranked WHERE rnk = 1),
      'confidence', (SELECT composite   FROM ranked WHERE rnk = 1)
    )
  WHEN (SELECT gap FROM ranked WHERE rnk = 1) > 0.20 THEN
    jsonb_build_object(
      'state',      'DEFINITIVE',
      'claim',      (SELECT claim_text  FROM ranked WHERE rnk = 1),
      'confidence', (SELECT composite   FROM ranked WHERE rnk = 1)
    )
```

```
  ELSE
    jsonb_build_object(
      'state',     'CONTESTED',
      'primary',   (SELECT claim_text FROM ranked WHERE rnk = 1),
      'competing', (SELECT claim_text FROM ranked WHERE rnk = 2),
      'note',      'Low confidence gap — both injected as competing hypotheses'
    )
END;
$$ LANGUAGE sql STABLE;

-- ─── 18. Scheduled Jobs ──────────────────────────────────────────────────────

-- Anti-pause ping (keeps Supabase free tier alive)
SELECT cron.schedule(
  'anti-pause-ping',
  '0 3 * * *',
  $$SELECT 1 FROM system_event LIMIT 1$$
);

-- Weekly cognitive health snapshot (Monday 4AM)
SELECT cron.schedule(
  'weekly-health-check',
  '0 4 * * 1',
  $$
  INSERT INTO cognitive_health_log (
    overall_score,
    embedding_coverage,
    entity_conflicts,
    extraction_success,
    raw_metrics
  )
  SELECT
    CASE
      WHEN emb_pct  > 90 AND ext_pct > 95 THEN 88
      WHEN emb_pct  > 70 OR  ext_pct > 85 THEN 65
      ELSE 40
    END,
    emb_pct,
    id_flags,
    ext_pct,
    jsonb_build_object(
      'embedding_pct',    emb_pct,
      'identity_flags',   id_flags,
      'extraction_pct',   ext_pct,
      'total_events',     total_ev,
      'total_entities',   total_en
    )
  FROM (
```

```
    SELECT
      ROUND(100.0 * COUNT(*) FILTER (WHERE embedding IS NOT NULL)
        / NULLIF(COUNT(*), 0), 2)                    AS emb_pct,
      (SELECT COUNT(*) FROM identity_flag WHERE resolved_at IS NULL) AS
id_flags,
      ROUND(100.0 * COUNT(*) FILTER (WHERE el.success = TRUE)
        / NULLIF(COUNT(*), 0), 2)                    AS ext_pct,
      (SELECT COUNT(*) FROM system_event)                AS total_ev,
      (SELECT COUNT(*) FROM canonical_entity)            AS total_en
    FROM semantic_chunk sc
    CROSS JOIN (
      SELECT success FROM extraction_log
      WHERE extracted_at > NOW() - INTERVAL '7 days'
    ) el
    LIMIT 1
  ) metrics
  $$
);

-- Monthly cache cleanup (1st of month, 2AM)
SELECT cron.schedule(
  'monthly-cache-clean',
  '0 2 1 * *',
  $$DELETE FROM response_cache WHERE expires_at < NOW()$$
);
-- ── 19. Trigram-based entity similarity (added post-launch fix)
──────────────────────────────
-- Catches spelling variants/typos. Does NOT catch nicknames (Mike/Michael) —
-- those are not character-similar and remain a Category C research-risk item.
CREATE OR REPLACE FUNCTION find_similar_entities(
  p_name       TEXT,
  p_exclude_id UUID
)
RETURNS TABLE (id UUID, canonical_name VARCHAR, sim REAL) AS $$
  SELECT id, canonical_name, similarity(canonical_name, p_name) AS sim
  FROM canonical_entity
  WHERE id <> p_exclude_id
    AND similarity(canonical_name, p_name) > 0.3
  ORDER BY sim DESC
  LIMIT 5;
$$ LANGUAGE sql STABLE;
-- ── End of schema
──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────
-- To verify: SELECT table_name FROM information_schema.tables
--            WHERE table_schema = 'public' ORDER BY table_name;


──────────────────────────────────────────────────────────────────────────────
──────────────────────────
```

FILE: supabase/seed-lifelong-core.sql

```
-- Lifelong AI core readiness seed.
-- Covers stable family facts, relationships, tasks, reminders, work context,
-- decisions, and stale-state behavior.

INSERT INTO system_event (source_tab, payload, metadata, recorded_at) VALUES
('THOUGHTS', 'My wife is Priya. We were married on 27 January 2016. We have two children. Vanya is my daughter and was born on 9 March 2018. Vaidik is my son and was born on 16 March 2021.', '{"source":"lifelong_seed","domain":"family"}', NOW() - INTERVAL '10 days'),
('THOUGHTS', 'During summer vacation I need to get Vaidik and Vanya homework completed before school reopens. This is important for this week.', '{"source":"lifelong_seed","domain":"family_tasks"}', NOW() - INTERVAL '9 days'),
('THOUGHTS', 'Next Thursday we are planning a 4 day family vacation. I need to finalize the destination, book hotels, check travel options, and make sure the children have everything packed.', '{"source":"lifelong_seed","domain":"travel"}', NOW() - INTERVAL '8 days'),
('THOUGHTS', 'At work next week there is a shutdown. There may still be a sales update required. I need to check with the sales team and close any pending company activity before the shutdown.', '{"source":"lifelong_seed","domain":"work"}', NOW() - INTERVAL '7 days'),
('THOUGHTS', 'I completed the first draft of my lifelong AI app. The goal is to use it as my second brain: dump thoughts, automatically derive tasks, remember relationships, and remind me when required.', '{"source":"lifelong_seed","domain":"project"}', NOW() - INTERVAL '6 days'),
('THOUGHTS', 'Decision: for the personal assistant project, reliability of memory and traceability are more important than a fancy chat UI. I want every answer to be source-backed when possible.', '{"source":"lifelong_seed","domain":"decision"}', NOW() - INTERVAL '5 days'),
('THOUGHTS', 'I prefer weekly planning on Sunday evening. The assistant should help me identify open loops, family commitments, important work, and overloaded weeks.', '{"source":"lifelong_seed","domain":"preference"}', NOW() - INTERVAL '4 days'),
('THOUGHTS', 'Reminder: ask Priya tonight about preferred destination options for the 4 day family vacation.', '{"source":"lifelong_seed","domain":"reminder"}', NOW() - INTERVAL '3 days'),
('TASKS', '{"action":"TASK_STATUS_UPDATE","task_title":"Get Vaidik and Vanya homework completed","new_status":"COMPLETED"}', '{"source":"lifelong_seed","mutation":true}', NOW() - INTERVAL '1 day'),
('CHAT', 'What are my active family commitments?', '{"source":"lifelong_seed","role":"user"}', NOW() - INTERVAL '12 hours'),
('CHAT', 'Your active family commitments are to finalize the 4 day vacation destination, book hotels, check travel options, and ask Priya about destination preferences. The children homework task has been completed.', '{"source":"lifelong_seed","role":"assistant"}', NOW() - INTERVAL '12 hours');

INSERT INTO evaluation_case (suite, question, expected_answer, expected_memory, category) VALUES
```

```
('core', 'Who is Priya?', 'Priya is my wife.', ARRAY['Priya', 'wife'], 'profile_fact'),
('core', 'When is Vaidik birthday?', 'Vaidik was born on 16 March 2021.',
ARRAY['Vaidik', '16 March 2021'], 'profile_fact'),
('core', 'When is Vanya birthday?', 'Vanya was born on 9 March 2018.',
ARRAY['Vanya', '9 March 2018'], 'profile_fact'),
('core', 'What is pending for the family vacation?', 'Finalize destination, book hotels,
check travel options, and ask Priya about destination preferences.', ARRAY['family
vacation', 'book hotels'], 'commitment'),
('core', 'What is my preference for weekly planning?', 'Weekly planning should
happen on Sunday evening.', ARRAY['Sunday evening', 'weekly planning'],
'preference'),
('core', 'What did I decide about the assistant project?', 'Reliability of memory and
traceability matter more than a fancy chat UI.', ARRAY['reliability', 'traceability'],
'decision');

SELECT
  source_tab,
  COUNT(*) AS count
FROM system_event
WHERE metadata->>'source' = 'lifelong_seed'
GROUP BY source_tab
ORDER BY source_tab;
```

FILE: supabase/seed-lifelong-stress.sql

```
-- Generated lifelong stress seed v2.
-- Controlled coverage: durable facts, a few real tasks, and non-actionable history.

DELETE FROM system_event
WHERE metadata->>'source' IN ('stress_seed', 'stress_seed_v2');

DELETE FROM evaluation_case
WHERE suite = 'stress';

INSERT INTO system_event (source_tab, payload, metadata, recorded_at) VALUES
('THOUGHTS', 'My wife is Priya. We married on 27 January 2016. Priya is my family
decision partner.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '60.00 days'),
('THOUGHTS', 'Vaidik is my son. Vaidik was born on 16 March 2021.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '59.65 days'),
('THOUGHTS', 'Vanya is my daughter. Vanya was born on 9 March 2018.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '59.30 days'),
('THOUGHTS', 'Priya prefers calm travel planning, clean hotel options, and not
deciding family trips in a rush.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL
'58.95 days'),
('THOUGHTS', 'I prefer Sunday evening weekly planning because it gives me a
clean view of family, work, and personal commitments.',
```

'{"source":"stress_seed_v2"}', NOW() - INTERVAL '58.60 days'),
('THOUGHTS', 'Decision: for the Lifelong AI assistant, memory correctness and source traceability matter more than a fancy chat UI.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '58.25 days'),
('THOUGHTS', 'Decision: the assistant must never pretend it changed tasks unless the database actually changed.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '57.90 days'),
('THOUGHTS', 'Goal: build the personal assistant as a lifelong second brain, not a short-term chatbot.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '57.55 days'),
('THOUGHTS', 'For the family vacation, I need to finalize the destination and compare hotel options.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '57.20 days'),
('THOUGHTS', 'I should ask Priya about destination options for the family vacation tonight.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '56.85 days'),
('THOUGHTS', 'The kids homework needs to be completed before school reopens.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '56.50 days'),
('THOUGHTS', 'Company shutdown is next week. I need to check with Rohit whether the sales update is still required.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '56.15 days'),
('THOUGHTS', 'Anita from finance asked for revised wealth dashboard projections before Friday.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '55.80 days'),
('THOUGHTS', 'Sanjay will coordinate shutdown handover. I should send him the open items list.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '55.45 days'),
('THOUGHTS', 'Meera reminded us that school reopens soon and homework should be submitted on the first day.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '55.10 days'),
('THOUGHTS', 'Knowledge: Vaidik needs maths and handwriting practice during school reopening prep.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '54.75 days'),
('THOUGHTS', 'Knowledge: Vanya needs reading practice and art homework during school reopening prep.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '54.40 days'),
('THOUGHTS', 'Preference: I want fewer, clearer tasks instead of duplicated task noise.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '54.05 days'),
('THOUGHTS', 'Preference: I like direct answers that cite what memory they used when the answer matters.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '53.70 days'),
('THOUGHTS', 'Routine: Sunday evening is the best time for a weekly family and work planning review.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '53.35 days'),
('THOUGHTS', 'Relationship: Rohit is the sales lead I should contact for sales-update questions.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '53.00 days'),
('THOUGHTS', 'Relationship: Anita is the finance reviewer for wealth dashboard projections.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '52.65 days'),
('THOUGHTS', 'Relationship: Meera is the school coordinator who gives homework and reopening updates.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '52.30 days'),
('THOUGHTS', 'Relationship: Sanjay is the operations manager for shutdown handover.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '51.95 days'),
('THOUGHTS', 'Reminder: check hotel rates tomorrow morning before prices

change.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '51.60 days'),
('THOUGHTS', 'Archive note 1: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '51.25 days'),
('THOUGHTS', 'Archive note 2: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '50.90 days'),
('THOUGHTS', 'Archive note 3: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '50.55 days'),
('THOUGHTS', 'Archive note 4: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '50.20 days'),
('THOUGHTS', 'Archive note 5: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '49.85 days'),
('THOUGHTS', 'Archive note 6: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '49.50 days'),
('THOUGHTS', 'Archive note 7: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '49.15 days'),
('THOUGHTS', 'Archive note 8: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '48.80 days'),
('THOUGHTS', 'Archive note 9: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '48.45 days'),
('THOUGHTS', 'Archive note 10: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '48.10 days'),
('THOUGHTS', 'Archive note 11: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '47.75 days'),
('THOUGHTS', 'Archive note 12: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '47.40 days'),
('THOUGHTS', 'Archive note 13: I noticed a pattern around assistant reliability. This

is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '47.05 days'),
('THOUGHTS', 'Archive note 14: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '46.70 days'),
('THOUGHTS', 'Archive note 15: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '46.35 days'),
('THOUGHTS', 'Archive note 16: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '46.00 days'),
('THOUGHTS', 'Archive note 17: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '45.65 days'),
('THOUGHTS', 'Archive note 18: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '45.30 days'),
('THOUGHTS', 'Archive note 19: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '44.95 days'),
('THOUGHTS', 'Archive note 20: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '44.60 days'),
('THOUGHTS', 'Archive note 21: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '44.25 days'),
('THOUGHTS', 'Archive note 22: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '43.90 days'),
('THOUGHTS', 'Archive note 23: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '43.55 days'),
('THOUGHTS', 'Archive note 24: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '43.20 days'),
('THOUGHTS', 'Archive note 25: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',

'{"source":"stress_seed_v2"}', NOW() - INTERVAL '42.85 days'),
('THOUGHTS', 'Archive note 26: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '42.50 days'),
('THOUGHTS', 'Archive note 27: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '42.15 days'),
('THOUGHTS', 'Archive note 28: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '41.80 days'),
('THOUGHTS', 'Archive note 29: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '41.45 days'),
('THOUGHTS', 'Archive note 30: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '41.10 days'),
('THOUGHTS', 'Archive note 31: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '40.75 days'),
('THOUGHTS', 'Archive note 32: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '40.40 days'),
('THOUGHTS', 'Archive note 33: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '40.05 days'),
('THOUGHTS', 'Archive note 34: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '39.70 days'),
('THOUGHTS', 'Archive note 35: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '39.35 days'),
('THOUGHTS', 'Archive note 36: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '39.00 days'),
('THOUGHTS', 'Archive note 37: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '38.65 days'),
('THOUGHTS', 'Archive note 38: I noticed a pattern around memory traceability. This

is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '38.30 days'),
('THOUGHTS', 'Archive note 39: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '37.95 days'),
('THOUGHTS', 'Archive note 40: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '37.60 days'),
('THOUGHTS', 'Archive note 41: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '37.25 days'),
('THOUGHTS', 'Archive note 42: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '36.90 days'),
('THOUGHTS', 'Archive note 43: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '36.55 days'),
('THOUGHTS', 'Archive note 44: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '36.20 days'),
('THOUGHTS', 'Archive note 45: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '35.85 days'),
('THOUGHTS', 'Archive note 46: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '35.50 days'),
('THOUGHTS', 'Archive note 47: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '35.15 days'),
('THOUGHTS', 'Archive note 48: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '34.80 days'),
('THOUGHTS', 'Archive note 49: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '34.45 days'),
('THOUGHTS', 'Archive note 50: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',

NOW() - INTERVAL '34.10 days'),
('THOUGHTS', 'Archive note 51: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '33.75 days'),
('THOUGHTS', 'Archive note 52: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '33.40 days'),
('THOUGHTS', 'Archive note 53: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '33.05 days'),
('THOUGHTS', 'Archive note 54: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '32.70 days'),
('THOUGHTS', 'Archive note 55: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '32.35 days'),
('THOUGHTS', 'Archive note 56: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '32.00 days'),
('THOUGHTS', 'Archive note 57: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '31.65 days'),
('THOUGHTS', 'Archive note 58: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '31.30 days'),
('THOUGHTS', 'Archive note 59: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '30.95 days'),
('THOUGHTS', 'Archive note 60: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}',
NOW() - INTERVAL '30.60 days'),
('THOUGHTS', 'Archive note 61: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '30.25 days'),
('THOUGHTS', 'Archive note 62: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '29.90 days'),
('THOUGHTS', 'Archive note 63: I noticed a pattern around assistant reliability. This

is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '29.55 days'),
('THOUGHTS', 'Archive note 64: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '29.20 days'),
('THOUGHTS', 'Archive note 65: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '28.85 days'),
('THOUGHTS', 'Archive note 66: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '28.50 days'),
('THOUGHTS', 'Archive note 67: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '28.15 days'),
('THOUGHTS', 'Archive note 68: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '27.80 days'),
('THOUGHTS', 'Archive note 69: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '27.45 days'),
('THOUGHTS', 'Archive note 70: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '27.10 days'),
('THOUGHTS', 'Archive note 71: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '26.75 days'),
('THOUGHTS', 'Archive note 72: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '26.40 days'),
('THOUGHTS', 'Archive note 73: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '26.05 days'),
('THOUGHTS', 'Archive note 74: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '25.70 days'),
('THOUGHTS', 'Archive note 75: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',

'{"source":"stress_seed_v2"}', NOW() - INTERVAL '25.35 days'),
('THOUGHTS', 'Archive note 76: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '25.00 days'),
('THOUGHTS', 'Archive note 77: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '24.65 days'),
('THOUGHTS', 'Archive note 78: I noticed a pattern around memory traceability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '24.30 days'),
('THOUGHTS', 'Archive note 79: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '23.95 days'),
('THOUGHTS', 'Archive note 80: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '23.60 days'),
('THOUGHTS', 'Archive note 81: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '23.25 days'),
('THOUGHTS', 'Archive note 82: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '22.90 days'),
('THOUGHTS', 'Archive note 83: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '22.55 days'),
('THOUGHTS', 'Archive note 84: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '22.20 days'),
('THOUGHTS', 'Archive note 85: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '21.85 days'),
('THOUGHTS', 'Archive note 86: I noticed a pattern around travel decision making. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '21.50 days'),
('THOUGHTS', 'Archive note 87: I noticed a pattern around weekly planning habit. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '21.15 days'),
('THOUGHTS', 'Archive note 88: I noticed a pattern around memory traceability. This

is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '20.80 days'),
('THOUGHTS', 'Archive note 89: I noticed a pattern around task hygiene. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '20.45 days'),
('THOUGHTS', 'Archive note 90: I noticed a pattern around calm prioritization. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.', '{"source":"stress_seed_v2"}', NOW() - INTERVAL '20.10 days'),
('THOUGHTS', 'Archive note 91: I noticed a pattern around family planning style. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '19.75 days'),
('THOUGHTS', 'Archive note 92: I noticed a pattern around school reopening rhythm. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '19.40 days'),
('THOUGHTS', 'Archive note 93: I noticed a pattern around assistant reliability. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '19.05 days'),
('THOUGHTS', 'Archive note 94: I noticed a pattern around work shutdown coordination. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '18.70 days'),
('THOUGHTS', 'Archive note 95: I noticed a pattern around wealth dashboard clarity. This is background context for future recall, not a new task. The useful signal is to preserve the context without adding duplicate commitments.',
'{"source":"stress_seed_v2"}', NOW() - INTERVAL '18.35 days');

INSERT INTO evaluation_case (suite, question, expected_answer, expected_memory, category) VALUES
('stress', 'Who is Priya?', 'Priya is my wife and family decision partner.', ARRAY['Priya','wife'], 'profile_fact'),
('stress', 'When is Vaidik birthday?', 'Vaidik was born on 16 March 2021.', ARRAY['Vaidik','16 March 2021'], 'profile_fact'),
('stress', 'When is Vanya birthday?', 'Vanya was born on 9 March 2018.', ARRAY['Vanya','9 March 2018'], 'profile_fact'),
('stress', 'What is pending for the family vacation?', 'Finalize the destination, compare hotel options, and ask Priya about destination options.', ARRAY['family vacation','hotel','Priya'], 'commitment'),
('stress', 'What did I decide about the assistant project?', 'Memory correctness and source traceability matter more than a fancy chat UI.', ARRAY['memory correctness','traceability'], 'decision'),
('stress', 'Who should I check with for sales update?', 'Check with Rohit about whether the sales update is still required.', ARRAY['Rohit','sales update'], 'work'),
('stress', 'What should never happen when I mark a task complete in chat?', 'The assistant should never pretend it changed tasks unless the database actually

changed.', ARRAY['database','changed'], 'trust');

SELECT
  'stress_seed_loaded' AS status,
  COUNT(*) AS events
FROM system_event
WHERE metadata->>'source' = 'stress_seed_v2';

FILE: supabase/seed-synthetic.sql

--

-- Lifelong AI — Synthetic Seed Dataset (50 events)
-- Run in Supabase SQL Editor to test the system before real usage.
--
-- To remove all seed data after testing:
-- DELETE FROM system_event WHERE metadata->>'source' = 'seed';
--

-- ─── Thoughts (35 events)

INSERT INTO system_event (source_tab, payload, metadata, recorded_at) VALUES
('THOUGHTS', 'Had a good call with Mike today about the Alpha project. He seems stressed about the Q3 deadline. Need to follow up with him next week.', '{"source":"seed"}', NOW() - INTERVAL '30 days'),
('THOUGHTS', 'Met with Sarah Chen from the design team. She showed me the new mockups for the dashboard. Really impressive work. Should loop her into the Alpha project.', '{"source":"seed"}', NOW() - INTERVAL '29 days'),
('THOUGHTS', 'I need to review the project proposal for Beta initiative before Friday. Michael Johnson from finance wants the numbers by end of week.', '{"source":"seed"}', NOW() - INTERVAL '28 days'),
('THOUGHTS', 'Goal for this year: increase income by 20% while keeping work hours under 45 per week. Feels hard to balance but important.', '{"source":"seed"}', NOW() - INTERVAL '27 days'),
('THOUGHTS', 'Call Mike Johnson to discuss Alpha project budget overrun. He mentioned costs are 15% over. Need to present options to leadership.', '{"source":"seed"}', NOW() - INTERVAL '26 days'),
('THOUGHTS', 'Had coffee with my manager. She thinks I should take on the Omega project but I already have Alpha and Beta running. Feeling stretched.', '{"source":"seed"}', NOW() - INTERVAL '25 days'),

('THOUGHTS', 'Thinking about starting a side business doing consulting. But I already feel overloaded. Maybe next quarter when Alpha wraps.', '{"source":"seed"}', NOW() - INTERVAL '24 days'),
('THOUGHTS', 'Sarah (the intern, different from Sarah Chen) submitted her first PR today. Needs some guidance on code review process.', '{"source":"seed"}', NOW() - INTERVAL '23 days'),
('THOUGHTS', 'Need to book flights for the company offsite in November. Deadline is next Monday or we lose the early bird rate.', '{"source":"seed"}', NOW() - INTERVAL '22 days'),
('THOUGHTS', 'Had a disagreement with Mike about the Alpha timeline. He wants to push to Q4 but client expects Q3 delivery. Tense conversation.', '{"source":"seed"}', NOW() - INTERVAL '21 days'),
('THOUGHTS', 'Finished the Beta proposal draft. Sent to Michael J for finance review. Waiting on his sign-off before I can move forward.', '{"source":"seed"}', NOW() - INTERVAL '20 days'),
('THOUGHTS', 'Good news: Alpha client extended the deadline by 3 weeks. This takes a lot of pressure off Mike and the team.', '{"source":"seed"}', NOW() - INTERVAL '19 days'),
('THOUGHTS', 'I believe remote work is best for deep focus work. Office is too distracting. Will push for a hybrid policy renewal.', '{"source":"seed"}', NOW() - INTERVAL '18 days'),
('THOUGHTS', 'Goal update: I want to spend more time with family. The consulting side business idea is probably not the right time.', '{"source":"seed"}', NOW() - INTERVAL '17 days'),
('THOUGHTS', 'Review Sarah Chens design specs for the new dashboard feature. She needs feedback by Thursday for the client presentation.', '{"source":"seed"}', NOW() - INTERVAL '16 days'),
('THOUGHTS', 'Mike is now officially leading the Alpha project as of today. He replaces David who moved to the Omega team.', '{"source":"seed"}', NOW() - INTERVAL '15 days'),
('THOUGHTS', 'Need to update the Q3 budget spreadsheet. Finance team (Michael Johnson) is asking for updated projections by end of month.', '{"source":"seed"}', NOW() - INTERVAL '14 days'),
('THOUGHTS', 'The intern Sarah completed her onboarding. She will be working on the Beta project documentation starting next week.', '{"source":"seed"}', NOW() - INTERVAL '13 days'),
('THOUGHTS', 'Interesting: I am starting to think in-person collaboration is actually better for creative work. Maybe my remote work belief was too strong.', '{"source":"seed"}', NOW() - INTERVAL '12 days'),
('THOUGHTS', 'Alpha project milestone 1 is complete. Team did great work. Need to send a status update to the client and schedule the milestone 2 kickoff.', '{"source":"seed"}', NOW() - INTERVAL '11 days'),
('THOUGHTS', 'Call with my manager tomorrow about the Omega project offer. I need to prepare my case for why I cannot take it on right now.', '{"source":"seed"}', NOW() - INTERVAL '10 days'),
('THOUGHTS', 'Decided to decline the Omega project. Too much on my plate with Alpha and Beta. Manager understood but wants revisit in Q1.', '{"source":"seed"}', NOW() - INTERVAL '9 days'),
('THOUGHTS', 'Set up weekly 1-1 with Mike. Every Tuesday 2pm. Good way to keep Alpha on track now that he is leading it.', '{"source":"seed"}', NOW() - INTERVAL '8

```
days'),
('THOUGHTS', 'Sarah Chen delivered the final designs. Everything looks great. Ready to hand off to development team next sprint.', '{"source":"seed"}', NOW() - INTERVAL '7 days'),
('THOUGHTS', 'Need to prepare performance review for the team. Deadine is end of next week. Three direct reports: Mike, Sarah, and David.', '{"source":"seed"}', NOW() - INTERVAL '6 days'),
('THOUGHTS', 'Beta project kicked off today. Michael Johnson approved the budget. First sprint planning is Friday morning.', '{"source":"seed"}', NOW() - INTERVAL '5 days'),
('THOUGHTS', 'Flight booking reminder: company offsite flights need to be booked by tomorrow or we lose the group rate.', '{"source":"seed"}', NOW() - INTERVAL '4 days'),
('THOUGHTS', 'Great feedback from Alpha client on milestone 1. They love the new dashboard. Milestone 2 scope looks manageable.', '{"source":"seed"}', NOW() - INTERVAL '3 days'),
('THOUGHTS', 'Should probably set up a proper filing system for all these project documents. Getting hard to find things.', '{"source":"seed"}', NOW() - INTERVAL '2 days'),
('THOUGHTS', 'Mike asked about bringing on a contractor for Alpha milestone 3. Need to check with finance (Michael J) on budget availability.', '{"source":"seed"}', NOW() - INTERVAL '1 day'),
('THOUGHTS', 'Weekly planning: this week I need to finish performance reviews, prep for Beta sprint 1, and send Alpha status report.', '{"source":"seed"}', NOW() - INTERVAL '12 hours'),
('THOUGHTS', 'Had lunch with Sarah Chen. She mentioned she is considering leaving the company for a startup. Should tell my manager.', '{"source":"seed"}', NOW() - INTERVAL '6 hours'),
('THOUGHTS', 'Performance review docs are half done. Need to finish Mike and David sections by Friday. Sarah section is complete.', '{"source":"seed"}', NOW() - INTERVAL '3 hours'),
('THOUGHTS', 'Reminder to myself: book team dinner for end of Beta sprint 1. Good for morale. Maybe Italian place near the office.', '{"source":"seed"}', NOW() - INTERVAL '1 hour'),
('THOUGHTS', 'Just had a thought: I should ask Mike to document the Alpha architecture decisions before we hand off to maintenance team.', '{"source":"seed"}', NOW() - INTERVAL '10 minutes');

-- ——— Chat events (10 events — to test stale context injection) ———————————————

INSERT INTO system_event (source_tab, payload, metadata, recorded_at) VALUES
('CHAT', 'What are the main things I need to do this week?',
'{"source":"seed","role":"user"}', NOW() - INTERVAL '5 days'),
('CHAT', 'Based on your notes, you have three main priorities this week: (1) Book company offsite flights before the group rate expires tomorrow, (2) Complete performance reviews for Mike, Sarah, and David by end of next week, (3) Prepare for Beta project sprint 1 planning on Friday. You also have an ongoing Alpha status report to send to the client.', '{"source":"seed","role":"assistant"}', NOW() - INTERVAL '5 days'),
('CHAT', 'Has the Alpha milestone 1 been completed?',
```

'{"source":"seed","role":"user"}', NOW() - INTERVAL '4 days'),
('CHAT', 'Yes, based on your notes from 3 days ago, Alpha milestone 1 is complete. The client gave great feedback on the new dashboard. Milestone 2 is now in scope.',
'{"source":"seed","role":"assistant"}', NOW() - INTERVAL '4 days'),
('CHAT', 'What is the current situation with Sarah Chen?',
'{"source":"seed","role":"user"}', NOW() - INTERVAL '2 days'),
('CHAT', 'From your notes: Sarah Chen is the designer who delivered the final designs last week and they were well received. She is now working on the Beta project as well. You also had lunch with her today and she mentioned she may be considering leaving for a startup — you noted you should tell your manager about this.', '{"source":"seed","role":"assistant"}', NOW() - INTERVAL '2 days'),
('CHAT', 'What do I know about the Omega project?', '{"source":"seed","role":"user"}', NOW() - INTERVAL '1 day'),
('CHAT', 'You were offered the Omega project by your manager but declined it 9 days ago. Your reasoning was that Alpha and Beta are already at capacity. Your manager accepted the decision and wants to revisit in Q1. David moved to the Omega team around the same time.', '{"source":"seed","role":"assistant"}', NOW() - INTERVAL '1 day'),
('CHAT', 'Remind me to send the Alpha status report to the client',
'{"source":"seed","role":"user"}', NOW() - INTERVAL '2 hours'),
('CHAT', 'Got it. I have noted that you need to send the Alpha status report to the client. This is now part of your active commitments. Would you like me to help you draft the update based on what you have shared about the project?',
'{"source":"seed","role":"assistant"}', NOW() - INTERVAL '2 hours');

-- ── Verify ──

```
SELECT
  source_tab,
  COUNT(*) AS count
FROM system_event
WHERE metadata->>'source' = 'seed'
GROUP BY source_tab
ORDER BY source_tab;
```

FILE: tailwind.config.ts

```
import type { Config } from 'tailwindcss';

const config: Config = {
  content: [
    './app/**/*.{js,ts,jsx,tsx,mdx}',
    './components/**/*.{js,ts,jsx,tsx,mdx}',
    './lib/**/*.{js,ts,jsx,tsx,mdx}',
  ],
```

```
  theme: {
    extend: {
      fontFamily: {
        sans: [
          'system-ui',
          '-apple-system',
          'BlinkMacSystemFont',
          'Segoe UI',
          'sans-serif',
        ],
        mono: [
          'ui-monospace',
          'SFMono-Regular',
          'Menlo',
          'Monaco',
          'monospace',
        ],
      },
      colors: {
        surface: '#0c1222',
        border: '#1a2740',
      },
      keyframes: {
        'typing-dot': {
          '0%, 60%, 100%': { opacity: '0.3', transform: 'translateY(0)' },
          '30%':           { opacity: '1',   transform: 'translateY(-3px)' },
        },
        'fade-in': {
          '0%':   { opacity: '0', transform: 'translateY(4px)' },
          '100%': { opacity: '1', transform: 'translateY(0)' },
        },
      },
      animation: {
        'typing-dot':   'typing-dot 1.4s infinite',
        'typing-dot-2': 'typing-dot 1.4s infinite 0.2s',
        'typing-dot-3': 'typing-dot 1.4s infinite 0.4s',
        'fade-in':      'fade-in 0.15s ease-out',
      },
    },
  },
  plugins: [],
};

export default config;
```

FILE: tsconfig.json

_____


```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": {
      "@/*": ["./*"]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts"
  ],
  "exclude": ["node_modules", "supabase/functions"]
}
```


==========
END OF DUMP
==========